MANUAL DE FORMACIÓN · EDICIÓN 2026

De cero a ingeniero de software

Lo básico que de verdad se paga: desarrollo full-stack, ingeniería profesional y empleo junior en España

Perfil objetivo: desarrollador full-stack JavaScript/TypeScript

Punto de partida: nociones sueltas de programación

Incluye: plan de estudio de 12 semanas, 3 proyectos guiados de portfolio, preparación de entrevistas técnicas y datos salariales del mercado español

Preparado para Álvaro · Septiembre de 2026

Índice

PARTE 0 · EL MAPA 4

1 Qué es realmente un ingeniero de software 5
2 El mercado en España, septiembre de 2026 7
3 El stack que vas a aprender 10
4 El plan de estudio 11

PARTE 1 · LOS FUNDAMENTOS DEL OFICIO 13

5 La terminal 14
6 Git y GitHub 17
7 Cómo funciona la web por dentro 21
8 El entorno de trabajo profesional 24

PARTE 2 · EL LENGUAJE: JAVASCRIPT Y TYPESCRIPT 27

9 Fundamentos de JavaScript 28
10 Funciones, ámbito y closures 32
11 Objetos, arrays y transformación de datos 36
12 Asincronía 40
13 Módulos, errores y organización 43
14 TypeScript 45

PARTE 3 · FRONTEND: LA PARTE VISIBLE 49

15 HTML semántico y accesibilidad 50
16 CSS moderno 54
17 React: el modelo mental 58
18 React en un proyecto real 63

PARTE 4 · BACKEND: DONDE ESTÁ EL NEGOCIO 66

19 Node y Express: tu primera API 67
20 Bases de datos y SQL 72
21 Modelado de datos y Prisma 76
22 Autenticación y seguridad 79
23 Diseño de APIs 82

PARTE 5 · INGENIERÍA: LO QUE SEPARA A UN JUNIOR DE UN PROFESIONAL 84

24 Pruebas automatizadas 85
25 Código limpio y refactorización 89
26 Arquitectura y principios de diseño 92
27 Fundamentos de informática que sí se preguntan 95

PARTE 6 · PRODUCCIÓN: QUE FUNCIONE FUERA DE TU PORTÁTIL 98

28 Docker 99

29 Integración y despliegue continuos 102

30 Despliegue, configuración y observabilidad 104

PARTE 7 · LA IA COMO HERRAMIENTA DE TRABAJO 107

31 Trabajar con asistentes de IA 108

PARTE 8 · EL PORTFOLIO: TRES PROYECTOS QUE SÍ CONVENCEN 111

32 Qué hace bueno a un proyecto de portfolio 112

33 Proyecto 1 — La aplicación completa 114

34 Proyectos 2 y 3 116

PARTE 9 · COBRAR POR ELLO: EMPLEO JUNIOR EN ESPAÑA 118

35 El CV y LinkedIn 119

36 Tu GitHub como carta de presentación 121

37 Dónde buscar y cómo presentarse 122

38 El proceso de selección, fase por fase 124

39 Preguntas de entrevista con respuestas modelo 126

40 Salario y negociación 129

41 Los primeros noventa días 131

APÉNDICES · MATERIAL DE CONSULTA 133

A Plan de estudio de 12 semanas 134

B Chuletas de comandos 137

C Glosario 139

D Recursos recomendados 141

E Listas de comprobación 143

F Fuentes 145

PARTE 0

El mapa

Antes de escribir una sola línea de código conviene saber qué es esto realmente, qué está pagando hoy el mercado español y en qué orden hay que aprender las cosas para no perder meses.

- Qué hace un ingeniero de software (y en qué se diferencia de "saber programar")
- Radiografía del mercado en España, septiembre de 2026
- Por qué el stack elegido y no otro
- El plan de estudio completo

1. Qué es realmente un ingeniero de software

Programar es escribir instrucciones que una máquina ejecuta. La ingeniería de software es el oficio de construir sistemas que siguen funcionando cuando tú no estás delante, cuando los usa gente que no conoces y cuando otra persona tiene que modificarlos dentro de dos años.

La diferencia es enorme y es exactamente lo que separa a alguien que "ha hecho un curso" de alguien a quien se le paga un sueldo. Un script que funciona en tu portátil no vale dinero. Lo que vale dinero es código que además está versionado, probado, desplegado, monitorizado, documentado y escrito de forma que otro humano pueda tocarlo sin romperlo.

Conviene interiorizarlo desde el primer día porque condiciona todo lo que vas a estudiar en este manual: aproximadamente la mitad de las páginas no van de sintaxis, van de *oficio*. De Git, de pruebas, de despliegue, de cómo leer código ajeno, de cómo estimar, de cómo preguntar. Esa mitad es la que decide si te contratan.

1.1 Las cinco actividades del trabajo real

Si te sientas al lado de un desarrollador profesional durante una semana, verás que su tiempo se reparte, más o menos, así:

Actividad	Tiempo aprox.	Qué significa
Leer y entender código existente	30–40 %	Casi nunca se empieza de cero. Se añade a algo que ya existe y que escribieron otros.
Escribir código nuevo	15–25 %	Lo que todo el mundo imagina que es el trabajo. Es la parte pequeña.
Depurar y arreglar	20–30 %	Reproducir un fallo, encontrar la causa, corregirla sin romper otra cosa.
Comunicación	15–20 %	Reuniones cortas, revisiones de código, aclarar requisitos, escribir tickets.
Herramientas y entorno	5–10 %	Configurar, desplegar, automatizar, pelearse con dependencias.

> **CONSECUENCIA PRÁCTICA**
>
> Si tu formación consiste en ver vídeos donde alguien escribe código desde cero, estás entrenando el 20 % del trabajo. Por eso en este manual hay ejercicios de leer código ajeno, de depurar fallos preparados y de escribir mensajes de commit y descripciones de *pull request*. No es relleno.

1.2 Los tres niveles de carrera

El sector usa una escala razonablemente estándar. Saber dónde estás y qué se espera de cada nivel te ahorra frustración y te ayuda a negociar.

Nivel	Experiencia	Qué se espera
Junior	0–2 años	Completa tareas bien definidas con supervisión. Pregunta cuando se atasca. Su código se revisa siempre. Se le mide por lo rápido que aprende, no por lo que ya sabe.
Mid / semi-senior	2–5 años	Coge una funcionalidad entera y la saca solo. Sabe descomponer un problema. Revisa código de otros. Se le mide por autonomía.
Senior	5+ años	Decide cómo se construyen las cosas, no solo las construye. Anticipa problemas. Forma a los demás. Se le mide por el impacto en el equipo entero.

Tu objetivo con este manual es llegar al punto de entrada de la primera fila: ser contratable como junior. Eso no significa saberlo todo. Significa demostrar tres cosas concretas:

1. **Que sabes construir algo completo** — una aplicación con interfaz, servidor, base de datos y despliegue, aunque sea modesta.
2. **Que trabajas como un profesional** — Git, pruebas, código legible, capacidad de explicar tus decisiones.
3. **Que aprendes rápido y solo** — porque nadie contrata a un junior por lo que sabe hoy, sino por lo que sabrá dentro de seis meses.

1.3 Una advertencia sobre los plazos

Existe un género entero de contenido que promete "programador en 30 días". Es honesto decir lo siguiente: en una semana intensiva se pueden adquirir los *conceptos básicos* y tener una visión completa del mapa; ser contratable requiere meses de práctica real, no de consumo de material.

Las cifras que maneja el sector, para alguien que parte de nociones sueltas y dedica entre 15 y 25 horas semanales, son:

Hito	Dedicación acumulada	Plazo típico
Entender el mapa completo y escribir programas pequeños	40–60 h	1–3 semanas
Construir una aplicación full-stack sencilla sin copiar un tutorial	200–300 h	2–4 meses
Portfolio y nivel técnico suficientes para superar entrevistas junior	500–800 h	6–12 meses

> **LO QUE SÍ SE PUEDE HACER EN UNA SEMANA**
>
> Terminar la Parte 0, la Parte 1 completa y la Parte 2 hasta asincronía. Al acabar esa semana sabrás usar la terminal, Git y GitHub, entenderás cómo funciona una aplicación web por dentro y escribirás JavaScript con soltura básica. Es un punto de partida excelente y es medible. Lo que no se puede hacer en una semana es estar listo para cobrar.

2. El mercado en España, septiembre de 2026

Esta sección tiene fecha porque caduca. Los números son de septiembre de 2026 y las fuentes están al final del manual. Léela entera: decide qué estudias y qué no.

2.1 El dato incómodo: la puerta de entrada se ha estrechado

Hay que empezar por aquí porque cambia la estrategia. Durante 2025 y 2026, la adopción masiva de asistentes de IA ha reducido la demanda de perfiles junior. Los datos disponibles:

- Las ofertas para **programador júnior en España cayeron alrededor de un 33 %**, mientras que las de perfil sénior subieron un 13 % (estudio *Radiografía de Empleos y Sectores Emergentes*, DigitalES).
- En Estados Unidos, el empleo de desarrolladores de **22 a 25 años ha caído cerca de un 20 %** desde 2024 (AI Index Report 2026, Universidad de Stanford).
- En el conjunto del mercado IT español, solo alrededor del **3 % de las ofertas** no exige experiencia previa; el 51 % busca perfiles de 3 a 5 años.

La lectura fácil sería "no merece la pena". La lectura correcta es distinta: lo que ha desaparecido es el junior que solo traducía especificaciones a código, porque eso ya lo hace una máquina en segundos. Lo que ha aparecido en su lugar tiene nombre en los informes del sector: el **"junior aumentado"**.

> **QUÉ ES UN "JUNIOR AUMENTADO"**
>
> Un perfil de entrada que mantiene los fundamentos técnicos sólidos *y además* integra la IA en su flujo de trabajo con criterio. No escribe código más rápido: entiende el contexto de negocio, sabe hacer las preguntas correctas, y sobre todo **sabe evaluar y verificar lo que la IA produce**, que es precisamente lo que la IA no puede hacer por sí sola.
>
> Traducción operativa: los fundamentos importan *más* que antes, no menos. Quien no entiende lo que lee no puede revisar el código generado, y quien no puede revisarlo no aporta nada sobre la máquina.

Esto tiene tres consecuencias directas sobre cómo debes estudiar, y las vas a ver aplicadas a lo largo del manual:

1. **Profundidad sobre superficie.** Saber tres tecnologías de verdad vale más que haber tocado quince. Un junior que explica por qué su consulta SQL es lenta vale más que uno que ha hecho diez tutoriales.
2. **Portfolio con problemas reales.** La lista de la compra y la calculadora ya no distinguen a nadie. Un proyecto que resuelva algo concreto y verificable sí.
3. **Saber usar IA y saber desconfiar de ella.** Hay un capítulo entero sobre esto (Parte 7) y no es opcional: hoy es una competencia evaluada en entrevistas.

2.2 Qué tecnologías se piden de verdad

Estos son los datos de un mes típico de 2026 en el principal portal de empleo IT español, sobre unas 3.000 ofertas activas. Cuenta el número de ofertas que mencionan cada tecnología:

Tecnología	Ofertas (aprox.)	Lectura
Java	1.336	El rey del backend corporativo español: banca, seguros, administración, consultoras grandes.
Python	1.131	Backend, datos, IA, automatización. El de crecimiento más rápido.
SQL	934	Transversal. Se pide para casi todo y casi nadie lo domina bien.
AWS	684	La nube dominante. Aparece en ofertas de backend, no solo de infraestructura.
Azure	456	Fuerte en empresas con ecosistema Microsoft.
JavaScript / TypeScript	alto y creciente	Imprescindible en todo lo que tenga interfaz web. TypeScript ya es el estándar, no una opción.
React	alto	Framework frontend dominante en ofertas españolas, con diferencia.
Docker / Kubernetes	medio-alto	Docker se da casi por supuesto; Kubernetes es más de perfiles con experiencia.

> **CÓMO LEER ESTA TABLA SIN EQUIVOCARSE**
>
> Que Java tenga más ofertas que JavaScript *no* significa que sea mejor camino para empezar. Esas ofertas de Java piden mayoritariamente 3–5 años de experiencia en entornos empresariales. El ecosistema JavaScript/TypeScript concentra más puestos de entrada, más startups, más trabajo remoto y una curva de aprendizaje mucho más corta hasta tener algo desplegado y enseñable. Por eso es el stack de este manual.
>
> SQL es la excepción: apréndelo sí o sí, vayas por donde vayas. Es la habilidad con mejor relación entre esfuerzo y demanda de toda la lista.

2.3 Salarios reales en España

Rangos brutos anuales en 2026, para jornada completa por cuenta ajena. Son rangos amplios porque dependen mucho de ciudad, sector y tipo de empresa:

Perfil	Junior (0–2 años)	Mid (2–5)	Senior (5–10)	Senior+ (10+)
Frontend	20.000 – 30.000 €	31.000 – 35.000 €	35.000 – 40.000 €	40.000 – 50.000 €
Backend	20.000 – 30.000 €	31.000 – 40.000 €	41.000 – 50.000 €	45.000 – 55.000 €
Full-stack	20.000 – 30.000 €	31.000 – 40.000 €	41.000 – 50.000 €	51.000 – 60.000 €
Arquitecto TIC	—	—	47.000 – 56.600 €	+60.000 €

Matices que importan más que la tabla:

- **El primer sueldo real de un junior sin experiencia** suele estar en la banda baja: 18.000–24.000 € en consultora, 22.000–28.000 € en empresa de producto. Es un peaje, no un destino: el salto grande está entre el primer y el tercer año.

- **Geografía:** Madrid concentra la mayoría de ofertas (unas 529 en el mes analizado), seguida de Barcelona (213) y Valencia (47). Fuera de esos polos, el remoto es tu mercado. Desde Cantabria, buscar remoto no es plan B: es el plan principal.
- **Tipo de contrato:** el 96 % de las ofertas IT son indefinidas. El sector tiene una estabilidad contractual muy por encima de la media española.
- **El factor IA:** las guías salariales de 2026 apuntan a diferencias de hasta el 50 % en el mismo puesto entre quien integra herramientas de IA en su trabajo y quien no. Tómalo como orden de magnitud, no como promesa.

2.4 Los tres tipos de empresa y cuál te conviene primero

Tipo	Cómo es	Para un primer empleo
Consultora / cárnica Indra, Accenture, NTT Data, Everis, Capgemini…	Te contratan y te "colocan" en un cliente. Sueldos bajos, tecnología a menudo antigua, mucha rotación, procesos de selección menos exigentes.	**Puerta de entrada más fácil.** Aceptable como primer año para conseguir experiencia oficial en el CV. Sal después de 12–24 meses.
Empresa de producto Startups, scale-ups, SaaS	Construyen su propio producto. Mejor tecnología, más responsabilidad, más aprendizaje, mejor sueldo. Proceso de selección más duro.	**El objetivo.** Difícil sin experiencia, posible con un portfolio muy bueno.
Empresa no tecnológica con equipo IT Industria, retail, sanidad, sector público	Mantienen y evolucionan sistemas internos. Ritmo tranquilo, buena conciliación, tecnología intermedia.	**Infravalorada.** Menos competencia por puesto, entrada más accesible y buena escuela de sistemas reales.

3. El stack que vas a aprender

Un "stack" es el conjunto de tecnologías que usas de arriba abajo para construir una aplicación. Aquí está el que vas a aprender, y por qué cada pieza.

Capa	Tecnología	Por qué esta
Lenguaje	**JavaScript + TypeScript**	Único lenguaje que corre en navegador y servidor: aprendes uno y cubres todo el stack. TypeScript le añade tipos y ya es el estándar profesional.
Interfaz	**HTML, CSS, React**	React domina las ofertas españolas de frontend. HTML y CSS son innegociables por debajo.
Estilos	**CSS moderno + Tailwind**	Flexbox y Grid son la base real. Tailwind es lo que usan hoy la mayoría de equipos.
Servidor	**Node.js + Express**	Mismo lenguaje que el frontend. Express es minimalista y se entiende entero.
Base de datos	**PostgreSQL + SQL**	SQL es la habilidad más transversal del sector. PostgreSQL es el motor relacional más pedido y es gratis.
Acceso a datos	**Prisma**	ORM moderno con tipos. Te ahorra errores y se pide cada vez más.
Control de versiones	**Git + GitHub**	No es opcional en ningún puesto del planeta.
Pruebas	**Vitest + Playwright**	Saber probar es lo que más distingue a un junior serio de uno de curso.
Empaquetado	**Docker**	Aparece en la mayoría de ofertas backend. Con el nivel básico basta para empezar.
Despliegue	**Vercel, Railway, GitHub Actions**	Permiten tener algo público y funcionando gratis, que es lo que enseñas en una entrevista.

> **SOBRE NO DISPERSARSE**
>
> Existen decenas de alternativas a cada una de estas piezas (Vue, Angular, Svelte, Django, Laravel, .NET, MongoDB, Supabase…). Todas son válidas. La peor decisión posible es cambiar de stack cada mes porque alguien en internet dijo que el suyo es mejor. Elige este, termínalo, construye tres proyectos con él. Cambiar de stack después es cuestión de semanas; no tener ninguno dominado es cuestión de no trabajar.

4. El plan de estudio

Dos planes superpuestos: una semana intensiva para tener el mapa completo, y doce semanas para ser contratable. La Parte 10 (apéndice A) lo desglosa día a día.

4.1 Semana 1 — Intensiva

Día	Contenido	Entregable al terminar
1	Parte 0 completa + cap. 5 (terminal)	Entorno montado, terminal manejada
2	Cap. 6 (Git y GitHub)	Primer repositorio público con commits limpios
3	Cap. 7–8 (cómo funciona la web, entorno)	Sabes explicar qué pasa al escribir una URL
4	Cap. 9–10 (JS: fundamentos y funciones)	15 ejercicios resueltos
5	Cap. 11 (objetos, arrays, map/filter/reduce)	Programa que procesa un conjunto de datos
6	Cap. 12 (asincronía y fetch)	Script que consume una API pública
7	Repaso + cap. 15–16 (HTML y CSS)	Una página propia publicada en GitHub Pages

4.2 Semanas 2 a 12 — Hasta ser contratable

Semanas	Foco	Hito
2–3	JavaScript avanzado, TypeScript, HTML/CSS a fondo	Interfaz estática compleja y responsive, en TypeScript
4–5	React: componentes, estado, efectos, formularios, routing	Aplicación frontend que consume una API pública
6–7	Node, Express, API REST, PostgreSQL, SQL, Prisma	API propia con base de datos y CRUD completo
8	Autenticación, seguridad, validación	**Proyecto 1 terminado y desplegado**
9	Pruebas, código limpio, refactorización	Proyecto 1 con cobertura de pruebas razonable
10	Docker, CI/CD, despliegue, fundamentos de CS	Despliegue automático desde GitHub
11	**Proyecto 2** (el diferenciador) + IA en el flujo de trabajo	Segundo proyecto público
12	CV, LinkedIn, GitHub, preparación de entrevistas	Candidatura enviada a 20 ofertas

> **LA REGLA DE LAS DOS HORAS**
>
> De cada tres horas de estudio, como máximo una debe ser de consumo (leer, ver vídeos) y al menos dos de producción (escribir código con el manual cerrado). La sensación de aprender que da un vídeo es en gran parte una ilusión: el conocimiento se consolida al recuperarlo con esfuerzo, no al reconocerlo. Si terminas un capítulo y no has escrito nada, no has estudiado ese capítulo.

4.3 Cómo está organizado este manual

Cada capítulo sigue la misma estructura para que puedas trabajar con él de forma sistemática:

- **Explicación** con el concepto y por qué existe (nunca "esto se hace así" sin el porqué).
- **Código comentado**, siempre ejecutable y siempre realista.
- **Cajas azules**: consejos prácticos y atajos.
- **Cajas naranjas**: errores típicos y trampas.
- **Cajas verdes**: qué se pregunta de esto en una entrevista.
- **Ejercicios** al final de cada bloque, en recuadro de línea discontinua. Hazlos.

PARTE 1

Los fundamentos del oficio

Esto es lo que ningún curso de "aprende a programar" enseña y lo primero que se da por supuesto en cualquier empleo. Antes que el lenguaje, las herramientas.

- 5. La terminal
- 6. Git y GitHub
- 7. Cómo funciona la web por dentro
- 8. El entorno de trabajo profesional

5. La terminal

La línea de comandos no es nostalgia: es la interfaz por la que se instala, se despliega, se depura y se automatiza. Un desarrollador que no se maneja en la terminal es un desarrollador lento.

5.1 Por qué existe

Una interfaz gráfica te ofrece las acciones que su diseñador previó. La terminal te ofrece *todas*, y además te deja encadenarlas, repetirlas y guardarlas en un fichero para ejecutarlas mil veces. Renombrar 400 ficheros con la misma regla es media hora con el ratón y tres segundos con un comando.

Además, todos los servidores del mundo se administran así. No hay escritorio en una máquina de producción en AWS.

5.2 El modelo mental: dónde estás y qué hay

La terminal siempre está "situada" en un directorio. Todo lo que hagas se interpreta desde ahí. Los tres comandos que responden a "dónde estoy" y "qué hay":

```
pwd                 # print working directory: en qué carpeta estoy
ls                  # list: qué hay aquí
ls -la              # lo mismo, en detalle y con ficheros ocultos
cd proyectos        # entrar en la carpeta "proyectos"
cd ..               # subir un nivel
cd ~                # ir a mi carpeta personal
cd -                # volver a la carpeta anterior
```

Las rutas se escriben de dos maneras. Una **ruta absoluta** empieza por `/` y describe el camino desde la raíz del sistema: `/home/alvaro/proyectos/web`. Una **ruta relativa** parte de donde estás: `proyectos/web`, o `../otra-carpeta`. En scripts, usa absolutas; escribiendo a mano, relativas.

5.3 Manipular ficheros

```
mkdir mi-proyecto           # crear carpeta
mkdir -p a/b/c              # crear el árbol entero aunque no exista
touch index.js              # crear un fichero vacío
cp origen.js destino.js     # copiar
cp -r carpeta/ copia/       # copiar carpeta entera (-r = recursivo)
mv viejo.js nuevo.js        # renombrar (o mover)
rm fichero.js               # borrar fichero
rm -r carpeta/              # borrar carpeta y contenido
```

> **RM NO TIENE PAPELERA**
>
> `rm` borra de verdad, sin confirmación y sin recuperación. `rm -rf /` destruye un sistema entero. Acostúmbrate a escribir la ruta con cuidado y, cuando dudes, ejecuta primero `ls` sobre esa misma ruta para ver exactamente qué vas a borrar.

5.4 Ver y buscar dentro de ficheros

```
cat fichero.txt              # volcar el contenido entero
head -n 20 fichero.log       # las 20 primeras líneas
tail -n 50 fichero.log       # las 50 últimas
tail -f servidor.log         # seguir el fichero en vivo (para ver logs)
less fichero.txt             # visor navegable (q para salir)

grep "error" servidor.log           # líneas que contienen "error"
grep -i "error" servidor.log        # sin distinguir mayúsculas
grep -rn "TODO" src/                # buscar recursivamente y mostrar línea
```

`grep -rn` sobre el código fuente es probablemente el comando que más vas a usar en tu vida profesional: es cómo encuentras dónde está definida una función en un proyecto de 2.000 ficheros.

5.5 Tuberías: el superpoder de la terminal

El carácter `|` (tubería o *pipe*) conecta la salida de un comando con la entrada del siguiente. Cada programa hace una cosa pequeña; encadenándolos resuelves problemas grandes.

```
# Cuántas veces aparece "ERROR" en el log
grep "ERROR" app.log | wc -l

# Los 5 tipos de error más frecuentes, ordenados
grep "ERROR" app.log | cut -d' ' -f4 | sort | uniq -c | sort -rn | head -5

# Procesos de node que están corriendo
ps aux | grep node
```

REDIRECCIÓN

`comando > fichero.txt` escribe la salida en un fichero (lo sobrescribe). `>>` añade al final. `2>&1` manda también los errores al mismo sitio. Ejemplo típico: `npm run build > build.log 2>&1`.

5.6 Permisos y procesos

```
chmod +x script.sh      # hacer un fichero ejecutable
./script.sh             # ejecutarlo
sudo comando            # ejecutar como administrador

ps aux | grep node      # ver procesos
kill 12345              # terminar el proceso con PID 12345
kill -9 12345           # terminarlo a la fuerza
lsof -i :3000           # qué proceso ocupa el puerto 3000
```

Ese último comando lo vas a necesitar el primer día que arranques un servidor y te diga "port 3000 already in use".

5.7 Variables de entorno

Son valores que el sistema pone a disposición de los programas. Se usan para configurar una aplicación sin tocar su código, y sobre todo para las claves secretas.

```
echo $HOME                      # ver una variable
export API_KEY="abc123"         # definirla en esta sesión
env | grep API                  # listar las que contienen "API"

# Definirla solo para un comando concreto
NODE_ENV=production node server.js
```

EN LA ENTREVISTA

"¿Dónde guardas las claves de API de tu proyecto?" La respuesta correcta es: en variables de entorno, en un fichero `.env` que está en `.gitignore`, y nunca en el repositorio. Si tienes una clave subida a GitHub en tu portfolio, es un suspenso directo en muchos procesos.

EJERCICIOS · CAPÍTULO 5

1. Crea la estructura `practica/src/components` y `practica/tests` con un solo comando.
2. Crea diez ficheros `nota1.txt` … `nota10.txt` y luego bórralos todos con un solo comando.
3. Descarga un fichero de texto largo (por ejemplo con `curl`) y averigua, en un solo encadenamiento, cuáles son las diez palabras más repetidas.
4. Averigua qué proceso está usando el puerto 8080 en tu máquina.

6. Git y GitHub

Git es el sistema de control de versiones que usa prácticamente toda la industria. No saber Git descarta tu candidatura en el minuto tres de la primera entrevista. Este capítulo es, con diferencia, el de mejor retorno del manual.

6.1 Qué problema resuelve

Sin control de versiones, el historial de un proyecto acaba siendo `proyecto_final.zip`, `proyecto_final_v2.zip`, `proyecto_final_BUENO.zip`. Y cuando dos personas tocan el mismo fichero, una pierde su trabajo.

Git guarda una fotografía completa del proyecto cada vez que decides guardar (un *commit*), con autor, fecha y mensaje. Eso te da cuatro cosas: historial navegable, capacidad de volver atrás, trabajo en paralelo sin pisarse y culpabilidad rastreable línea a línea.

6.2 Los tres espacios

Este es el modelo mental que hay que tener claro, porque explica el 90 % de la confusión inicial:

Espacio	Qué es	Cómo se mueve algo hacia él
Directorio de trabajo	Los ficheros tal y como están ahora en tu disco	Editando con tu editor
Área de preparación (*staging*)	Lo que has marcado para incluir en el próximo commit	`git add`
Repositorio	El historial permanente de commits	`git commit`

El área de preparación existe para que puedas elegir *qué* cambios entran en cada commit. Si has arreglado un fallo y además has cambiado el color de un botón, son dos commits distintos, aunque los hayas hecho en la misma sesión.

6.3 El flujo diario

```
git init                          # crear repositorio en esta carpeta
git status                        # ¿qué ha cambiado? (úsalo constantemente)
git add index.js                  # preparar un fichero concreto
git add .                         # preparar todo lo cambiado
git commit -m "Añade validación de email en el registro"
git log --oneline --graph         # ver el historial de forma compacta
git diff                          # ver qué he cambiado y no he preparado aún
git diff --staged                 # ver qué he preparado y no he confirmado
```

MENSAJES DE COMMIT QUE NO DAN VERGÜENZA

En imperativo, en presente, describiendo *qué hace* el cambio, no qué has tocado. Bien: `Corrige cálculo del IVA en facturas exentas`. Mal: `cambios`, `fix`, `asdf`, `ya funciona`.

Muchos equipos usan el convenio *Conventional Commits*: `feat:` para funcionalidad nueva, `fix:` para correcciones, `docs:`, `test:`, `refactor:`, `chore:`. Adóptalo en tus proyectos personales: un historial así en tu GitHub dice mucho de ti antes de que nadie lea tu código.

6.4 Ramas

Una rama es una línea de desarrollo independiente. Permite trabajar en algo nuevo sin tocar la versión estable. Es la base del trabajo en equipo.

```
git branch                          # ver ramas
git switch -c feat/login            # crear rama y cambiar a ella
git switch main                     # volver a la principal
git merge feat/login                # integrar la rama en la actual
git branch -d feat/login            # borrarla una vez integrada
```

El flujo real en una empresa es casi siempre este:

1. Partes de `main` actualizada: `git switch main && git pull`
2. Creas una rama por tarea: `git switch -c feat/exportar-pdf`
3. Trabajas y haces varios commits pequeños.
4. Subes la rama: `git push -u origin feat/exportar-pdf`
5. Abres un *pull request* en GitHub.
6. Un compañero lo revisa, comenta, corriges.
7. Se integra en `main` y se borra la rama.

6.5 Conflictos

Un conflicto ocurre cuando dos ramas modifican las mismas líneas. Git no adivina y te pide que decidas. El fichero queda así:

```
<<<<<<< HEAD
const precio = base * 1.21;
=======
const precio = base * (1 + IVA);
>>>>>>> feat/impuestos
```

Arriba está tu versión, abajo la de la otra rama. Editas el fichero dejando el resultado correcto, borras las tres líneas de marcadores, y después:

```
git add fichero-en-conflicto.js
git commit                    # (git ya propone un mensaje de merge)
```

No hay magia. Un conflicto es simplemente una decisión humana que Git no puede tomar por ti. Verlos como algo normal, y no como una catástrofe, es parte de la madurez profesional.

6.6 Deshacer cosas

Situación	Comando
He cambiado un fichero y quiero descartarlo	`git restore fichero.js`
He hecho `git add` y no quería	`git restore --staged fichero.js`
Quiero corregir el mensaje del último commit	`git commit --amend -m "Mensaje bueno"`
Quiero deshacer el último commit pero conservar los cambios	`git reset --soft HEAD~1`
Quiero deshacer el último commit y tirar los cambios	`git reset --hard HEAD~1` (peligroso)
Quiero revertir un commit ya publicado	`git revert <hash>` (crea un commit inverso: seguro)
Necesito guardar cambios a medias y cambiar de rama	`git stash` / `git stash pop`
La he liado y quiero ver todo lo que he hecho	`git reflog` (casi todo es recuperable)

> **LA REGLA DE ORO**
>
> Nunca reescribas (con `reset --hard`, `rebase` o `push --force`) una historia que ya has compartido con otros. Para deshacer algo público, `git revert`. Reescribir historia compartida es de las pocas cosas en Git que provocan daño real a un equipo.

6.7 Trabajo con GitHub

```
git remote add origin https://github.com/usuario/repo.git
git push -u origin main          # subir por primera vez
git push                         # subir cambios posteriores
git pull                         # traer cambios de otros
git clone https://github.com/usuario/repo.git    # copiar un repo entero
```

El fichero `.gitignore` indica qué no debe subirse nunca. Un mínimo razonable para un proyecto Node:

```
node_modules/
.env
.env.local
dist/
build/
*.log
.DS_Store
coverage/
```

EN LA ENTREVISTA

Preguntas casi garantizadas: *diferencia entre merge y rebase* (merge conserva la historia real y crea un commit de unión; rebase reescribe tus commits encima de la rama destino, dejando una historia lineal); *qué haces ante un conflicto*; *qué es un pull request y para qué sirve la revisión de código*. Y la más reveladora: *cuéntame la última vez que la liaste con Git y cómo lo arreglaste*. Ten una respuesta honesta preparada.

EJERCICIOS · CAPÍTULO 6

1. Crea un repositorio local, haz cinco commits con mensajes en formato *Conventional Commits* y súbelo a GitHub.
2. Provoca un conflicto a propósito: crea dos ramas que cambien la misma línea y resuélvelo.
3. Haz un commit con un error en el mensaje y corrígelo con `--amend`.
4. Sube un fichero por error, bórralo del repositorio y añádelo a `.gitignore`.
5. Busca un repositorio pequeño de código abierto, clónalo y lee su historial con `git log --oneline --graph --all`. Intenta reconstruir cómo trabaja ese equipo.

7. Cómo funciona la web por dentro

"¿Qué ocurre exactamente cuando escribes una URL en el navegador y pulsas Enter?" Es la pregunta de entrevista más clásica que existe, y no es casualidad: quien la responde bien entiende el sistema entero.

7.1 El recorrido completo

1. **DNS.** El navegador necesita una dirección IP, no un nombre. Consulta el sistema de nombres de dominio, que traduce `ejemplo.com` a algo como `93.184.216.34`. Primero mira su caché, luego la del sistema operativo, luego pregunta a un servidor DNS.
2. **Conexión TCP.** Se abre una conexión con esa IP en el puerto 443 (HTTPS) o 80 (HTTP) mediante un saludo en tres pasos.
3. **Negociación TLS.** En HTTPS se intercambian certificados y se acuerda una clave de cifrado. A partir de aquí todo viaja cifrado.
4. **Petición HTTP.** El navegador envía algo como `GET / HTTP/1.1` con sus cabeceras.
5. **Procesamiento en el servidor.** El servidor decide qué devolver: un fichero estático, o el resultado de ejecutar código que probablemente consulte una base de datos.
6. **Respuesta HTTP.** Vuelve un código de estado, unas cabeceras y un cuerpo (normalmente HTML o JSON).
7. **Renderizado.** El navegador parsea el HTML, construye el DOM, descarga CSS, JavaScript e imágenes (nuevas peticiones), aplica estilos, calcula posiciones y pinta.
8. **Interactividad.** El JavaScript se ejecuta y la página empieza a responder al usuario, a menudo haciendo más peticiones en segundo plano.

7.2 HTTP: verbos y códigos

HTTP es un protocolo de petición y respuesta, y es **sin estado**: el servidor no recuerda nada de una petición a la siguiente. Todo lo que necesita saber tiene que venir en la petición (por eso existen las cookies y los tokens).

Verbo	Uso	¿Modifica?	¿Idempotente?
GET	Obtener un recurso	No	Sí
POST	Crear un recurso	Sí	No
PUT	Reemplazar un recurso entero	Sí	Sí
PATCH	Modificar parte de un recurso	Sí	No necesariamente
DELETE	Borrar un recurso	Sí	Sí

Idempotente significa que repetir la misma petición varias veces produce el mismo resultado que hacerla una. Importa mucho en reintentos automáticos.

Familia	Significado	Ejemplos que debes conocer
2xx	Éxito	200 OK · 201 Created · 204 No Content
3xx	Redirección	301 permanente · 302 temporal · 304 Not Modified
4xx	Error del cliente	400 Bad Request · 401 no autenticado · 403 no autorizado · 404 no existe · 409 conflicto · 422 datos inválidos · 429 demasiadas peticiones
5xx	Error del servidor	500 error interno · 502 Bad Gateway · 503 no disponible

401 FRENTE A 403

401 significa "no sé quién eres, identifícate". 403 significa "sé quién eres y no tienes permiso". Confundirlos es un error clásico de junior y se detecta en cualquier revisión de código.

7.3 Una petición real

```
GET /api/usuarios/42 HTTP/1.1
Host: api.ejemplo.com
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
User-Agent: Mozilla/5.0

--- respuesta ---

HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Cache-Control: max-age=60

{
  "id": 42,
  "nombre": "Ana Ruiz",
  "email": "ana@ejemplo.com"
}
```

7.4 JSON

Es el formato en el que viajan los datos entre frontend y backend. Reglas: claves siempre entre comillas dobles, sin comas finales, sin comentarios. Tipos admitidos: cadena, número, booleano, null, array y objeto.

```
{
  "id": 42,
  "activo": true,
  "puntuacion": 9.5,
  "etiquetas": ["nuevo", "verificado"],
  "direccion": { "ciudad": "Santander", "cp": "39001" },
  "borradoEn": null
}
```

7.5 Frontend, backend y base de datos

Capa	Dónde se ejecuta	De qué se ocupa
Frontend	Navegador del usuario	Mostrar información, recoger entradas, validar por comodidad, llamar a la API. Todo su código es público y modificable por el usuario.
Backend	Servidor	Lógica de negocio, validación real, autorización, acceso a datos. Es la única capa en la que puedes confiar.
Base de datos	Servidor (a menudo otro)	Guardar la información de forma persistente, consistente y consultable.

> **LA REGLA QUE EVITA EL 80 % DE LOS AGUJEROS DE SEGURIDAD**
>
> **Nunca confíes en el cliente.** Si validas el formulario solo en el navegador, cualquiera puede saltarse la validación con dos clics en las herramientas de desarrollador o mandando la petición directamente. Toda validación del frontend es cortesía hacia el usuario; la validación que cuenta se hace siempre en el servidor.

7.6 Conceptos que aparecerán constantemente

- **API.** Interfaz que expone un programa para que otros lo usen. En la web, normalmente un conjunto de rutas HTTP que devuelven JSON.
- **REST.** Estilo de diseño de APIs basado en recursos (`/usuarios/42`) y verbos HTTP. Es el estándar de facto.
- **CORS.** Mecanismo de seguridad del navegador que bloquea peticiones a otro dominio salvo que el servidor lo autorice con cabeceras. Te dará errores; ahora ya sabes de qué van.
- **Cookie.** Pequeño dato que el servidor guarda en el navegador y que este reenvía en cada petición. Base de las sesiones.
- **Caché.** Copia temporal para no repetir trabajo. Está en todas partes: navegador, CDN, servidor, base de datos.
- **CDN.** Red de servidores repartidos geográficamente que sirven los ficheros estáticos desde cerca del usuario.

EJERCICIOS · CAPÍTULO 7

1. Abre las herramientas de desarrollador de tu navegador (`F12`), pestaña Red, y carga una web cualquiera. Identifica: la primera petición, cuántas peticiones totales hay, cuál tarda más y cuáles devuelven 304.
2. Con `curl -i https://api.github.com/users/torvalds` , observa cabeceras y cuerpo.
3. Escribe con tus palabras, en menos de diez líneas, qué pasa al pulsar Enter en una URL. Sin mirar. Repítelo hasta que salga fluido: es respuesta de entrevista.

8. El entorno de trabajo profesional

8.1 Qué instalar

Herramienta	Para qué
Node.js (versión LTS)	Ejecutar JavaScript fuera del navegador. Incluye `npm`.
Visual Studio Code	El editor que usa la mayoría del sector.
Git	Control de versiones.
Un navegador con buenas devtools	Chrome o Firefox.
Docker Desktop	Más adelante, para bases de datos y despliegue.

En Windows, instala además **WSL2** (subsistema Linux) y trabaja dentro de él. Te ahorra una categoría entera de problemas con rutas, permisos y herramientas, y además te acerca al entorno real de los servidores.

8.2 npm: el gestor de paquetes

```
npm init -y                 # crear package.json
npm install express         # instalar una dependencia
npm install -D vitest       # dependencia solo de desarrollo
npm install                 # instalar todo lo del package.json
npm run dev                 # ejecutar un script definido
npm outdated                # ver qué está desactualizado
npm audit                   # ver vulnerabilidades conocidas
```

El fichero `package.json` describe tu proyecto:

```
{
  "name": "mi-api",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "node --watch src/server.js",
    "test": "vitest",
    "lint": "eslint src"
  },
  "dependencies": {
    "express": "^4.19.2"
  },
  "devDependencies": {
    "vitest": "^2.0.0"
  }
}
```

El `^` significa "esta versión o cualquier versión menor superior". El fichero `package-lock.json` fija las versiones exactas instaladas: **súbelo siempre al repositorio**, es lo que garantiza que tu compañero y el servidor instalen

exactamente lo mismo que tú.

8.3 Configuración mínima de VS Code

Extensiones que valen la pena desde el primer día: *ESLint*, *Prettier*, *GitLens*, *Error Lens*, y las específicas de tu stack (Tailwind CSS IntelliSense, Prisma).

Atajos que debes automatizar hasta no pensarlos:

Atajo	Acción
`Ctrl` + `P`	Abrir fichero por nombre
`Ctrl` + `Shift` + `F`	Buscar en todo el proyecto
`F12`	Ir a la definición
`Shift` + `F12`	Ver todas las referencias
`F2`	Renombrar símbolo en todo el proyecto
`Ctrl` + `` ` ``	Terminal integrada
`Alt` + `↑/↓`	Mover línea

8.4 Linter y formateador

Un **formateador** (Prettier) decide el estilo visual: comillas, sangrías, punto y coma. Un **linter** (ESLint) detecta errores probables: variables no usadas, comparaciones sospechosas, promesas sin gestionar.

```
npm install -D eslint prettier eslint-config-prettier
npx eslint --init
```

Configura VS Code para formatear al guardar. Es un detalle pequeño con un efecto grande: elimina para siempre las discusiones de estilo en las revisiones de código y hace que tus *diffs* contengan solo cambios reales.

8.5 Depurar de verdad

`console.log` es legítimo y todo el mundo lo usa, pero el depurador es otra liga. En VS Code, poner un punto de interrupción y ejecutar con `F5` te permite parar la ejecución y examinar el valor de cada variable en ese instante.

El método de depuración que funciona, y que conviene hacer explícito porque casi nadie lo enseña:

1. **Reproduce** el fallo de forma fiable. Si no puedes reproducirlo, no puedes arreglarlo.
2. **Reduce** el caso al mínimo que lo provoca.
3. **Formula una hipótesis** concreta: "creo que el array llega vacío".
4. **Compruébala** con un punto de interrupción o un log.
5. **Confirma o descarta** y repite. Una hipótesis cada vez.

LEER LOS MENSAJES DE ERROR

El error dice tres cosas: el tipo, el mensaje y la traza de llamadas. La traza se lee de arriba abajo: la primera línea que apunte a *tu* código (y no a `node_modules`) es casi siempre donde está el problema. Un porcentaje sorprendente de juniors no lee el error entero, y ahí suele estar la respuesta literal.

PARTE 2

El lenguaje: JavaScript y TypeScript

Un solo lenguaje para el navegador y el servidor. Aquí está todo lo que se usa a diario en un trabajo real, sin rincones exóticos.

- 9. Fundamentos: datos, variables, control de flujo
- 10. Funciones, ámbito y closures
- 11. Objetos, arrays y transformación de datos
- 12. Asincronía: promesas, async/await y fetch
- 13. Módulos, errores y organización del código
- 14. TypeScript: el estándar profesional

9. Fundamentos de JavaScript

9.1 Variables

Tres formas de declarar, de las cuales solo debes usar dos:

```
const nombre = "Álvaro";   // no se puede reasignar. Tu opción por defecto.
let contador = 0;          // se puede reasignar. Úsala solo si hace falta.
var viejo = true;          // obsoleta. No la uses nunca en código nuevo.
```

La regla profesional es simple: **usa `const` siempre; cambia a `let` solo cuando el compilador te obligue**. Un código lleno de `const` es más fácil de razonar porque sabes que cada nombre significa lo mismo de principio a fin del bloque.

CONST NO SIGNIFICA INMUTABLE

`const` impide reasignar la variable, no modificar su contenido. Esto es perfectamente válido:

```
const lista = [1, 2];
lista.push(3);          // correcto: la lista cambia
lista = [4, 5];         // error: no puedes reasignar
```

9.2 Tipos de datos

JavaScript tiene siete tipos primitivos y un tipo objeto. En la práctica manejarás sobre todo estos:

```
// Primitivos
const texto = "hola";              // string
const numero = 42;                 // number (enteros y decimales, todo junto)
const decimal = 3.14;              // también number
const verdadero = true;            // boolean
const nada = null;                 // ausencia intencionada de valor
let sinAsignar;                    // undefined: declarada pero sin valor
const grande = 9007199254740993n;  // bigint (raro)
const simbolo = Symbol("id");      // symbol (raro)

// Objeto: todo lo demás
const persona = { nombre: "Ana", edad: 30 };
const numeros = [1, 2, 3];
const fecha = new Date();
const fn = function () {};

typeof texto;      // "string"
typeof numero;     // "number"
typeof persona;    // "object"
typeof numeros;    // "object"  (!)
typeof null;       // "object"  (!! error histórico del lenguaje)
Array.isArray(numeros);   // true  <- así se comprueba un array
```

9.3 Cadenas de texto

```
const nombre = "Álvaro";
const apellido = 'Crespo';

// Plantillas: usa siempre estas, no concatenes con +
const saludo = `Hola, ${nombre} ${apellido}. Tienes ${2 + 3} mensajes.`;

// Multilínea sin trucos
const html = `
  <div>
    <h1>${nombre}</h1>
  </div>
`;

// Métodos que usarás constantemente
"  hola  ".trim();                 // "hola"
"hola".toUpperCase();              // "HOLA"
"hola mundo".split(" ");           // ["hola", "mundo"]
"hola".includes("ol");             // true
"hola".startsWith("ho");           // true
"hola".replace("o", "0");          // "h0la"
"hola".replaceAll("o", "0");       // "h0la"
"hola".slice(1, 3);                // "ol"
"hola".at(-1);                     // "a"  (último carácter)
"5".padStart(3, "0");              // "005"
```

9.4 Números y sus trampas

```
0.1 + 0.2;                   // 0.30000000000000004  (!)
0.1 + 0.2 === 0.3;           // false

// Por eso NUNCA se guarda dinero en decimales.
// Se guarda en céntimos, como enteros:
const precioCentimos = 1999;         // 19,99 €
const total = precioCentimos * 3;    // 5997 céntimos = 59,97 €

Math.round(4.6);             // 5
Math.floor(4.9);             // 4
Math.ceil(4.1);              // 5
Math.max(3, 7, 2);           // 7
Math.random();               // decimal entre 0 y 1

Number("42");                // 42
parseInt("42px", 10);        // 42
parseFloat("3.14kg");        // 3.14
Number("hola");              // NaN
Number.isNaN(Number("hola")); // true
(1234.5678).toFixed(2);      // "1234.57"  (devuelve string)
```

EN LA ENTREVISTA

"¿Por qué `0.1 + 0.2` no es `0.3`?" Respuesta: porque los números en JavaScript son de coma flotante de doble precisión (IEEE 754) y ciertos decimales no tienen representación binaria exacta, igual que 1/3 no la tiene en decimal. Consecuencia práctica: para importes se trabaja con enteros (céntimos) o con una librería de decimales.

9.5 Comparaciones y valores "falsy"

```
// === compara valor Y tipo. Úsalo siempre.
1 === "1";       // false
1 == "1";        // true   <- conversión implícita: fuente de errores

// Valores que se evalúan como falso:
false, 0, -0, 0n, "", null, undefined, NaN
// Todo lo demás es verdadero, incluidos [] y {}

if ([]) console.log("esto se ejecuta");   // sí, un array vacío es truthy
```

```
// Operadores modernos que te ahorrarán muchos errores

// ?? devuelve la derecha solo si la izquierda es null o undefined
const puerto = config.puerto ?? 3000;

// || devuelve la derecha si la izquierda es CUALQUIER falsy
const puertoMal = config.puerto || 3000;   // si puerto es 0, ¡da 3000!

// ?. accede sin reventar si algo es null/undefined
const ciudad = usuario?.direccion?.ciudad;    // undefined en vez de error

// ??= asigna solo si está vacío
opciones.timeout ??= 5000;
```

9.6 Control de flujo

```
// Condicional
if (edad >= 18) {
  console.log("mayor");
} else if (edad >= 16) {
  console.log("casi");
} else {
  console.log("menor");
}

// Ternario: para expresiones cortas
const etiqueta = edad >= 18 ? "adulto" : "menor";

// switch
switch (estado) {
  case "pendiente":
    procesar();
    break;
  case "pagado":
  case "enviado":
    archivar();
    break;
  default:
    registrarDesconocido(estado);
}
```

```
// Bucles
for (let i = 0; i < 5; i++) { }

const frutas = ["pera", "uva"];
for (const fruta of frutas) { }              // sobre valores (arrays, strings, sets)

const persona = { nombre: "Ana", edad: 30 };
for (const [clave, valor] of Object.entries(persona)) { }

while (quedanTareas()) { }

do { intento++; } while (intento < 3);

// break sale del bucle, continue salta a la siguiente vuelta
```

CLÁUSULAS DE GUARDA

Anidar condiciones hace el código ilegible. Invierte las condiciones y sal pronto:

```
// Mal
function procesar(pedido) {
  if (pedido) {
    if (pedido.pagado) {
      if (pedido.lineas.length > 0) {
        enviar(pedido);
      }
    }
  }
}

// Bien
function procesar(pedido) {
  if (!pedido) return;
  if (!pedido.pagado) return;
  if (pedido.lineas.length === 0) return;
  enviar(pedido);
}
```

EJERCICIOS · CAPÍTULO 9

1. Escribe una función que reciba un precio en céntimos y devuelva una cadena como `"19,99 €"`.
2. FizzBuzz: imprime del 1 al 100; múltiplos de 3 "Fizz", de 5 "Buzz", de ambos "FizzBuzz". Es un clásico de entrevista.
3. Dada una cadena, comprueba si es un palíndromo ignorando espacios y acentos.
4. Escribe una función que valide una contraseña: mínimo 8 caracteres, una mayúscula, un número. Devuelve un array con los fallos, no un booleano.

10. Funciones, ámbito y closures

10.1 Las tres formas de declarar una función

```
// Declaración: se eleva (hoisting), puedes llamarla antes de definirla
function sumar(a, b) {
  return a + b;
}

// Expresión
const restar = function (a, b) {
  return a - b;
};

// Función flecha: la más usada hoy
const multiplicar = (a, b) => a * b;

// Con cuerpo
const dividir = (a, b) => {
  if (b === 0) throw new Error("División por cero");
  return a / b;
};

// Devolver un objeto: hay que envolverlo en paréntesis
const crearPunto = (x, y) => ({ x, y });
```

10.2 Parámetros

```
// Valores por defecto
function saludar(nombre, saludo = "Hola") {
  return `${saludo}, ${nombre}`;
}

// Resto: agrupa el resto de argumentos en un array
function sumarTodos(...numeros) {
  return numeros.reduce((a, b) => a + b, 0);
}
sumarTodos(1, 2, 3, 4);        // 10

// Objeto de opciones: el patrón profesional cuando hay 3+ parámetros
function crearUsuario({ nombre, email, rol = "usuario", activo = true }) {
  return { nombre, email, rol, activo };
}
crearUsuario({ nombre: "Ana", email: "ana@x.com" });
```

POR QUÉ EL OBJETO DE OPCIONES

Comparar estas dos llamadas: `crear("Ana", "ana@x.com", true, false, null)` frente a `crear({ nombre: "Ana", email: "ana@x.com", activo: true })`. La segunda se entiende sin abrir la definición, el orden da igual y añadir un parámetro nuevo no rompe el código existente. Es un hábito que distingue inmediatamente a alguien con criterio.

10.3 Ámbito (scope)

```
const global = "visible en todo el fichero";

function ejemplo() {
  const local = "solo dentro de esta función";

  if (true) {
    const bloque = "solo dentro de este if";
    let tambienBloque = "igual";
    var fuga = "visible en TODA la función (por eso var es mala)";
  }

  console.log(bloque);   // ReferenceError
  console.log(fuga);     // funciona, y es confuso
}
```

La regla: `const` y `let` viven dentro de las llaves donde se declaran. `var` vive en toda la función. Por eso `var` está desterrada.

10.4 Closures

Un closure es una función que "recuerda" el ámbito en el que fue creada, incluso después de que ese ámbito haya terminado. Suena abstracto; es una de las herramientas más usadas del lenguaje.

```
function crearContador() {
  let cuenta = 0;                 // variable privada

  return {
    incrementar() { cuenta++; return cuenta; },
    valor()       { return cuenta; },
  };
}

const contador = crearContador();
contador.incrementar();     // 1
contador.incrementar();     // 2
contador.valor();           // 2
contador.cuenta;            // undefined: no hay acceso directo
```

La variable `cuenta` sigue viva porque las funciones devueltas la referencian, pero nadie más puede tocarla. Así se implementa encapsulación sin clases.

```
// Uso realista: una función que limita la frecuencia de llamadas
function debounce(fn, ms) {
  let temporizador;                      // capturada por el closure
  return (...args) => {
    clearTimeout(temporizador);
    temporizador = setTimeout(() => fn(...args), ms);
  };
}

// Solo busca 300 ms después de que el usuario deje de teclear
const buscarConRetardo = debounce(buscar, 300);
input.addEventListener("input", (e) => buscarConRetardo(e.target.value));
```

EN LA ENTREVISTA

"Explícame qué es un closure y dame un caso real." Es una de las tres preguntas más repetidas en entrevistas de JavaScript. Responde con la definición en una frase y **el ejemplo del debounce o del contador privado**. Enseñar un uso concreto vale más que la definición académica.

10.5 Funciones de orden superior

En JavaScript las funciones son valores: se pueden pasar como argumento y devolver como resultado. Es la base de todo el estilo moderno del lenguaje.

```
// Recibe una función
function repetir(n, accion) {
  for (let i = 0; i < n; i++) accion(i);
}
repetir(3, (i) => console.log(`vuelta ${i}`));

// Devuelve una función
const multiplicarPor = (factor) => (n) => n * factor;
const doble = multiplicarPor(2);
doble(21);                          // 42
```

10.6 this y las funciones flecha

Las funciones flecha no tienen su propio `this`: heredan el del contexto donde se definieron. Esto resuelve el error clásico de perder el contexto dentro de un callback.

```
const temporizador = {
  segundos: 0,

  arrancarMal() {
    setInterval(function () {
      this.segundos++;        // this NO es temporizador aquí
    }, 1000);
  },

  arrancarBien() {
    setInterval(() => {
      this.segundos++;        // flecha: this sigue siendo temporizador
    }, 1000);
  },
};
```

EJERCICIOS · CAPÍTULO 10

1. Escribe `memoizar(fn)`: devuelve una versión de `fn` que guarda en caché los resultados ya calculados.
2. Escribe `once(fn)`: una función que solo se ejecuta la primera vez y después devuelve siempre el mismo resultado.
3. Escribe `componer(f, g)` que devuelva una función que aplique primero `g` y luego `f`.
4. Implementa `throttle(fn, ms)`: ejecuta como mucho una vez cada `ms` milisegundos.

11. Objetos, arrays y transformación de datos

Aquí está el 70 % del código que escribirás en tu vida: coger unos datos, filtrarlos, transformarlos y agruparlos. Domina este capítulo por encima de los demás.

11.1 Objetos

```
const usuario = {
  nombre: "Ana",
  edad: 30,
  direccion: { ciudad: "Santander", cp: "39001" },
  saludar() { return `Soy ${this.nombre}`; },
};

usuario.nombre;                  // "Ana"
usuario["nombre"];               // igual, útil con claves dinámicas
usuario.direccion?.ciudad;       // acceso seguro

// Añadir, modificar, borrar
usuario.email = "ana@x.com";
delete usuario.edad;

// Comprobar existencia
"email" in usuario;                          // true
Object.hasOwn(usuario, "email");             // true (preferible)

// Recorrer
Object.keys(usuario);        // ["nombre", "direccion", "saludar", "email"]
Object.values(usuario);
Object.entries(usuario);     // [["nombre","Ana"], ...]
```

11.2 Desestructuración

```
const { nombre, edad = 0, direccion: { ciudad } } = usuario;
// nombre = "Ana", edad = 0 (por defecto), ciudad = "Santander"

// Renombrar
const { nombre: nombreUsuario } = usuario;

// En arrays
const [primero, segundo, ...resto] = [1, 2, 3, 4, 5];

// Intercambiar sin variable auxiliar
let a = 1, b = 2;
[a, b] = [b, a];

// En parámetros: patrón habitual en React y Express
function Tarjeta({ titulo, descripcion }) { }
app.get("/u/:id", ({ params: { id } }, res) => { });
```

11.3 Propagación (spread) y copias

```
const base = { nombre: "Ana", rol: "user" };

const copia = { ...base };                      // copia superficial
const modificado = { ...base, rol: "admin" };   // copia con un cambio
const fusion = { ...base, ...otrosDatos };      // fusionar

const nums = [1, 2, 3];
const masNums = [...nums, 4, 5];
const combinado = [...nums, ...otros];
```

COPIA SUPERFICIAL FRENTE A COPIA PROFUNDA

`{ ...objeto }` solo copia el primer nivel. Los objetos anidados siguen siendo *el mismo objeto*:

```
const a = { datos: { x: 1 } };
const b = { ...a };
b.datos.x = 99;
a.datos.x;              // 99  (!)

// Copia profunda de verdad:
const c = structuredClone(a);
```

Este error causa fallos difíciles de rastrear, sobre todo en React, donde modificar accidentalmente un estado compartido hace que la interfaz no se actualice o se actualice de más.

11.4 Los métodos de array que hay que dominar

```
const productos = [
  { id: 1, nombre: "Teclado", precio: 4500, stock: 12, categoria: "periferico" },
  { id: 2, nombre: "Monitor", precio: 19900, stock: 0,  categoria: "pantalla" },
  { id: 3, nombre: "Ratón",   precio: 2500, stock: 40, categoria: "periferico" },
  { id: 4, nombre: "Webcam",  precio: 6900, stock: 3,  categoria: "periferico" },
];
```

MAP — TRANSFORMAR CADA ELEMENTO

```
const nombres = productos.map((p) => p.nombre);
// ["Teclado", "Monitor", "Ratón", "Webcam"]

const conIva = productos.map((p) => ({ ...p, precioConIva: Math.round(p.precio * 1.21) }));
```

FILTER — QUEDARSE CON ALGUNOS

```
const disponibles = productos.filter((p) => p.stock > 0);
const baratos = productos.filter((p) => p.precio < 5000);
```

FIND / FINDINDEX / SOME / EVERY — BUSCAR Y COMPROBAR

```
productos.find((p) => p.id === 3);           // el objeto, o undefined
productos.findIndex((p) => p.id === 3);      // 2, o -1
productos.some((p) => p.stock === 0);        // true: ¿alguno agotado?
productos.every((p) => p.precio > 1000);     // true: ¿todos por encima?
productos.includes(x);                        // para valores primitivos
```

REDUCE — REDUCIR A UN SOLO VALOR

```
// Total del inventario
const valorTotal = productos.reduce((suma, p) => suma + p.precio * p.stock, 0);

// Agrupar por categoría: el uso más útil de reduce
const porCategoria = productos.reduce((acc, p) => {
  acc[p.categoria] ??= [];
  acc[p.categoria].push(p);
  return acc;
}, {});
// { periferico: [...3 productos], pantalla: [...1] }

// Índice por id, para búsquedas O(1)
const porId = productos.reduce((acc, p) => ({ ...acc, [p.id]: p }), {});
```

SORT — ORDENAR

```
// ¡sort modifica el array original! Copia primero:
const porPrecio = [...productos].sort((a, b) => a.precio - b.precio);   // ascendente
const porPrecioDesc = [...productos].sort((a, b) => b.precio - a.precio);

// Texto, respetando acentos y español
const porNombre = [...productos].sort((a, b) => a.nombre.localeCompare(b.nombre, "es"));

// Desde 2023 existe la versión que no muta:
const ordenado = productos.toSorted((a, b) => a.precio - b.precio);
```

ENCADENAR: ASÍ SE ESCRIBE CÓDIGO REAL

```
const resumen = productos
  .filter((p) => p.stock > 0)
  .sort((a, b) => b.precio - a.precio)
  .slice(0, 2)
  .map((p) => `${p.nombre}: ${(p.precio / 100).toFixed(2)} €`);

// ["Webcam: 69.00 €", "Teclado: 45.00 €"]
```

CÓMO ELEGIR EL MÉTODO

¿Quiero la misma cantidad de elementos pero transformados? `map`. ¿Menos elementos? `filter`. ¿Uno solo? `find`. ¿Un booleano? `some` o `every`. ¿Un único valor agregado (suma, objeto agrupado, cadena)? `reduce`. Si te sorprendes usando `forEach` para construir un array nuevo, casi siempre querías `map` o `filter`.

11.5 Otros métodos útiles

```
productos.flatMap((p) => p.etiquetas ?? []);   // map + aplanar
[[1,2],[3,[4]]].flat(2);                        // [1,2,3,4]
Array.from({ length: 5 }, (_, i) => i * 2);     // [0,2,4,6,8]
[...new Set([1,1,2,3,3])];                      // [1,2,3]  quitar duplicados
Object.groupBy(productos, (p) => p.categoria);  // agrupar (moderno)
productos.at(-1);                               // último elemento
```

11.6 Map y Set

```
// Map: como un objeto, pero con claves de cualquier tipo y orden garantizado
const cache = new Map();
cache.set("clave", valor);
cache.get("clave");
cache.has("clave");
cache.delete("clave");
cache.size;

// Set: colección sin duplicados
const vistos = new Set();
vistos.add(1);
vistos.add(1);        // no hace nada
vistos.has(1);        // true
vistos.size;          // 1
```

EJERCICIOS · CAPÍTULO 11

1. Con el array `productos`: calcula el precio medio, encuentra el más caro sin usar `sort`, y agrupa por si tienen stock o no.
2. Dado un array de pedidos con `{ cliente, importe }`, obtén el total gastado por cliente ordenado de mayor a menor.
3. Implementa tu propia versión de `map` y de `filter` usando solo `reduce`.
4. Dado un array de objetos con posibles duplicados por `id`, devuelve el array sin duplicados conservando el primero de cada id.

12. Asincronía

JavaScript es de un solo hilo: solo puede hacer una cosa a la vez. La asincronía es cómo consigue no bloquearse mientras espera al disco, a la red o al usuario. Es el concepto que más cuesta y el que más se pregunta.

12.1 El modelo de ejecución

Hay una pila de llamadas donde se ejecuta el código, y un bucle de eventos que va metiendo en esa pila las tareas que van terminando (una respuesta de red, un temporizador). Mientras la pila está ocupada, nada más se ejecuta.

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() => console.log("3"));
console.log("4");

// Imprime: 1, 4, 3, 2
```

Explicación: `1` y `4` son síncronos y van primero. Luego se vacía la *cola de microtareas* (promesas), de ahí el `3`. Y por último la cola de macrotareas (temporizadores), el `2`. Aunque el timeout sea de 0 ms.

> **EN LA ENTREVISTA**
>
> Ese fragmento exacto, o una variante, aparece en un porcentaje altísimo de entrevistas de JavaScript. Memoriza la explicación: pila → microtareas (promesas) → macrotareas (timeouts).

12.2 Promesas

Una promesa representa un valor que todavía no está disponible. Tiene tres estados: pendiente, cumplida o rechazada.

```
const promesa = new Promise((resolver, rechazar) => {
  setTimeout(() => {
    if (Math.random() > 0.5) resolver("¡Éxito!");
    else rechazar(new Error("Ha fallado"));
  }, 1000);
});

promesa
  .then((resultado) => console.log(resultado))
  .catch((error) => console.error(error.message))
  .finally(() => console.log("Termine como termine"));
```

12.3 async / await

Es azúcar sintáctico sobre las promesas, y es como se escribe código asíncrono hoy. Permite escribir código asíncrono que se lee como si fuera secuencial.

```
async function obtenerUsuario(id) {
  try {
    const respuesta = await fetch(`https://api.ejemplo.com/usuarios/${id}`);

    if (!respuesta.ok) {
      throw new Error(`HTTP ${respuesta.status}: ${respuesta.statusText}`);
    }

    const usuario = await respuesta.json();
    return usuario;
  } catch (error) {
    console.error("No se pudo obtener el usuario:", error.message);
    throw error;                // relanzar para que el llamante decida
  }
}

// Uso
const usuario = await obtenerUsuario(42);
```

FETCH NO LANZA ERROR EN UN 404

Este es el error número uno de los principiantes con `fetch`: solo rechaza la promesa si hay un fallo de red. Un 404 o un 500 se consideran respuestas válidas. **Siempre hay que comprobar `respuesta.ok`**, como en el ejemplo anterior.

12.4 Secuencial frente a paralelo

```
// MAL: tres esperas encadenadas = 3 segundos
const a = await tarea();   // 1s
const b = await tarea();   // 1s
const c = await tarea();   // 1s

// BIEN: las tres a la vez = 1 segundo
const [a, b, c] = await Promise.all([tarea(), tarea(), tarea()]);
```

Las cuatro combinaciones de promesas que debes conocer:

Método	Comportamiento
`Promise.all`	Espera a todas. Si una falla, falla todo inmediatamente.
`Promise.allSettled`	Espera a todas y devuelve el resultado de cada una, fallen o no. Ideal cuando quieres lo que se pueda.
`Promise.race`	Devuelve la primera que termine, sea éxito o error. Se usa para timeouts.
`Promise.any`	Devuelve la primera que tenga éxito. Solo falla si fallan todas.

```
// Petición con límite de tiempo
const controlador = new AbortController();
setTimeout(() => controlador.abort(), 5000);

const r = await fetch(url, { signal: controlador.signal });
```

12.5 Bucles con await

```
// Secuencial: útil si el orden importa o para no saturar una API
for (const id of ids) {
  const u = await obtenerUsuario(id);
  procesar(u);
}

// Paralelo
const usuarios = await Promise.all(ids.map((id) => obtenerUsuario(id)));

// TRAMPA: forEach ignora async. Esto NO espera a nada.
ids.forEach(async (id) => {
  await obtenerUsuario(id);       // el forEach termina antes que las promesas
});
```

12.6 Un cliente HTTP reutilizable

```
async function api(ruta, opciones = {}) {
  const respuesta = await fetch(`${BASE_URL}${ruta}`, {
    headers: {
      "Content-Type": "application/json",
      ...(opciones.token && { Authorization: `Bearer ${opciones.token}` }),
      ...opciones.headers,
    },
    ...opciones,
  });

  if (!respuesta.ok) {
    const cuerpo = await respuesta.json().catch(() => ({}));
    throw new Error(cuerpo.mensaje ?? `Error ${respuesta.status}`);
  }

  if (respuesta.status === 204) return null;
  return respuesta.json();
}

// Uso
const usuarios = await api("/usuarios");
const nuevo = await api("/usuarios", {
  method: "POST",
  body: JSON.stringify({ nombre: "Ana" }),
});
```

EJERCICIOS · CAPÍTULO 12

1. Consume una API pública gratuita (por ejemplo la de Pokémon o la del tiempo) y muestra cinco resultados por consola con manejo de errores completo.
2. Escribe `reintentar(fn, intentos)` : ejecuta una función asíncrona y reintenta con espera creciente si falla.
3. Escribe `conTimeout(promesa, ms)` usando `Promise.race` .
4. Descarga diez recursos en paralelo pero con un máximo de tres peticiones simultáneas.

13. Módulos, errores y organización

13.1 Módulos ES

```
// matematicas.js
export function sumar(a, b) { return a + b; }
export const PI = 3.14159;
export default class Calculadora { }

// otro-fichero.js
import Calculadora, { sumar, PI } from "./matematicas.js";
import * as mates from "./matematicas.js";
import { sumar as suma } from "./matematicas.js";
```

En Node, para usar esta sintaxis hay que poner `"type": "module"` en el `package.json`. La sintaxis antigua (`require` / `module.exports`) aún se encuentra en muchísimo código existente, así que conviene reconocerla:

```
const express = require("express");
module.exports = { sumar };
```

13.2 Errores

```
// Lanzar
throw new Error("Algo ha ido mal");

// Errores propios: permiten distinguir casos al capturarlos
class ErrorValidacion extends Error {
  constructor(campo, mensaje) {
    super(mensaje);
    this.name = "ErrorValidacion";
    this.campo = campo;
    this.status = 422;
  }
}

try {
  validar(datos);
} catch (error) {
  if (error instanceof ErrorValidacion) {
    responder(error.status, { campo: error.campo, mensaje: error.message });
  } else {
    registrar(error);
    responder(500, { mensaje: "Error interno" });
  }
}
```

TRES ANTIPATRONES QUE VERÁS Y NO DEBES COPIAR

- `catch (e) { }` — silenciar un error. El fallo sigue ahí, pero ahora es invisible.
- `catch (e) { console.log(e) }` y continuar como si nada. El programa sigue en estado inválido.
- Capturar en cada nivel y relanzar el mismo error. Captura donde puedas *hacer algo*; si no, deja que suba.

13.3 Cómo organizar un proyecto

```
src/
  routes/          # define las rutas HTTP, nada más
  controllers/     # traduce petición <-> caso de uso
  services/        # lógica de negocio (aquí está el valor real)
  repositories/    # acceso a datos
  models/          # esquemas y tipos
  middleware/      # autenticación, errores, logs
  utils/           # funciones auxiliares genéricas
  config/          # configuración y variables de entorno
tests/
```

La regla que hace útil esta estructura: **las dependencias apuntan hacia dentro**. Una ruta puede llamar a un servicio; un servicio nunca debe saber que existe Express. Así puedes probar tu lógica de negocio sin levantar un servidor, y cambiar de framework sin reescribir el núcleo.

14. TypeScript

TypeScript es JavaScript con tipos comprobados antes de ejecutar. En 2026 no es una alternativa: es lo que piden la mayoría de ofertas de frontend y una parte creciente de las de backend Node.

14.1 Qué problema resuelve

```
// JavaScript: esto revienta en producción, a las 3 de la mañana
function calcularTotal(pedido) {
  return pedido.lineas.reduce((s, l) => s + l.precio * l.cantidad, 0);
}
calcularTotal({ items: [] });     // TypeError: pedido.lineas is undefined

// TypeScript: esto no compila, y el error aparece mientras escribes
type Linea = { precio: number; cantidad: number };
type Pedido = { lineas: Linea[] };

function calcularTotal(pedido: Pedido): number {
  return pedido.lineas.reduce((s, l) => s + l.precio * l.cantidad, 0);
}
calcularTotal({ items: [] });     // Error de compilación
```

14.2 Tipos básicos

```
let nombre: string = "Ana";
let edad: number = 30;
let activo: boolean = true;
let etiquetas: string[] = ["a", "b"];
let par: [string, number] = ["x", 1];        // tupla
let cualquiera: any;                          // evítalo: desactiva TypeScript
let desconocido: unknown;                     // seguro: obliga a comprobar antes de usar

// Unión: puede ser una cosa u otra
let id: string | number;

// Literal: solo estos valores exactos
let estado: "pendiente" | "pagado" | "enviado";

// Opcional
function saludar(nombre: string, apellido?: string) { }
```

14.3 Interfaces y tipos

```
interface Usuario {
  id: number;
  nombre: string;
  email: string;
  rol: "admin" | "usuario";
  ultimoAcceso?: Date;            // opcional
  readonly creadoEn: Date;        // no modificable
}

type Producto = {
  id: number;
  precio: number;
};

// Composición
interface Empleado extends Usuario {
  departamento: string;
}

type ConTimestamps<T> = T & { creadoEn: Date; actualizadoEn: Date };
```

En la práctica: usa `interface` para la forma de los objetos y `type` para todo lo demás (uniones, alias, tipos calculados). La diferencia real es menor de lo que sugieren los debates de internet.

14.4 Utilidades que se usan a diario

```
type UsuarioParcial = Partial<Usuario>;              // todo opcional
type UsuarioObligatorio = Required<Usuario>;         // todo obligatorio
type SoloDatos = Omit<Usuario, "creadoEn">;          // quitar campos
type Credenciales = Pick<Usuario, "email" | "id">;   // quedarse con algunos
type Inmutable = Readonly<Usuario>;
type PorId = Record<number, Usuario>;                // diccionario

// Typescript deduce el tipo de retorno, no hace falta escribirlo siempre
function activos(us: Usuario[]) {
  return us.filter((u) => u.ultimoAcceso !== undefined);
}   // deducido: Usuario[]
```

14.5 Genéricos

```
// Una función que funciona con cualquier tipo sin perder la información de tipo
function primero<T>(lista: T[]): T | undefined {
  return lista[0];
}

primero([1, 2, 3]);          // number | undefined
primero(["a", "b"]);         // string | undefined

// Caso realísimo: una respuesta de API tipada
interface Respuesta<T> {
  datos: T;
  error: string | null;
}

async function obtener<T>(ruta: string): Promise<Respuesta<T>> {
  const r = await fetch(ruta);
  return { datos: await r.json(), error: null };
}

const usuarios = await obtener<Usuario[]>("/api/usuarios");
// usuarios.datos está tipado como Usuario[]
```

14.6 Puesta en marcha

```
npm install -D typescript tsx @types/node
npx tsc --init
```

```
// tsconfig.json: lo importante
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "strict": true,              // innegociable: actívalo
    "noUncheckedIndexedAccess": true,
    "outDir": "dist"
  }
}
```

STRICT: TRUE DESDE EL PRIMER DÍA

Sin `strict`, TypeScript deja pasar `null` y `undefined` en todas partes y pierde la mitad de su utilidad. Activarlo al principio duele un par de días; activarlo en un proyecto de 20.000 líneas duele un mes. Y en una entrevista, decir que trabajas en modo estricto es señal de criterio.

EJERCICIOS · CAPÍTULO 14

1. Convierte a TypeScript los ejercicios del capítulo 11, tipando el array de productos.
2. Define los tipos de un sistema de reservas: `Sala`, `Reserva`, `Usuario`, con estados como tipos literales.
3. Escribe una función genérica `agruparPor<T, K>` correctamente tipada.
4. Coge un fichero JavaScript tuyo de los capítulos anteriores y tipalo entero con `strict` activado. Anota cuántos errores reales encuentra.

PARTE 3

Frontend: la parte visible

HTML y CSS son la base sobre la que se apoya todo lo demás. React es lo que piden las ofertas. Los tres son necesarios y en ese orden.

- 15. HTML semántico y accesibilidad
- 16. CSS moderno: Flexbox, Grid y Tailwind
- 17. React: el modelo mental y lo esencial
- 18. React en un proyecto real

15. HTML semántico y accesibilidad

Todo el mundo cree que sabe HTML y casi nadie lo escribe bien. Es la parte más fácil de aprender y la que más rápido delata a un junior en una revisión de código.

15.1 Estructura de un documento

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Título descriptivo de la página</title>
    <meta name="description" content="Resumen de 150 caracteres para buscadores">
    <link rel="stylesheet" href="estilos.css">
  </head>
  <body>
    <header>...</header>
    <nav>...</nav>
    <main>...</main>
    <footer>...</footer>
    <script src="app.js" type="module"></script>
  </body>
</html>
```

El atributo `lang="es"` no es decorativo: afecta a los lectores de pantalla, a la división de palabras y al corrector del navegador. La etiqueta `viewport` es lo que hace que una web se vea correctamente en móvil; sin ella, el móvil finge ser un escritorio y encoge todo.

15.2 Semántica: usar la etiqueta que significa lo que es

```
<!-- Mal: todo son divs, nada significa nada -->
<div class="cabecera">
  <div class="titulo">Mi blog</div>
  <div class="menu">
    <div onclick="ir('/inicio')">Inicio</div>
  </div>
</div>

<!-- Bien -->
<header>
  <h1>Mi blog</h1>
  <nav>
    <ul>
      <li><a href="/inicio">Inicio</a></li>
    </ul>
  </nav>
</header>
```

Etiqueta	Cuándo usarla

`<header>`	Cabecera de la página o de una sección
`<nav>`	Bloque de navegación principal
`<main>`	Contenido principal. Uno solo por página
`<article>`	Contenido con sentido propio: un post, una noticia, una ficha
`<section>`	Agrupación temática dentro de un documento
`<aside>`	Contenido secundario: barra lateral, notas
`<footer>`	Pie de página o de sección
`<button>`	Cualquier cosa clicable que ejecute una acción
`<a>`	Cualquier cosa clicable que lleve a otro sitio
`<figure>` / `<figcaption>`	Imagen o gráfico con su pie
`<time datetime="...">`	Fechas legibles por máquinas

LA REGLA DEL BOTÓN Y EL ENLACE

Si al pulsarlo **navegas**, es un `<a>`. Si al pulsarlo **pasa algo** (enviar, abrir un modal, borrar), es un `<button>`. Un `<div onclick>` no es ninguna de las dos: no se puede enfocar con el teclado, no responde a Enter, no lo anuncia un lector de pantalla y no aparece en el orden de tabulación. Es un fallo de accesibilidad y aparece literalmente en las pruebas técnicas.

15.3 Formularios

```
<form action="/registro" method="post">
  <div>
    <label for="email">Correo electrónico</label>
    <input
      type="email"
      id="email"
      name="email"
      required
      autocomplete="email"
      aria-describedby="ayuda-email"
    >
    <small id="ayuda-email">No lo compartiremos con nadie</small>
  </div>

  <div>
    <label for="pass">Contraseña</label>
    <input type="password" id="pass" name="password"
           minlength="8" required autocomplete="new-password">
  </div>

  <fieldset>
    <legend>Plan</legend>
    <label><input type="radio" name="plan" value="free" checked> Gratuito</label>
    <label><input type="radio" name="plan" value="pro"> Profesional</label>
  </fieldset>

  <button type="submit">Crear cuenta</button>
</form>
```

Puntos que importan: cada `input` necesita un `label` asociado por `for` / `id` (si no, el lector de pantalla no sabe qué se pide y además el clic en la etiqueta no enfoca el campo); el `type` correcto hace que el móvil muestre el teclado adecuado; y `autocomplete` es la diferencia entre un formulario cómodo y uno odioso.

15.4 Accesibilidad: el mínimo profesional

No es caridad, es requisito legal en el sector público español y europeo, y es criterio de calidad en cualquier equipo serio. Lo mínimo:

- **Imágenes con `alt`.** Descriptivo si aporta información; `alt=""` si es puramente decorativa.
- **Jerarquía de encabezados correcta.** Un solo `h1`, sin saltarse niveles. Muchos usuarios navegan saltando de encabezado en encabezado.
- **Contraste suficiente.** Mínimo 4,5:1 para texto normal (criterio WCAG AA).
- **Todo accesible con teclado.** Prueba a recorrer tu página solo con `Tab` y `Enter`. Si no puedes hacer algo, está roto.
- **Foco visible.** Nunca escribas `outline: none` sin poner una alternativa.
- **ARIA solo cuando haga falta.** La primera regla de ARIA es no usar ARIA: si existe un elemento HTML nativo que ya hace eso, úsalo.

EN LA ENTREVISTA

Mencionar accesibilidad sin que te pregunten es una de las formas más baratas de destacar como junior, porque casi nadie lo hace. Si en una prueba técnica añades etiquetas correctas, contraste y navegación por teclado, y lo explicas en el README, subes un escalón entero frente al resto de candidatos.

16. CSS moderno

16.1 El modelo de caja

```
/* Pon esto SIEMPRE al principio de tu CSS */
*, *::before, *::after {
  box-sizing: border-box;
}
```

Por defecto, si das a un elemento `width: 200px` y le añades `padding: 20px`, ocupa 240 píxeles. Con `border-box`, ocupa los 200 que pediste y el relleno va por dentro. Es lo que cualquier persona esperaría, y es la primera línea de cualquier hoja de estilos profesional.

16.2 Selectores y especificidad

```
.tarjeta { }                  /* clase: tu herramienta principal */
#unico { }                    /* id: evítalo para estilos */
.tarjeta .titulo { }          /* descendiente */
.tarjeta > .titulo { }        /* hijo directo */
.tarjeta:hover { }            /* estado */
.lista li:nth-child(odd) { }  /* posición */
input:focus-visible { }       /* foco por teclado, no por ratón */
.tarjeta:has(img) { }         /* el padre que contiene una imagen (moderno) */
```

La especificidad decide qué regla gana cuando dos afectan al mismo elemento: los estilos en línea ganan a los ids, que ganan a las clases, que ganan a las etiquetas. Peleas de especificidad e `!important` son síntoma de un CSS mal estructurado, no una solución.

16.3 Flexbox: una dimensión

Para organizar elementos en una fila o en una columna. Es lo que usas el 70 % de las veces.

```
.contenedor {
  display: flex;
  flex-direction: row;          /* row | column */
  justify-content: space-between; /* reparto en el eje principal */
  align-items: center;          /* alineación en el eje transversal */
  gap: 1rem;                    /* separación entre elementos */
  flex-wrap: wrap;              /* permitir salto de línea */
}

.elemento {
  flex: 1;                      /* crece para repartir el espacio libre */
  flex: 0 0 200px;              /* ancho fijo de 200px, no crece ni encoge */
}
```

```css
/* Los tres patrones que resuelven casi todo */

/* 1. Barra de navegación: logo a la izquierda, menú a la derecha */
.nav { display: flex; justify-content: space-between; align-items: center; }

/* 2. Centrado perfecto, vertical y horizontal */
.centro { display: flex; justify-content: center; align-items: center; min-height: 100vh; }

/* 3. Pie de página pegado abajo aunque haya poco contenido */
body { display: flex; flex-direction: column; min-height: 100vh; }
main { flex: 1; }
```

16.4 Grid: dos dimensiones

```css
.galeria {
  display: grid;
  grid-template-columns: repeat(3, 1fr);   /* tres columnas iguales */
  gap: 1rem;
}

/* Rejilla responsive sin una sola media query: el patrón más útil de CSS */
.galeria-auto {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
  gap: 1.5rem;
}

/* Maquetación por zonas con nombre */
.layout {
  display: grid;
  grid-template-areas:
    "cabecera cabecera"
    "lateral  principal"
    "pie      pie";
  grid-template-columns: 250px 1fr;
  min-height: 100vh;
}
.cabecera  { grid-area: cabecera; }
.lateral   { grid-area: lateral; }
.principal { grid-area: principal; }
.pie       { grid-area: pie; }
```

FLEXBOX O GRID

Flexbox cuando el contenido decide el tamaño y colocas en una línea. Grid cuando tú decides la estructura en filas y columnas. En la práctica se combinan: Grid para la maquetación general de la página, Flexbox dentro de cada componente.

16.5 Diseño adaptable

```
/* Móvil primero: escribes los estilos base para pantalla pequeña
   y vas añadiendo a medida que crece. Es el orden correcto. */

.contenedor { padding: 1rem; }

@media (min-width: 768px) {
  .contenedor { padding: 2rem; max-width: 720px; margin-inline: auto; }
}

@media (min-width: 1200px) {
  .contenedor { max-width: 1140px; }
}
```

```
/* Unidades: cuál usar */
rem     /* relativa al tamaño base del navegador. Para tipografía y espaciado */
em      /* relativa al elemento actual. Para espaciados internos proporcionales */
%       /* relativa al contenedor */
vw/vh   /* porcentaje de la ventana */
fr      /* fracción del espacio disponible (solo en grid) */
ch      /* ancho del carácter "0". Útil para limitar líneas de texto */

/* Tamaño fluido sin media queries */
h1 { font-size: clamp(1.75rem, 5vw, 3rem); }

/* Ancho de lectura cómodo */
p { max-width: 65ch; }
```

16.6 Variables CSS

```
:root {
  --color-principal: #1f5fd6;
  --color-texto: #1a1d23;
  --espacio: 1rem;
  --radio: 8px;
}

.boton {
  background: var(--color-principal);
  padding: var(--espacio) calc(var(--espacio) * 2);
  border-radius: var(--radio);
}

/* Modo oscuro casi gratis */
@media (prefers-color-scheme: dark) {
  :root {
    --color-texto: #e8eaed;
    --color-fondo: #16181d;
  }
}
```

16.7 Tailwind CSS

Tailwind aplica estilos mediante clases utilitarias directamente en el HTML. Es, con diferencia, lo que más se usa hoy en proyectos nuevos, y aparece cada vez más en ofertas de empleo.

```
<!-- CSS tradicional -->
<div class="tarjeta">...</div>
<!-- + 12 líneas en un fichero .css aparte -->

<!-- Tailwind -->
<div class="rounded-lg border border-gray-200 bg-white p-6 shadow-sm
            hover:shadow-md transition-shadow">
  <h3 class="text-lg font-semibold text-gray-900">Título</h3>
  <p class="mt-2 text-sm text-gray-600">Descripción</p>
</div>

<!-- Responsive: prefijos por punto de ruptura -->
<div class="grid grid-cols-1 gap-4 md:grid-cols-2 lg:grid-cols-3"></div>
```

La objeción habitual ("ensucia el HTML") tiene respuesta: no hay que inventar nombres de clase, no hay CSS muerto acumulándose, los estilos viven junto al marcado que afectan y el sistema de diseño (espaciados, colores, tamaños) viene impuesto y es coherente. Aprende CSS de verdad primero, y usa Tailwind después: sin la base, Tailwind es memorizar abreviaturas sin entender qué hacen.

EJERCICIOS · CAPÍTULOS 15–16

1. Maqueta una página de perfil: cabecera con avatar, tres tarjetas de estadísticas y una lista. Responsive, sin librerías.
2. Reproduce la maquetación de una web que uses a diario, solo con Grid y Flexbox.
3. Recorre tu página solo con teclado y corrige todo lo que no sea alcanzable.
4. Repite el primer ejercicio con Tailwind y compara el tiempo que tardas.

17. React: el modelo mental

React es la librería de interfaces dominante en las ofertas españolas. Su idea central es una sola: la interfaz es una función del estado. Si entiendes eso, lo demás es sintaxis.

17.1 La idea

Antes de React, se programaba la interfaz dando instrucciones paso a paso: "busca este elemento, cambia su texto, añade esta clase, oculta aquello". Con veinte estados posibles, eso se vuelve inmanejable.

React invierte el planteamiento: tú **describes** cómo debe verse la interfaz para cada estado posible, y cuando el estado cambia, React se encarga de actualizar el DOM por ti, calculando la diferencia mínima.

```
interfaz = f(estado)
```

17.2 Componentes y JSX

```
function Saludo({ nombre, edad }) {
  return (
    <div className="saludo">
      <h2>Hola, {nombre}</h2>
      {edad >= 18 && <p>Eres mayor de edad</p>}
      <p>{edad >= 18 ? "Puedes pasar" : "Ven con un adulto"}</p>
    </div>
  );
}

// Uso
<Saludo nombre="Ana" edad={30} />
```

JSX es JavaScript con sintaxis parecida a HTML. Reglas que dan problemas al principio: se usa `className` en vez de `class`, los atributos van en *camelCase* (`onClick`, `htmlFor`), todo debe devolverse dentro de un único elemento raíz (o de un fragmento `<>...</>`), y las expresiones JavaScript van entre llaves.

17.3 Estado: useState

```
import { useState } from "react";

function Contador() {
  const [cuenta, setCuenta] = useState(0);

  return (
    <div>
      <p>Llevas {cuenta} clics</p>
      <button onClick={() => setCuenta(cuenta + 1)}>Sumar</button>

      {/* Si el nuevo valor depende del anterior, usa la forma de función.
          Es lo correcto siempre que haya actualizaciones seguidas. */}
      <button onClick={() => setCuenta((c) => c + 1)}>Sumar bien</button>
      <button onClick={() => setCuenta(0)}>Reiniciar</button>
    </div>
  );
}
```

EL ESTADO ES INMUTABLE

Nunca modifiques el estado directamente. React compara referencias para decidir si debe repintar; si mutas el objeto, la referencia no cambia y la interfaz no se actualiza.

```
// Mal
tareas.push(nueva);
setTareas(tareas);              // no repinta

// Bien
setTareas([...tareas, nueva]);
setTareas(tareas.filter((t) => t.id !== id));
setTareas(tareas.map((t) => (t.id === id ? { ...t, hecha: true } : t)));
setUsuario({ ...usuario, email: nuevoEmail });
```

17.4 Listas y claves

```
function ListaTareas({ tareas }) {
  return (
    <ul>
      {tareas.map((tarea) => (
        <li key={tarea.id}>{tarea.titulo}</li>
      ))}
    </ul>
  );
}
```

La `key` debe ser un identificador estable y único del elemento. **No uses el índice del array** si la lista se puede reordenar, filtrar o modificar: React usará las claves para emparejar elementos entre renders y con índices acabarás con estados que saltan de una fila a otra. Es uno de los errores más comunes y se pregunta en entrevistas.

17.5 Efectos: useEffect

```
import { useEffect, useState } from "react";

function PerfilUsuario({ id }) {
  const [usuario, setUsuario] = useState(null);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const controlador = new AbortController();

    async function cargar() {
      setCargando(true);
      setError(null);
      try {
        const r = await fetch(`/api/usuarios/${id}`, { signal: controlador.signal });
        if (!r.ok) throw new Error("No se pudo cargar el usuario");
        setUsuario(await r.json());
      } catch (e) {
        if (e.name !== "AbortError") setError(e.message);
      } finally {
        setCargando(false);
      }
    }

    cargar();
    return () => controlador.abort();     // limpieza al desmontar o cambiar id
  }, [id]);                                // se vuelve a ejecutar si cambia id

  if (cargando) return <p>Cargando…</p>;
  if (error) return <p role="alert">{error}</p>;
  return <h1>{usuario.nombre}</h1>;
}
```

El array de dependencias controla cuándo se vuelve a ejecutar el efecto: vacío significa "solo al montar"; con valores, "cuando alguno de estos cambie"; sin array, "después de cada render" (casi nunca es lo que quieres). La función que devuelves es la limpieza, y es lo que evita fugas de memoria y peticiones obsoletas.

> **USEEFFECT SE USA MENOS DE LO QUE CREES**
>
> Un error muy extendido es usar efectos para cosas que no lo necesitan. No hace falta un efecto para transformar datos de cara al render (calcúlalo directamente durante el render), ni para responder a un evento del usuario (ponlo en el manejador). Los efectos son para *sincronizar con algo externo a React*: la red, el DOM, un temporizador, una suscripción.

17.6 Formularios controlados

```
function FormularioLogin({ onEnviar }) {
  const [datos, setDatos] = useState({ email: "", password: "" });
  const [enviando, setEnviando] = useState(false);

  function cambiar(e) {
    const { name, value } = e.target;
    setDatos((d) => ({ ...d, [name]: value }));
  }

  async function enviar(e) {
    e.preventDefault();
    setEnviando(true);
    try {
      await onEnviar(datos);
    } finally {
      setEnviando(false);
    }
  }

  return (
    <form onSubmit={enviar}>
      <label htmlFor="email">Email</label>
      <input id="email" name="email" type="email" value={datos.email}
             onChange={cambiar} required />

      <label htmlFor="password">Contraseña</label>
      <input id="password" name="password" type="password" value={datos.password}
             onChange={cambiar} required />

      <button disabled={enviando}>{enviando ? "Entrando…" : "Entrar"}</button>
    </form>
  );
}
```

17.7 Comunicación entre componentes

Los datos bajan por *props* y los eventos suben por funciones. Cuando dos componentes hermanos necesitan compartir estado, ese estado se "eleva" al padre común.

```
function Padre() {
  const [filtro, setFiltro] = useState("");

  return (
    <>
      <Buscador valor={filtro} onCambio={setFiltro} />
      <Resultados filtro={filtro} />
    </>
  );
}
```

Cuando un dato lo necesita medio árbol de componentes (el usuario autenticado, el tema visual, el idioma), se usa **Context** para no ir pasándolo por diez niveles:

```
const ContextoAuth = createContext(null);

export function ProveedorAuth({ children }) {
  const [usuario, setUsuario] = useState(null);
  return (
    <ContextoAuth.Provider value={{ usuario, setUsuario }}>
      {children}
    </ContextoAuth.Provider>
  );
}

export const useAuth = () => useContext(ContextoAuth);
```

17.8 Hooks propios

Un hook propio es simplemente una función que empieza por `use` y llama a otros hooks. Sirve para reutilizar lógica con estado entre componentes.

```
function useFetch(url) {
  const [datos, setDatos] = useState(null);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let cancelado = false;
    setCargando(true);

    fetch(url)
      .then((r) => {
        if (!r.ok) throw new Error(`Error ${r.status}`);
        return r.json();
      })
      .then((d) => { if (!cancelado) setDatos(d); })
      .catch((e) => { if (!cancelado) setError(e.message); })
      .finally(() => { if (!cancelado) setCargando(false); });

    return () => { cancelado = true; };
  }, [url]);

  return { datos, cargando, error };
}

// Uso: tres líneas en cualquier componente
const { datos: productos, cargando, error } = useFetch("/api/productos");
```

EN LA ENTREVISTA

Preguntas habituales sobre React: *por qué hacen falta las keys*; *qué es el estado y por qué debe ser inmutable*; *para qué sirve el array de dependencias de useEffect*; *diferencia entre estado y props*; *cuándo usarías Context y cuándo no*. Todas están respondidas en este capítulo; ensáyalas en voz alta.

18. React en un proyecto real

18.1 Arrancar el proyecto

```
npm create vite@latest mi-app -- --template react-ts
cd mi-app
npm install
npm run dev
```

Vite es la herramienta estándar hoy para proyectos React sin framework completo: arranca en menos de un segundo y recarga los cambios al instante. Si el proyecto necesita renderizado en servidor, rutas por fichero y optimización para buscadores, la opción es **Next.js**, que también aparece mucho en ofertas.

18.2 Estructura de carpetas

```
src/
  components/      # componentes reutilizables y tontos (Boton, Modal, Tabla)
  features/        # agrupado por funcionalidad, no por tipo de fichero
    productos/
      ListaProductos.tsx
      FichaProducto.tsx
      useProductos.ts
      api.ts
  hooks/           # hooks genéricos
  lib/             # cliente http, utilidades
  pages/           # una por ruta
  types/           # tipos compartidos
  App.tsx
  main.tsx
```

Organizar por funcionalidad (`features/`) en vez de por tipo de fichero escala mucho mejor: todo lo relacionado con "productos" está junto, y borrar una funcionalidad es borrar una carpeta.

18.3 Rutas

```
import { createBrowserRouter, RouterProvider, Link, useParams } from "react-router-dom";

const router = createBrowserRouter([
  { path: "/", element: <Inicio /> },
  { path: "/productos", element: <ListaProductos /> },
  { path: "/productos/:id", element: <FichaProducto /> },
  { path: "*", element: <NoEncontrado /> },
]);

function FichaProducto() {
  const { id } = useParams();
  // ...
}
```

18.4 Obtención de datos en serio

El `useFetch` del capítulo anterior sirve para aprender. En proyectos reales se usa **TanStack Query**, que resuelve caché, reintentos, revalidación, estados de carga y sincronización entre pestañas:

```
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";

function ListaProductos() {
  const { data, isLoading, error } = useQuery({
    queryKey: ["productos"],
    queryFn: () => api("/productos"),
    staleTime: 60_000,
  });

  if (isLoading) return <Esqueleto />;
  if (error) return <Error mensaje={error.message} />;
  return <ul>{data.map((p) => <li key={p.id}>{p.nombre}</li>)}</ul>;
}

function useCrearProducto() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: (nuevo) => api("/productos", { method: "POST", body: JSON.stringify(nuevo) }),
    onSuccess: () => qc.invalidateQueries({ queryKey: ["productos"] }),
  });
}
```

18.5 Rendimiento: lo que de verdad importa

- **No optimices antes de medir.** Usa React DevTools Profiler para saber qué se está repintando y por qué.
- **Carga diferida de rutas** con `React.lazy` y `Suspense`: reduce el tamaño del paquete inicial, que es lo que nota el usuario.
- **`useMemo` y `useCallback`** solo para cálculos caros o para mantener referencias estables. Usarlos en todas partes empeora el rendimiento y la legibilidad.
- **Listas largas:** virtualización con `@tanstack/react-virtual` a partir de unos cientos de elementos.

18.6 Errores típicos de junior en React

Error	Consecuencia	Solución
Mutar el estado	La interfaz no se actualiza	Crear copias nuevas
Usar el índice como `key`	Estado y foco se mezclan al reordenar	Usar identificadores reales
Dependencias mal puestas en `useEffect`	Bucles infinitos o datos obsoletos	Activar la regla de ESLint y hacerle caso
Componentes de 500 líneas	Imposible de probar y mantener	Extraer componentes y hooks
Efecto para derivar datos	Renders de más y estados desincronizados	Calcular durante el render

Estado global para todo	Repintados innecesarios y acoplamiento	Estado tan local como sea posible
Sin estados de carga ni error	Pantallas en blanco y fallos silenciosos	Los tres estados siempre: cargando, error, datos

EJERCICIOS · CAPÍTULOS 17–18

1. Lista de tareas con añadir, marcar, borrar, filtrar y persistencia en `localStorage`.
2. Buscador que consulta una API pública con *debounce* de 300 ms y muestra los tres estados.
3. Extrae un hook `useLocalStorage` y reutilízalo en los dos ejercicios anteriores.
4. Aplicación con tres rutas, navegación y una ruta con parámetro que carga un detalle.

PARTE 4

Backend: donde está el negocio

La parte que el usuario no ve y en la que vive todo lo importante: los datos, las reglas y la seguridad. También es donde están los sueldos más altos.

- 19. Node y Express: tu primera API
- 20. Bases de datos y SQL
- 21. Modelado de datos y Prisma
- 22. Autenticación y seguridad
- 23. Diseño de APIs que no dan vergüenza

19. Node y Express: tu primera API

19.1 Qué es Node

Node.js es JavaScript ejecutándose fuera del navegador, sobre el motor V8 de Chrome. Te da acceso al sistema de ficheros, a la red y a procesos: todo lo que el navegador prohíbe por seguridad. Es lo que permite usar un solo lenguaje en las dos mitades del stack.

Su característica definitoria es que es **no bloqueante**: mientras espera al disco o a la red, atiende otras peticiones. Por eso maneja bien miles de conexiones simultáneas con un solo hilo, y por eso encaja tan bien con APIs y servicios de red.

19.2 Un servidor mínimo

```
// src/server.js
import express from "express";

const app = express();
app.use(express.json());              // parsea cuerpos JSON

app.get("/salud", (req, res) => {
  res.json({ estado: "ok", hora: new Date().toISOString() });
});

const PUERTO = process.env.PORT ?? 3000;
app.listen(PUERTO, () => console.log(`Escuchando en http://localhost:${PUERTO}`));
```

```
npm init -y
npm pkg set type=module
npm install express
npm install -D tsx
node --watch src/server.js
```

19.3 Rutas y parámetros

```
// Parámetro de ruta:      /productos/42
app.get("/productos/:id", (req, res) => {
  const { id } = req.params;
});

// Parámetros de consulta: /productos?categoria=libros&pagina=2
app.get("/productos", (req, res) => {
  const { categoria, pagina = 1, limite = 20 } = req.query;
});

// Cuerpo de la petición
app.post("/productos", (req, res) => {
  const { nombre, precio } = req.body;
  res.status(201).json({ id: 1, nombre, precio });
});

// Cabeceras
app.get("/privado", (req, res) => {
  const token = req.headers.authorization?.replace("Bearer ", "");
});
```

19.4 Middleware

Un middleware es una función que se ejecuta entre la petición y la respuesta. Puede leerlas, modificarlas, cortarlas o dejarlas pasar al siguiente. Es el mecanismo central de Express y el concepto que hay que entender bien.

```
// Middleware de registro
app.use((req, res, next) => {
  const inicio = Date.now();
  res.on("finish", () => {
    console.log(`${req.method} ${req.originalUrl} ${res.statusCode} ${Date.now() - inicio}ms`);
  });
  next();                       // sin next(), la petición se queda colgada
});

// Middleware de autenticación
function requiereAuth(req, res, next) {
  const token = req.headers.authorization?.split(" ")[1];
  if (!token) return res.status(401).json({ mensaje: "No autenticado" });

  try {
    req.usuario = verificarToken(token);
    next();
  } catch {
    res.status(401).json({ mensaje: "Token inválido" });
  }
}

app.get("/perfil", requiereAuth, (req, res) => {
  res.json(req.usuario);
});

// Gestor de errores: SIEMPRE el último, y con cuatro parámetros
app.use((err, req, res, next) => {
  console.error(err);
  const status = err.status ?? 500;
  res.status(status).json({
    mensaje: status === 500 ? "Error interno del servidor" : err.message,
  });
});
```

NUNCA DEVUELVAS EL ERROR REAL EN UN 500

Un mensaje como `"connect ECONNREFUSED 10.0.3.14:5432"` le está diciendo a un atacante la IP y el puerto de tu base de datos. Registra el error completo en el servidor y devuelve un mensaje genérico al cliente. Esta distinción —qué se registra frente a qué se responde— se valora mucho en una revisión técnica.

19.5 Estructura en capas

```
// routes/productos.js -- solo define rutas
import { Router } from "express";
import * as controlador from "../controllers/productos.js";

const router = Router();
router.get("/", controlador.listar);
router.get("/:id", controlador.obtener);
router.post("/", requiereAuth, controlador.crear);
export default router;

// controllers/productos.js -- traduce HTTP a casos de uso
import * as servicio from "../services/productos.js";

export async function listar(req, res, next) {
  try {
    const { pagina = 1, limite = 20 } = req.query;
    const resultado = await servicio.listarProductos({ pagina: +pagina, limite: +limite });
    res.json(resultado);
  } catch (e) { next(e); }
}

// services/productos.js -- la lógica de negocio. Nada de HTTP aquí.
import * as repo from "../repositories/productos.js";

export async function listarProductos({ pagina, limite }) {
  if (limite > 100) limite = 100;                 // regla de negocio
  const [items, total] = await Promise.all([
    repo.buscar({ salto: (pagina - 1) * limite, limite }),
    repo.contar(),
  ]);
  return { items, total, pagina, paginas: Math.ceil(total / limite) };
}
```

Fíjate en que `services/productos.js` no importa nada de Express. Esa es la prueba de fuego: si tu lógica de negocio conoce a `req` y a `res`, no puedes probarla sin levantar un servidor, y has perdido.

19.6 Validación de entrada

```
import { z } from "zod";

const EsquemaProducto = z.object({
  nombre: z.string().min(1).max(200),
  precio: z.number().int().positive(),        // en céntimos
  categoria: z.enum(["libro", "musica", "video"]),
  descripcion: z.string().max(2000).optional(),
});

export function validar(esquema) {
  return (req, res, next) => {
    const resultado = esquema.safeParse(req.body);
    if (!resultado.success) {
      return res.status(422).json({
        mensaje: "Datos inválidos",
        errores: resultado.error.issues.map((i) => ({
          campo: i.path.join("."),
          error: i.message,
        })),
      });
    }
    req.body = resultado.data;        // datos ya limpios y tipados
    next();
  };
}

router.post("/", requiereAuth, validar(EsquemaProducto), controlador.crear);
```

EJERCICIOS · CAPÍTULO 19

1. Monta una API con CRUD completo de "notas" guardadas en memoria, con los cinco verbos y los códigos de estado correctos.
2. Añade middleware de registro, validación con Zod y gestor de errores centralizado.
3. Prueba todos los endpoints con `curl` y documenta cada uno en el README.
4. Añade paginación a la lista y devuelve el total de elementos y de páginas.

20. Bases de datos y SQL

SQL aparece en casi mil ofertas al mes en España y es transversal a cualquier stack. Es la habilidad con mejor relación entre esfuerzo de aprendizaje y demanda de todo el manual.

20.1 Relacional frente a documental

	Relacional (PostgreSQL, MySQL)	Documental (MongoDB)
Estructura	Tablas con esquema fijo	Documentos JSON flexibles
Relaciones	Nativas, con integridad garantizada	Manuales, sin garantías
Consultas	SQL, muy potente	Más limitado en consultas complejas
Cuándo	Casi siempre. Es el valor por defecto correcto	Datos muy heterogéneos o de esquema cambiante

Consejo directo: empieza por PostgreSQL y no te plantees otra cosa hasta que tengas un motivo concreto. Es lo que más se pide, es gratis, y aprender SQL te sirve para el resto de tu carrera independientemente del lenguaje que uses.

20.2 Crear tablas

```sql
CREATE TABLE usuarios (
  id          SERIAL PRIMARY KEY,
  email       VARCHAR(255) NOT NULL UNIQUE,
  nombre      VARCHAR(100) NOT NULL,
  password    VARCHAR(255) NOT NULL,
  rol         VARCHAR(20)  NOT NULL DEFAULT 'usuario',
  creado_en   TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE TABLE pedidos (
  id          SERIAL PRIMARY KEY,
  usuario_id  INTEGER NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
  total       INTEGER NOT NULL CHECK (total >= 0),   -- en céntimos
  estado      VARCHAR(20) NOT NULL DEFAULT 'pendiente',
  creado_en   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_pedidos_usuario ON pedidos(usuario_id);
```

Cada restricción de esa tabla evita una categoría entera de errores: `NOT NULL` impide filas incompletas, `UNIQUE` impide dos cuentas con el mismo correo, `REFERENCES` impide pedidos de usuarios inexistentes, `CHECK` impide totales negativos. La base de datos es tu última línea de defensa y conviene que sea estricta.

20.3 Consultas

```sql
-- Leer
SELECT id, nombre, email FROM usuarios WHERE rol = 'admin';
SELECT * FROM productos WHERE precio BETWEEN 1000 AND 5000;
SELECT * FROM usuarios WHERE nombre ILIKE '%ana%';     -- sin distinguir mayúsculas
SELECT * FROM pedidos WHERE estado IN ('pendiente', 'pagado');
SELECT * FROM usuarios WHERE creado_en >= NOW() - INTERVAL '30 days';

-- Ordenar y limitar
SELECT * FROM productos ORDER BY precio DESC LIMIT 10 OFFSET 20;

-- Escribir
INSERT INTO usuarios (email, nombre, password)
VALUES ('ana@x.com', 'Ana', '$2b$10$...')
RETURNING id, creado_en;

UPDATE pedidos SET estado = 'enviado' WHERE id = 42;
DELETE FROM pedidos WHERE estado = 'cancelado' AND creado_en < NOW() - INTERVAL '1 year';
```

UPDATE Y DELETE SIN WHERE

`DELETE FROM usuarios;` borra la tabla entera y no hay deshacer. Costumbre que salva carreras: escribe primero la consulta como `SELECT` con el mismo `WHERE`, comprueba cuántas filas salen, y solo entonces cámbiala a `DELETE` o `UPDATE`.

20.4 JOIN: la parte que separa a quien sabe SQL de quien no

```sql
-- INNER JOIN: solo las filas que casan en ambas tablas
SELECT p.id, p.total, u.nombre, u.email
FROM pedidos p
INNER JOIN usuarios u ON u.id = p.usuario_id
WHERE p.estado = 'pagado';

-- LEFT JOIN: todas las de la izquierda, aunque no tengan pareja
SELECT u.nombre, COUNT(p.id) AS num_pedidos
FROM usuarios u
LEFT JOIN pedidos p ON p.usuario_id = u.id
GROUP BY u.id, u.nombre
ORDER BY num_pedidos DESC;
-- Con INNER JOIN, los usuarios sin pedidos desaparecerían del resultado

-- Varias tablas
SELECT u.nombre, pr.nombre AS producto, lp.cantidad
FROM pedidos pe
JOIN usuarios u ON u.id = pe.usuario_id
JOIN lineas_pedido lp ON lp.pedido_id = pe.id
JOIN productos pr ON pr.id = lp.producto_id
WHERE pe.creado_en >= '2026-01-01';
```

20.5 Agregación

```sql
SELECT
  DATE_TRUNC('month', creado_en) AS mes,
  COUNT(*)                       AS num_pedidos,
  SUM(total)                     AS facturacion,
  ROUND(AVG(total))              AS ticket_medio,
  MAX(total)                     AS mayor_pedido
FROM pedidos
WHERE estado = 'pagado'
GROUP BY mes
HAVING COUNT(*) > 5
ORDER BY mes DESC;
```

La diferencia entre `WHERE` y `HAVING` es de examen y de entrevista: `WHERE` filtra filas *antes* de agrupar; `HAVING` filtra grupos *después* de agrupar.

20.6 Índices y rendimiento

Un índice es una estructura auxiliar que permite encontrar filas sin recorrer la tabla entera. Sin índice, buscar un email entre un millón de usuarios implica leer un millón de filas; con índice, una decena de saltos.

```sql
CREATE INDEX idx_usuarios_email ON usuarios(email);
CREATE INDEX idx_pedidos_usuario_estado ON pedidos(usuario_id, estado);

-- Ver qué plan usa la base de datos: la herramienta fundamental
EXPLAIN ANALYZE
SELECT * FROM pedidos WHERE usuario_id = 42 AND estado = 'pagado';
```

Cuándo crear un índice: en columnas que aparecen en `WHERE`, `JOIN` u `ORDER BY` de consultas frecuentes. Cuándo no: en tablas pequeñas, en columnas con muy pocos valores distintos, o "por si acaso" en todas. Cada índice acelera las lecturas y ralentiza las escrituras, y ocupa espacio.

> **EN LA ENTREVISTA**
>
> El problema N+1 se pregunta constantemente. Es cuando haces una consulta para obtener N pedidos y después una consulta por cada pedido para obtener su usuario: 1 + N consultas en lugar de una. La solución es un `JOIN` o una carga anticipada (`include` en Prisma). Saber detectarlo y nombrarlo es un indicador claro de nivel.

20.7 Transacciones

```sql
BEGIN;
  UPDATE cuentas SET saldo = saldo - 10000 WHERE id = 1;
  UPDATE cuentas SET saldo = saldo + 10000 WHERE id = 2;
COMMIT;
-- Si algo falla entre medias: ROLLBACK, y no ha pasado nada
```

Una transacción garantiza que un conjunto de operaciones ocurre *entero o nada*. Es lo que impide que el dinero salga de una cuenta y no llegue a la otra. Se resume con el acrónimo ACID: atomicidad, consistencia, aislamiento y durabilidad.

EJERCICIOS · CAPÍTULO 20

1. Levanta PostgreSQL con Docker y crea el esquema de una tienda: usuarios, productos, pedidos y líneas de pedido.
2. Inserta datos de prueba y escribe diez consultas: total facturado por mes, top 5 de clientes, productos nunca vendidos, ticket medio por categoría.
3. Ejecuta `EXPLAIN ANALYZE` sobre una consulta lenta, crea el índice adecuado y compara los tiempos.
4. Escribe una transacción que cree un pedido y descuente stock, y comprueba que un fallo intermedio deshace todo.

21. Modelado de datos y Prisma

21.1 Normalización, en dos minutos

Normalizar es evitar repetir la misma información en varios sitios, porque cada duplicado es una oportunidad de que las copias se contradigan.

```
-- Mal: la dirección del cliente repetida en cada pedido.
-- Si el cliente se muda, ¿qué pedidos actualizas?
pedidos(id, cliente_nombre, cliente_email, cliente_direccion, total)

-- Bien
clientes(id, nombre, email, direccion)
pedidos(id, cliente_id, total)
```

Excepción importante: los datos históricos se copian a propósito. La dirección de envío de un pedido de hace dos años debe ser la de entonces, no la actual. Eso no es desnormalizar mal, es modelar el tiempo correctamente.

21.2 Tipos de relación

Relación	Cómo se implementa	Ejemplo
Uno a uno	Clave foránea única en una de las tablas	Usuario ↔ Perfil
Uno a muchos	Clave foránea en el lado "muchos"	Usuario → Pedidos
Muchos a muchos	Tabla intermedia con ambas claves	Productos ↔ Etiquetas

21.3 Prisma

```
npm install prisma @prisma/client
npx prisma init
```

```
// prisma/schema.prisma
generator client { provider = "prisma-client-js" }

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Usuario {
  id        Int      @id @default(autoincrement())
  email     String   @unique
  nombre    String
  password  String
  rol       Rol      @default(USUARIO)
  pedidos   Pedido[]
  creadoEn  DateTime @default(now())
}

model Pedido {
  id         Int          @id @default(autoincrement())
  usuario    Usuario      @relation(fields: [usuarioId], references: [id], onDelete: Cascade)
  usuarioId  Int
  total      Int
  estado     EstadoPedido @default(PENDIENTE)
  lineas     LineaPedido[]
  creadoEn   DateTime     @default(now())

  @@index([usuarioId, estado])
}

enum Rol { ADMIN USUARIO }
enum EstadoPedido { PENDIENTE PAGADO ENVIADO CANCELADO }
```

```
npx prisma migrate dev --name inicial    # crea y aplica la migración
npx prisma studio                         # interfaz visual para ver los datos
```

21.4 Consultas con Prisma

```
import { PrismaClient } from "@prisma/client";
const prisma = new PrismaClient();

// Crear
const usuario = await prisma.usuario.create({
  data: { email: "ana@x.com", nombre: "Ana", password: hash },
});

// Buscar con relaciones (evita el problema N+1)
const pedidos = await prisma.pedido.findMany({
  where: { estado: "PAGADO", creadoEn: { gte: new Date("2026-01-01") } },
  include: { usuario: { select: { nombre: true, email: true } }, lineas: true },
  orderBy: { creadoEn: "desc" },
  take: 20,
  skip: 0,
});

// Actualizar
await prisma.pedido.update({ where: { id: 42 }, data: { estado: "ENVIADO" } });

// Agregar
const stats = await prisma.pedido.aggregate({
  where: { estado: "PAGADO" },
  _sum: { total: true },
  _avg: { total: true },
  _count: true,
});

// Transacción
await prisma.$transaction(async (tx) => {
  const pedido = await tx.pedido.create({ data: { usuarioId, total } });
  await tx.producto.update({
    where: { id: productoId },
    data: { stock: { decrement: cantidad } },
  });
  return pedido;
});
```

APRENDE SQL ANTES QUE EL ORM

Un ORM como Prisma te ahorra escribir SQL, pero no te exime de entenderlo. Cuando una consulta vaya lenta en producción, vas a tener que mirar el SQL que genera y leer un plan de ejecución. Los desarrolladores que solo saben el ORM se quedan bloqueados exactamente en ese punto, que además es donde el problema es urgente.

22. Autenticación y seguridad

22.1 Contraseñas

```
import bcrypt from "bcrypt";

// Al registrar
const hash = await bcrypt.hash(password, 12);
await prisma.usuario.create({ data: { email, password: hash } });

// Al entrar
const usuario = await prisma.usuario.findUnique({ where: { email } });
const valida = usuario && await bcrypt.compare(password, usuario.password);

if (!valida) {
  // Mismo mensaje tanto si el email no existe como si la clave falla:
  // decir "ese email no existe" permite enumerar usuarios registrados.
  return res.status(401).json({ mensaje: "Credenciales incorrectas" });
}
```

REGLAS ABSOLUTAS SOBRE CONTRASEÑAS

- Nunca se guardan en claro. Nunca. Bajo ninguna circunstancia.
- Nunca se cifran de forma reversible: se les aplica una función hash lenta y con sal (bcrypt, argon2).
- Nunca se escriben en un log, ni siquiera durante la depuración.
- Nunca se envían por correo electrónico.

22.2 Sesiones frente a JWT

	Sesión con cookie	JSON Web Token
Dónde vive el estado	En el servidor	En el propio token
Revocar acceso	Inmediato: borras la sesión	Difícil: hay que esperar a que caduque o mantener una lista negra
Escalado horizontal	Requiere almacén compartido (Redis)	Sin estado, escala trivialmente
Cuándo usarlo	Aplicación web clásica con un solo backend	APIs, microservicios, clientes móviles

```
import jwt from "jsonwebtoken";

// Emitir
const token = jwt.sign(
  { sub: usuario.id, rol: usuario.rol },
  process.env.JWT_SECRET,
  { expiresIn: "15m" },
);

// Verificar
const carga = jwt.verify(token, process.env.JWT_SECRET);
```

El patrón profesional combina un token de acceso de vida corta (15 minutos) con un token de refresco de vida larga guardado en una cookie `httpOnly`. Así, si roban el token de acceso, la ventana de daño es pequeña, y el de refresco no es accesible desde JavaScript.

22.3 Autenticación y autorización

```
// Autenticación: ¿quién eres?
function requiereAuth(req, res, next) { /* verifica el token */ }

// Autorización: ¿puedes hacer esto?
function requiereRol(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.usuario.rol)) {
      return res.status(403).json({ mensaje: "Sin permiso" });
    }
    next();
  };
}

router.delete("/usuarios/:id", requiereAuth, requiereRol("ADMIN"), borrar);
```

EL FALLO DE SEGURIDAD NÚMERO UNO DEL SECTOR

Según el OWASP Top 10, el riesgo más frecuente es el *control de acceso roto*: comprobar que el usuario está identificado, pero no que el recurso al que accede es suyo.

```
// VULNERABLE: cualquier usuario autenticado ve pedidos ajenos
app.get("/pedidos/:id", requiereAuth, async (req, res) => {
  res.json(await prisma.pedido.findUnique({ where: { id: +req.params.id } }));
});

// CORRECTO
app.get("/pedidos/:id", requiereAuth, async (req, res) => {
  const pedido = await prisma.pedido.findFirst({
    where: { id: +req.params.id, usuarioId: req.usuario.id },
  });
  if (!pedido) return res.status(404).json({ mensaje: "No encontrado" });
  res.json(pedido);
});
```

22.4 Vulnerabilidades que debes conocer por su nombre

Ataque	En qué consiste	Defensa
Inyección SQL	Meter SQL dentro de un dato de entrada	Consultas parametrizadas o un ORM. Nunca concatenar cadenas.
XSS	Inyectar JavaScript que se ejecuta en el navegador de otro usuario	Escapar la salida; React lo hace por defecto salvo con `dangerouslySetInnerHTML`
CSRF	Hacer que el navegador de la víctima envíe una petición autenticada sin que lo sepa	Tokens CSRF, cookies `SameSite`
Control de acceso roto	Acceder a recursos de otros cambiando un id en la URL	Comprobar propiedad en cada consulta
Exposición de secretos	Claves de API subidas al repositorio	Variables de entorno y `.gitignore`
Dependencias vulnerables	Usar librerías con fallos conocidos	`npm audit`, Dependabot

```
-- Inyección SQL: el ejemplo clásico
-- Si password viene del usuario y vale:  ' OR '1'='1
"SELECT * FROM usuarios WHERE email = '" + email + "' AND password = '" + password + "'"
-- ...la consulta se convierte en siempre verdadera y entra sin contraseña.

-- Parametrizada: el valor nunca se interpreta como SQL
db.query("SELECT * FROM usuarios WHERE email = $1 AND password = $2", [email, password]);
```

22.5 Higiene básica de un backend

```
import helmet from "helmet";
import rateLimit from "express-rate-limit";
import cors from "cors";

app.use(helmet());                                    // cabeceras de seguridad
app.use(cors({ origin: process.env.FRONTEND_URL }));  // solo tu frontend
app.use(express.json({ limit: "100kb" }));            // limitar tamaño del cuerpo

app.use("/api/auth", rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 10,                                            // 10 intentos cada 15 min
  message: { mensaje: "Demasiados intentos" },
}));
```

23. Diseño de APIs

23.1 Convenciones REST

Operación	Verbo y ruta	Respuesta
Listar productos	`GET /productos`	200 + array paginado
Obtener uno	`GET /productos/42`	200 + objeto, o 404
Crear	`POST /productos`	201 + objeto creado + cabecera `Location`
Reemplazar	`PUT /productos/42`	200 + objeto
Modificar parcialmente	`PATCH /productos/42`	200 + objeto
Borrar	`DELETE /productos/42`	204 sin cuerpo
Subrecurso	`GET /usuarios/7/pedidos`	200 + array

Reglas: sustantivos en plural, nunca verbos en la ruta (`/productos/42` , no `/obtenerProducto?id=42`), jerarquía para relaciones, y filtros y ordenación en la cadena de consulta.

23.2 Formato de respuesta coherente

```
// Lista paginada
{
  "items": [ ... ],
  "meta": { "pagina": 2, "limite": 20, "total": 137, "paginas": 7 }
}

// Error: siempre la misma forma, en toda la API
{
  "mensaje": "Datos inválidos",
  "codigo": "VALIDATION_ERROR",
  "errores": [
    { "campo": "email", "error": "Formato no válido" },
    { "campo": "precio", "error": "Debe ser positivo" }
  ]
}
```

La coherencia importa más que la elección concreta: si el frontend puede escribir un único manejador de errores para toda la API, has diseñado bien.

23.3 Paginación, filtrado y ordenación

```
GET /productos?pagina=2&limite=20&orden=-precio&categoria=libro&buscar=redes

// Interpretación en el servidor
const pagina = Math.max(1, +req.query.pagina || 1);
const limite = Math.min(100, Math.max(1, +req.query.limite || 20));  // tope duro
const [campo, direccion] = req.query.orden?.startsWith("-")
  ? [req.query.orden.slice(1), "desc"]
  : [req.query.orden ?? "creadoEn", "asc"];
```

El tope duro en el límite no es un detalle: sin él, alguien pedirá `?limite=1000000` y tumbará tu servidor, a veces sin mala intención.

23.4 Documentación

Una API sin documentación no existe. El estándar es OpenAPI (antes Swagger), y se puede generar automáticamente desde los esquemas de Zod. Para un proyecto de portfolio, un README con la lista de endpoints, un ejemplo de petición y otro de respuesta por cada uno ya te distingue del 90 % de los candidatos.

EN LA ENTREVISTA

Diseño de API es una prueba habitual: "diséñame la API de un sistema de reservas". Lo que evalúan: si usas los verbos y códigos correctos, si tus rutas son coherentes, si contemplas paginación, si defines el formato de error, si distingues autenticación de autorización, y si preguntas por los requisitos antes de empezar a dibujar. Esto último puntúa más de lo que la gente cree.

EJERCICIOS · CAPÍTULOS 21–23

1. Conecta la API del capítulo 19 a PostgreSQL con Prisma y sustituye el almacenamiento en memoria.
2. Añade registro, inicio de sesión con JWT y rutas protegidas. Comprueba que un usuario no puede ver recursos de otro.
3. Diseña sobre papel la API completa de un sistema de reservas de salas: rutas, verbos, códigos y formato de error. Después impleméntala.
4. Revisa tu código contra la tabla de vulnerabilidades del capítulo 22, punto por punto.

PARTE 5

Ingeniería: lo que separa a un junior de un profesional

Escribir código que funcione lo hace mucha gente. Escribir código que se pueda probar, leer y cambiar sin miedo lo hace mucha menos. Esta parte es la que más peso tiene en una entrevista técnica seria.

- 24. Pruebas automatizadas
- 25. Código limpio y refactorización
- 26. Arquitectura y principios de diseño
- 27. Fundamentos de informática que sí se preguntan

24. Pruebas automatizadas

Un candidato junior que sabe escribir pruebas está por delante de la inmensa mayoría. No porque sea difícil, sino porque casi nadie lo practica antes de su primer empleo.

24.1 Por qué se prueba

La razón habitual que se da es "para encontrar errores". Es la menos importante. Las razones reales son:

- **Para poder cambiar el código sin miedo.** Sin pruebas, tocar algo que funciona es una apuesta. Con pruebas, ejecutas y sabes.
- **Para documentar el comportamiento esperado.** Un test bien nombrado dice qué debe hacer el sistema mejor que cualquier comentario.
- **Para diseñar mejor.** El código difícil de probar es casi siempre código mal estructurado. La dificultad de escribir el test es una señal de diseño.
- **Para no repetir comprobaciones a mano.** Lo que compruebas manualmente una vez al día, una máquina lo comprueba cien veces.

24.2 La pirámide de pruebas

Tipo	Qué prueba	Velocidad	Cantidad
Unitaria	Una función o módulo aislado	Milisegundos	Muchas
Integración	Varias piezas juntas: API + base de datos	Segundos	Algunas
Extremo a extremo	El flujo completo desde el navegador	Decenas de segundos	Pocas

La proporción sana es una base ancha de pruebas unitarias, una capa media de integración y unas pocas de extremo a extremo que cubran los caminos críticos (registrarse, comprar, pagar).

24.3 Anatomía de una prueba

```
import { describe, it, expect } from "vitest";
import { calcularTotal } from "./carrito.js";

describe("calcularTotal", () => {
  it("suma el precio por la cantidad de cada línea", () => {
    // Preparar
    const lineas = [
      { precio: 1000, cantidad: 2 },
      { precio: 500,  cantidad: 3 },
    ];

    // Actuar
    const total = calcularTotal(lineas);

    // Comprobar
    expect(total).toBe(3500);
  });

  it("devuelve 0 con un carrito vacío", () => {
    expect(calcularTotal([])).toBe(0);
  });

  it("lanza un error si una cantidad es negativa", () => {
    expect(() => calcularTotal([{ precio: 100, cantidad: -1 }]))
      .toThrow("La cantidad debe ser positiva");
  });
});
```

El patrón es siempre el mismo: **preparar, actuar, comprobar**. Y el nombre del test describe el comportamiento esperado, no la función. `it("devuelve 0 con un carrito vacío")` es útil; `it("test 2")` no dice nada cuando falla en el servidor de integración a las dos de la mañana.

24.4 Qué probar y qué no

Prueba esto	No pierdas tiempo con esto
Lógica de negocio: cálculos, reglas, validaciones	Getters y setters triviales
Casos límite: vacío, cero, negativo, nulo, muy grande	Librerías de terceros (ya están probadas)
Caminos de error y excepciones	Detalles de implementación internos
Todo fallo que aparezca en producción (test primero, arreglo después)	Estilos CSS y textos literales

PRUEBA COMPORTAMIENTO, NO IMPLEMENTACIÓN

Un test que comprueba que "se llamó al método X con el argumento Y" se rompe en cuanto refactorizas, aunque el comportamiento no haya cambiado. Un test que comprueba que "dado un carrito con dos artículos, el total es 3500" sobrevive a cualquier reescritura interna. El segundo tipo te ayuda; el primero te estorba.

24.5 Pruebas de integración de una API

```
import request from "supertest";
import { app } from "../src/app.js";
import { prisma } from "../src/lib/prisma.js";

describe("POST /api/productos", () => {
  let token;

  beforeEach(async () => {
    await prisma.producto.deleteMany();
    token = await obtenerTokenDePrueba();
  });

  it("crea un producto y devuelve 201", async () => {
    const respuesta = await request(app)
      .post("/api/productos")
      .set("Authorization", `Bearer ${token}`)
      .send({ nombre: "Teclado", precio: 4500, categoria: "periferico" });

    expect(respuesta.status).toBe(201);
    expect(respuesta.body).toMatchObject({ nombre: "Teclado", precio: 4500 });
    expect(respuesta.body.id).toBeDefined();
  });

  it("rechaza un precio negativo con 422", async () => {
    const respuesta = await request(app)
      .post("/api/productos")
      .set("Authorization", `Bearer ${token}`)
      .send({ nombre: "Teclado", precio: -100, categoria: "periferico" });

    expect(respuesta.status).toBe(422);
    expect(respuesta.body.errores[0].campo).toBe("precio");
  });

  it("devuelve 401 sin token", async () => {
    const respuesta = await request(app).post("/api/productos").send({});
    expect(respuesta.status).toBe(401);
  });
});
```

24.6 Pruebas de interfaz

```
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import { FormularioLogin } from "./FormularioLogin";

it("envía las credenciales al pulsar Entrar", async () => {
  const onEnviar = vi.fn();
  render(<FormularioLogin onEnviar={onEnviar} />);

  await userEvent.type(screen.getByLabelText(/email/i), "ana@x.com");
  await userEvent.type(screen.getByLabelText(/contraseña/i), "secreto123");
  await userEvent.click(screen.getByRole("button", { name: /entrar/i }));

  expect(onEnviar).toHaveBeenCalledWith({ email: "ana@x.com", password: "secreto123" });
});
```

Observa que los elementos se buscan por su etiqueta y su rol, igual que haría un usuario o un lector de pantalla, no por clases CSS. Esto tiene un efecto secundario excelente: si tu componente no es accesible, el test no lo encuentra y te obliga a arreglarlo.

24.7 Extremo a extremo con Playwright

```
import { test, expect } from "@playwright/test";

test("un usuario puede registrarse y entrar", async ({ page }) => {
  await page.goto("/registro");

  await page.getByLabel("Email").fill(`prueba-${Date.now()}@x.com`);
  await page.getByLabel("Contraseña").fill("Secreto123");
  await page.getByRole("button", { name: "Crear cuenta" }).click();

  await expect(page).toHaveURL("/panel");
  await expect(page.getByRole("heading", { name: /bienvenido/i })).toBeVisible();
});
```

24.8 Cobertura

```
npx vitest run --coverage
```

La cobertura mide qué porcentaje de líneas se ejecutan durante las pruebas. Es un indicador útil y un objetivo malo: el 100 % de cobertura no significa que el código sea correcto, solo que se ha ejecutado. Un 70–80 % con pruebas bien pensadas sobre la lógica importante vale más que un 100 % de pruebas vacías.

> **EN LA ENTREVISTA**
>
> "¿Escribes pruebas?" — si la respuesta es no, ya has perdido puntos. Ten preparado: qué tipos de prueba conoces, un ejemplo concreto de tu portfolio, qué es un doble de prueba (*mock*) y cuándo usarlo, y por qué la cobertura no es el objetivo. Y si en una prueba técnica te dan tiempo, escribe al menos tres tests. Es el detalle que más recuerdan los evaluadores.

EJERCICIOS · CAPÍTULO 24

1. Escribe pruebas unitarias para las funciones del capítulo 11 incluyendo todos los casos límite.
2. Añade pruebas de integración a la API del capítulo 19: un caso feliz, uno de validación y uno de autorización por endpoint.
3. Provoca un fallo a propósito, escribe primero la prueba que lo detecta y luego arréglalo.
4. Mide la cobertura de tu proyecto y localiza qué parte importante se queda sin probar.

25. Código limpio y refactorización

25.1 El coste real del código

Un programa se escribe una vez y se lee decenas de veces: por otros, y por ti mismo dentro de seis meses, que a efectos prácticos es otra persona. Optimizar para la escritura y no para la lectura es la decisión equivocada casi siempre.

25.2 Nombres

```
// Mal
const d = new Date();
const x = u.filter((i) => i.a);
function proc(data) { }
let flag = true;

// Bien
const fechaCreacion = new Date();
const usuariosActivos = usuarios.filter((u) => u.activo);
function calcularImpuestos(factura) { }
let hayCambiosSinGuardar = true;
```

Reglas prácticas: los booleanos empiezan por `es`, `tiene`, `hay`, `puede`; las funciones empiezan por verbo; las colecciones van en plural; y la longitud del nombre debe ser proporcional al ámbito (una `i` en un bucle de tres líneas está bien, una `d` con alcance de módulo no).

25.3 Funciones

```
// Mal: hace cuatro cosas, y hay que leerla entera para saber cuáles
async function procesarPedido(datos) {
  if (!datos.email || !datos.email.includes("@")) throw new Error("Email inválido");
  if (!datos.lineas || datos.lineas.length === 0) throw new Error("Vacío");

  let total = 0;
  for (const l of datos.lineas) {
    const p = await db.producto.findUnique({ where: { id: l.productoId } });
    if (p.stock < l.cantidad) throw new Error("Sin stock");
    total += p.precio * l.cantidad;
  }
  if (datos.cupon === "VERANO") total = total * 0.9;

  const pedido = await db.pedido.create({ data: { total, email: datos.email } });
  await enviarEmail(datos.email, "Pedido confirmado", `Total: ${total}`);
  return pedido;
}

// Bien: cada paso tiene nombre y se puede probar por separado
async function procesarPedido(datos) {
  validarDatosPedido(datos);
  const lineas = await resolverLineas(datos.lineas);
  comprobarStock(lineas);

  const total = aplicarDescuento(calcularTotal(lineas), datos.cupon);
  const pedido = await crearPedido({ email: datos.email, total });

  await notificarConfirmacion(pedido);
  return pedido;
}
```

La segunda versión se lee como un resumen de lo que ocurre. Cada función auxiliar es corta, tiene una responsabilidad y se puede probar aislada. Y lo más importante: cuando haya que cambiar la regla del descuento, sabes exactamente dónde mirar.

25.4 Comentarios

```
// Comentario inútil: repite lo que ya dice el código
// incrementa el contador
contador++;

// Comentario útil: explica el porqué, que el código no puede expresar
// La pasarela de pago rechaza importes con más de 2 decimales,
// por eso redondeamos antes de enviar. Ver incidencia #4821.
const importe = Math.round(total * 100) / 100;
```

La regla: el código dice *qué* hace; el comentario dice *por qué*. Si necesitas un comentario para explicar qué hace un bloque, probablemente ese bloque debería ser una función con un buen nombre.

25.5 Olores de código

Olor	Síntoma	Refactorización

Función larga	Más de 30–40 líneas, hay que hacer scroll	Extraer funciones
Muchos parámetros	Cuatro o más argumentos	Objeto de opciones
Código duplicado	El mismo bloque en tres sitios	Extraer a función común
Números mágicos	`if (estado === 3)`	Constantes con nombre o enum
Anidamiento profundo	Cuatro niveles de `if`	Cláusulas de guarda, salida temprana
Comentarios explicativos	Un párrafo antes de cada bloque	Extraer y nombrar
Clase o módulo enorme	Hace de todo	Dividir por responsabilidad
Parámetro booleano	`guardar(datos, true)`	Dos funciones distintas

25.6 Cómo refactorizar sin romper nada

1. Asegúrate de que hay pruebas que cubren el comportamiento actual. Si no las hay, escríbelas *antes*.
2. Haz un cambio pequeño.
3. Ejecuta las pruebas.
4. Haz commit.
5. Repite.

Refactorizar significa cambiar la estructura **sin cambiar el comportamiento**. Si además arreglas un fallo o añades una funcionalidad, eso ya no es refactorizar, y mezclar las dos cosas en un mismo commit es la forma más eficaz de que una revisión de código se vuelva imposible.

> **LA REGLA DEL CAMPAMENTO**
>
> Deja el código un poco mejor de como lo encontraste. Un nombre mejorado, un comentario obsoleto eliminado, una función de 60 líneas partida en dos. No hace falta una gran reescritura: la mejora continua en incrementos pequeños es lo que mantiene sano un proyecto a lo largo de años, y es una actitud que los equipos notan enseguida.

26. Arquitectura y principios de diseño

26.1 Acoplamiento y cohesión

Dos conceptos que explican casi todo lo demás:

- **Acoplamiento**: cuánto depende un módulo de otro. Quieres poco.
- **Cohesión**: cuánto tienen que ver entre sí las cosas que están juntas en un módulo. Quieres mucha.

Un fichero `utils.js` con cuarenta funciones que no tienen nada que ver entre ellas es baja cohesión. Un servicio que importa directamente el cliente de base de datos, el de correo y el framework HTTP es alto acoplamiento. Ambos son síntomas de que el sistema va a costar cada vez más de cambiar.

26.2 SOLID, explicado sin misticismo

Principio	Qué significa en la práctica
S — Responsabilidad única	Cada módulo tiene un motivo para cambiar. Si tu servicio de usuarios también manda correos y genera PDFs, tiene tres.
O — Abierto/cerrado	Añadir un caso nuevo no debería obligarte a modificar el código existente. Si cada nuevo método de pago te hace tocar el mismo `switch`, algo falla.
L — Sustitución de Liskov	Si algo dice ser de un tipo, debe comportarse como ese tipo. Una subclase que lanza "no soportado" en un método heredado lo incumple.
I — Segregación de interfaces	Mejor varias interfaces pequeñas que una enorme que obliga a implementar métodos que nadie usa.
D — Inversión de dependencias	Depende de abstracciones, no de implementaciones concretas. Tu lógica de negocio no debería saber si los datos vienen de PostgreSQL o de un fichero.

```
// Sin inversión de dependencias: imposible de probar sin base de datos
import { prisma } from "../lib/prisma.js";

export async function registrarUsuario(datos) {
  return prisma.usuario.create({ data: datos });
}

// Con inversión: el servicio recibe lo que necesita
export function crearServicioUsuarios({ repositorio, emisorEmail }) {
  return {
    async registrar(datos) {
      const usuario = await repositorio.crear(datos);
      await emisorEmail.bienvenida(usuario.email);
      return usuario;
    },
  };
}

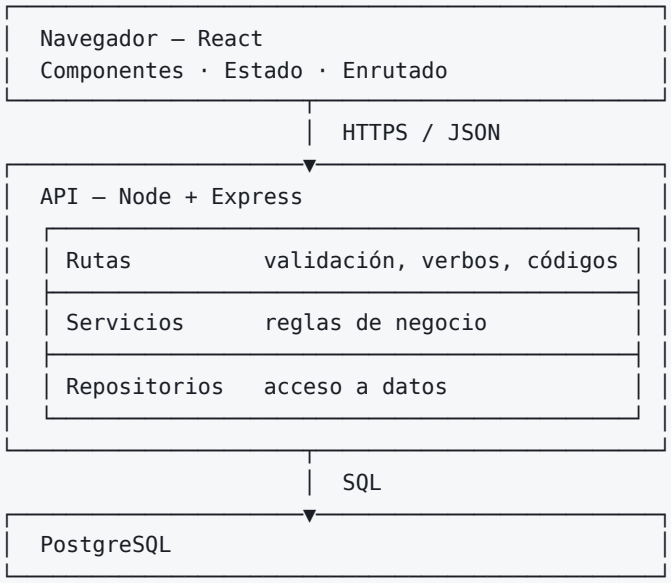
// En producción
const servicio = crearServicioUsuarios({ repositorio: repoPrisma, emisorEmail: smtp });

// En las pruebas
const servicio = crearServicioUsuarios({
  repositorio: { crear: async (d) => ({ id: 1, ...d }) },
  emisorEmail: { bienvenida: vi.fn() },
});
```

26.3 Principios que usarás más que SOLID

- **KISS** — mantenlo simple. La solución más sencilla que funcione suele ser la correcta.
- **YAGNI** — no lo vas a necesitar. No construyas para requisitos hipotéticos: el 80 % de esa flexibilidad no se usa nunca y sí se paga en complejidad.
- **DRY** — no te repitas. Con matiz: DRY va de no duplicar *conocimiento*, no de no duplicar *líneas*. Dos trozos de código parecidos que cambian por motivos distintos deben seguir separados. Una abstracción prematura hace más daño que una duplicación.
- **Principio de mínima sorpresa** — el código debe hacer lo que su nombre sugiere, sin efectos ocultos.

26.4 Arquitectura de una aplicación web típica



26.5 Monolito o microservicios

Pregunta habitual en entrevistas, y con una respuesta que conviene tener clara porque la mayoría de candidatos responde mal:

	Monolito	Microservicios
Despliegue	Uno solo, sencillo	Muchos, con orquestación
Complejidad	En el código	En la infraestructura y la red
Escalado	Todo junto	Por servicio
Equipo	Funciona bien hasta ~20 personas	Pensado para muchos equipos autónomos
Depuración	Una traza de pila	Trazas distribuidas, observabilidad

La respuesta madura: empieza con un monolito bien modularizado. Los microservicios resuelven un problema organizativo (equipos que se bloquean entre sí), no técnico, y traen consigo una complejidad operativa enorme. Adoptarlos antes de tener ese problema es el error de arquitectura más caro y más común de la última década.

27. Fundamentos de informática que sí se preguntan

No necesitas una carrera de informática para trabajar, pero sí unos mínimos: aparecen en pruebas técnicas y, sobre todo, explican por qué un programa que va bien con 100 registros se muere con 100.000.

27.1 Notación O grande

Describe cómo crece el tiempo de ejecución cuando crece la entrada. No mide segundos: mide forma de crecimiento.

Complejidad	Nombre	Ejemplo	n = 1.000.000
O(1)	Constante	Acceder a `array[5]`, buscar en un Map	1 operación
O(log n)	Logarítmica	Búsqueda binaria, índice de base de datos	~20
O(n)	Lineal	Recorrer un array	1.000.000
O(n log n)	Lineal-logarítmica	Ordenar bien	~20.000.000
O(n^2)	Cuadrática	Bucle dentro de bucle	10^{12}: inviable

```
// O(n²): para cada elemento, recorre todos los demás. Con 10.000 pedidos
// son 100 millones de comparaciones.
const resultado = pedidos.map((p) => ({
  ...p,
  usuario: usuarios.find((u) => u.id === p.usuarioId),   // find es O(n)
}));

// O(n): construye un índice una vez y busca en tiempo constante
const porId = new Map(usuarios.map((u) => [u.id, u]));
const resultado = pedidos.map((p) => ({ ...p, usuario: porId.get(p.usuarioId) }));
```

Ese cambio de tres líneas es una de las optimizaciones más rentables que existen y aparece constantemente en código real de junior.

27.2 Estructuras de datos

Estructura	Buena en	Mala en	Cuándo usarla
Array	Acceso por índice O(1), recorrido	Buscar por valor O(n), insertar al principio	Listas ordenadas, colecciones
Objeto / Map	Buscar por clave O(1)	Orden (el objeto no garantiza), recorrido	Índices, cachés, diccionarios
Set	Comprobar pertenencia O(1), unicidad	Acceso por posición	Eliminar duplicados, "ya visto"

Pila (LIFO)	Añadir y quitar por el final	Acceso al medio	Deshacer, historial, recursión
Cola (FIFO)	Añadir al final, quitar del principio	Acceso al medio	Tareas pendientes, procesamiento por lotes
Árbol	Búsqueda ordenada O(log n)	Complejidad de implementación	Índices de bases de datos, jerarquías
Grafo	Relaciones entre entidades	Algoritmos costosos	Redes sociales, rutas, dependencias

27.3 Algoritmos que debes saber implementar

```
// Búsqueda binaria: O(log n). Requiere array ordenado.
function busquedaBinaria(lista, objetivo) {
  let inicio = 0;
  let fin = lista.length - 1;

  while (inicio <= fin) {
    const medio = Math.floor((inicio + fin) / 2);
    if (lista[medio] === objetivo) return medio;
    if (lista[medio] < objetivo) inicio = medio + 1;
    else fin = medio - 1;
  }
  return -1;
}

// Dos punteros: encontrar dos números que sumen un objetivo, O(n)
function dosSuman(numeros, objetivo) {
  const vistos = new Map();
  for (let i = 0; i < numeros.length; i++) {
    const falta = objetivo - numeros[i];
    if (vistos.has(falta)) return [vistos.get(falta), i];
    vistos.set(numeros[i], i);
  }
  return null;
}

// Recursión con memoria
function fibonacci(n, memoria = new Map()) {
  if (n <= 1) return n;
  if (memoria.has(n)) return memoria.get(n);
  const r = fibonacci(n - 1, memoria) + fibonacci(n - 2, memoria);
  memoria.set(n, r);
  return r;
}
```

SOBRE LAS PRUEBAS DE ALGORITMOS

En España, las entrevistas de estilo "LeetCode" son menos habituales que en las grandes tecnológicas estadounidenses, pero aparecen: en empresas de producto, en procesos remotos internacionales y en algunas consultoras grandes. Con resolver unos 30–50 problemas de nivel fácil e intermedio tendrás cubierto lo razonable. No dediques meses a esto a costa de construir proyectos: para el mercado español, el portfolio pesa más.

27.4 Concurrencia y memoria, lo justo

- **Proceso y hilo.** Un proceso tiene su propia memoria; los hilos la comparten. Node usa un hilo principal más un grupo de hilos para operaciones de entrada/salida.
- **Bloqueante y no bloqueante.** Una operación bloqueante detiene el hilo mientras espera. En Node, bloquear el hilo principal (un bucle pesado, una lectura síncrona de fichero) congela el servidor entero para todos los usuarios.
- **Condición de carrera.** Dos operaciones concurrentes sobre el mismo dato cuyo resultado depende del orden. En bases de datos se resuelve con transacciones y bloqueos.
- **Fuga de memoria.** Referencias que nunca se liberan: escuchadores de eventos no eliminados, cachés que solo crecen, temporizadores sin limpiar. En React, es la razón de que los efectos necesiten función de limpieza.

EJERCICIOS · CAPÍTULOS 25–27

1. Coge la función más larga que hayas escrito hasta ahora y refactorízala en funciones pequeñas con nombre, sin cambiar su comportamiento. Comprueba con pruebas que sigue igual.
2. Busca en tu código un bucle anidado $O(n^2)$ y conviértelo en $O(n)$ con un Map.
3. Implementa una pila y una cola desde cero, con sus pruebas.
4. Resuelve diez problemas de nivel fácil en una plataforma de algoritmos y explica en voz alta la complejidad de cada solución.

PARTE 6

Producción: que funcione fuera de tu portátil

Un proyecto que solo corre en tu máquina no cuenta. Saber desplegar es lo que convierte un ejercicio en algo que puedes enseñar en una entrevista y que alguien puede usar.

- 28. Docker
- 29. Integración y despliegue continuos
- 30. Despliegue, configuración y observabilidad

28. Docker

28.1 Qué problema resuelve

"En mi máquina funciona" es una frase que ha costado millones de euros al sector. Docker empaqueta tu aplicación junto con su sistema operativo, sus dependencias y su configuración en una **imagen**. Esa imagen se ejecuta idéntica en tu portátil, en el de tu compañero y en el servidor de producción.

La diferencia con una máquina virtual es que un contenedor comparte el núcleo del sistema anfitrión en vez de emular una máquina entera: arranca en milisegundos y ocupa megas en lugar de gigas.

28.2 Conceptos

Término	Qué es
Dockerfile	La receta: instrucciones para construir una imagen
Imagen	El paquete inmutable resultante
Contenedor	Una instancia en ejecución de una imagen
Volumen	Almacenamiento persistente: lo que sobrevive al borrar el contenedor
Registro	Repositorio de imágenes (Docker Hub, GitHub Container Registry)

28.3 Un Dockerfile real

```
# Construcción por etapas: la imagen final no lleva herramientas de compilación
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci                      # ci usa el lockfile: instalación reproducible
COPY . .
RUN npm run build

FROM node:22-alpine AS produccion
WORKDIR /app
ENV NODE_ENV=production

COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force

COPY --from=build /app/dist ./dist

# No ejecutar como root: buena práctica de seguridad
USER node

EXPOSE 3000
HEALTHCHECK --interval=30s CMD node -e "fetch('http://localhost:3000/salud')"
CMD ["node", "dist/server.js"]
```

Dos detalles que marcan la diferencia y que se valoran en una revisión: copiar primero `package*.json` y luego el resto aprovecha la caché de capas de Docker (si no cambian las dependencias, no se reinstalan); y la construcción por etapas deja fuera de la imagen final todo lo que solo hacía falta para compilar.

```
# .dockerignore -- tan importante como el .gitignore
node_modules
npm-debug.log
.git
.env
dist
coverage
```

28.4 Comandos

```
docker build -t mi-api:1.0 .
docker run -p 3000:3000 --env-file .env mi-api:1.0
docker ps                        # contenedores en marcha
docker ps -a                     # todos, incluidos los parados
docker logs -f <id>              # ver logs en vivo
docker exec -it <id> sh          # abrir una shell dentro del contenedor
docker stop <id>
docker system prune -a           # limpiar todo lo no usado (libera mucho disco)
```

28.5 Docker Compose

Para levantar varios servicios a la vez. Esto es lo que usarás a diario en desarrollo: una base de datos lista en un comando, sin instalar nada en tu sistema.

```yaml
# docker-compose.yml
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: dev
      POSTGRES_PASSWORD: dev
      POSTGRES_DB: miapp
    ports:
      - "5432:5432"
    volumes:
      - datos_pg:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U dev"]
      interval: 5s

  api:
    build: .
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgresql://dev:dev@db:5432/miapp
    depends_on:
      db:
        condition: service_healthy

volumes:
  datos_pg:
```

```bash
docker compose up -d          # levantar en segundo plano
docker compose logs -f api    # ver logs de un servicio
docker compose down           # parar y eliminar
docker compose down -v        # ...y borrar también los volúmenes
```

EL NIVEL DE DOCKER QUE NECESITA UN JUNIOR

Saber escribir un Dockerfile razonable, levantar una base de datos con Compose y depurar un contenedor que no arranca. Kubernetes no es tu problema todavía: aparece en ofertas de perfiles con experiencia y de DevOps, y estudiarlo ahora es tiempo que quitas a cosas con más retorno.

29. Integración y despliegue continuos

29.1 Qué son

- **Integración continua (CI).** Cada vez que alguien sube código, una máquina ejecuta automáticamente el linter, las pruebas y la compilación. Si algo falla, se sabe en minutos y no en la demo del viernes.
- **Despliegue continuo (CD).** Si todo pasa, el cambio se publica solo, sin intervención manual.

29.2 Un flujo de GitHub Actions completo

```yaml
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  calidad:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: test
          POSTGRES_DB: test
        options: >-
          --health-cmd pg_isready --health-interval 5s --health-retries 5
        ports: ["5432:5432"]

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - run: npm ci
      - run: npm run lint
      - run: npx tsc --noEmit                # comprobación de tipos
      - run: npx prisma migrate deploy
        env:
          DATABASE_URL: postgresql://postgres:test@localhost:5432/test
      - run: npm test -- --coverage
        env:
          DATABASE_URL: postgresql://postgres:test@localhost:5432/test
      - run: npm run build
```

Añadir este fichero a tus proyectos de portfolio y que el repositorio muestre la insignia verde de "CI passing" es una señal potentísima para un reclutador técnico: dice que trabajas como en una empresa antes de haber estado en

una.

29.3 Ramas protegidas

En GitHub, en los ajustes del repositorio, se puede exigir que `main` solo acepte cambios vía pull request, con la CI en verde y al menos una aprobación. Es el estándar en cualquier equipo profesional y se configura en dos minutos.

30. Despliegue, configuración y observabilidad

30.1 Dónde desplegar

Plataforma	Para qué	Coste inicial
Vercel	Frontend y Next.js. Despliegue con cada push, vista previa por rama	Gratis para proyectos personales
Netlify	Sitios estáticos y funciones	Gratis
Railway / Render	API de Node + PostgreSQL, sin configurar servidores	Capa gratuita limitada
Fly.io	Contenedores Docker cerca del usuario	Capa gratuita
VPS (Hetzner, DigitalOcean)	Control total. Tú administras el servidor	4–10 €/mes
AWS / Azure / GCP	Lo que usan las empresas grandes	Capa gratuita, con cuidado

Para tu portfolio: frontend en Vercel, API y base de datos en Railway o Render. Cero euros y despliegue automático desde GitHub. Montar un VPS a mano es un ejercicio excelente *después*, cuando quieras entender qué hacen esas plataformas por ti.

30.2 Configuración y secretos

```
# .env.example -- este SÍ se sube al repositorio, con valores falsos
DATABASE_URL=postgresql://usuario:clave@localhost:5432/miapp
JWT_SECRET=cambia-esto-en-produccion
PORT=3000
NODE_ENV=development
FRONTEND_URL=http://localhost:5173
```

```
// src/config/index.js -- validar la configuración al arrancar
import { z } from "zod";

const Esquema = z.object({
  DATABASE_URL: z.string().url(),
  JWT_SECRET: z.string().min(32),
  PORT: z.coerce.number().default(3000),
  NODE_ENV: z.enum(["development", "test", "production"]).default("development"),
});

const resultado = Esquema.safeParse(process.env);
if (!resultado.success) {
  console.error("Configuración inválida:", resultado.error.flatten().fieldErrors);
  process.exit(1);            // fallar al arrancar, no en la primera petición
}

export const config = resultado.data;
```

Fallar rápido y con un mensaje claro al arrancar es infinitamente mejor que descubrir a las tres de la mañana que faltaba una variable de entorno. Este patrón de veinte líneas es de los que más impresionan en una revisión de portfolio.

30.3 Logs

```
import pino from "pino";

export const log = pino({
  level: config.NODE_ENV === "production" ? "info" : "debug",
  redact: ["req.headers.authorization", "password", "*.password"],
});

log.info({ usuarioId: 42, accion: "login" }, "Inicio de sesión correcto");
log.error({ err, pedidoId }, "No se pudo procesar el pago");
```

Tres reglas: logs estructurados en JSON (no cadenas concatenadas, porque luego hay que buscar en ellos); niveles distintos (debug, info, warn, error); y **nunca** registrar contraseñas, tokens ni datos personales. Esto último, además de sentido común, es una obligación bajo el RGPD.

30.4 Observabilidad

- **Logs** — qué ha pasado.
- **Métricas** — cuántas peticiones, con qué latencia, cuántos errores.
- **Trazas** — el recorrido de una petición concreta a través del sistema.
- **Alertas** — que alguien se entere cuando algo se rompe, sin tener que mirar.

Para un proyecto personal, con un endpoint `/salud`, logs estructurados y Sentry (gratis) para capturar excepciones vas sobrado, y además puedes hablar de ello con propiedad en una entrevista.

30.5 Lista de comprobación antes de desplegar

	Comprobación
☐	Ningún secreto en el repositorio; `.env` en `.gitignore`
☐	Variables de entorno configuradas en la plataforma de destino
☐	Migraciones de base de datos aplicadas
☐	HTTPS activo
☐	CORS limitado al dominio del frontend, no `*`
☐	Límite de peticiones en rutas de autenticación
☐	Los errores 500 no filtran detalles internos
☐	Endpoint de salud respondiendo

☐	Copias de seguridad de la base de datos configuradas
☐	Pruebas en verde en CI
☐	README con instrucciones para arrancar el proyecto desde cero

30.6 Lo mínimo de AWS

AWS aparece en unas 680 ofertas mensuales en España. No necesitas certificarte, pero sí saber de qué habla la gente:

Servicio	Qué es
EC2	Máquinas virtuales
S3	Almacenamiento de ficheros (imágenes, copias de seguridad)
RDS	Bases de datos gestionadas (PostgreSQL, MySQL)
Lambda	Funciones sin servidor: código que se ejecuta por eventos
CloudFront	CDN
IAM	Usuarios, roles y permisos
VPC	Red privada

> **CUIDADO CON LA FACTURA**
>
> La capa gratuita de AWS es generosa pero tiene trampas, y hay historias de facturas de cuatro cifras por un recurso olvidado. Si experimentas con AWS: activa alertas de facturación desde el primer día, usa solo servicios de la capa gratuita y apaga lo que no uses.

PARTE 7

La IA como herramienta de trabajo

En 2026 esto no es un extra: es la competencia que define el perfil de entrada. El capítulo va tanto de cómo usarla como de cuándo no fiarse.

- 31. Trabajar con asistentes de IA sin dejar de aprender

31. Trabajar con asistentes de IA

El mercado ya no contrata a quien escribe código a mano ni a quien copia y pega lo que le dice una máquina. Contrata a quien dirige la herramienta y responde de lo que sale.

31.1 El equilibrio: la trampa del andamio

Hay una tensión real que conviene nombrar. Un asistente de IA te permite producir código que todavía no sabrías escribir. Eso acelera muchísimo el trabajo *y* puede impedir que aprendas, porque el aprendizaje ocurre precisamente en el esfuerzo de recuperar y construir, que es lo que la herramienta te ahorra.

> **REGLA PARA LA FASE DE ESTUDIO**
>
> Mientras estés aprendiendo un concepto nuevo (las tres primeras veces que lo usas), escríbelo tú. Usa la IA **después**: para revisar lo que has escrito, para explicarte lo que no entiendes y para proponerte alternativas. Delegar la primera escritura de algo que no dominas es hipotecar tu criterio futuro, y en la entrevista se nota inmediatamente: te preguntan por qué elegiste ese enfoque y no hay respuesta.

31.2 Dónde sí acelera muchísimo

Tarea	Por qué encaja bien
Explicar código ajeno	Reduce a minutos lo que tardarías horas en entender de un proyecto grande
Escribir pruebas de casos límite	Se le dan bien los casos que a un humano se le olvidan
Código repetitivo (*boilerplate*)	Configuraciones, esquemas, conversiones, CRUD estándar
Traducir entre lenguajes o frameworks	De JavaScript a TypeScript, de Express a Fastify
Primeras versiones de documentación	README, comentarios de API, mensajes de commit
Depurar mensajes de error	Le pegas la traza y te orienta en segundos
Revisión previa	"¿Qué problemas de seguridad ves aquí?" antes de pedir revisión humana

31.3 Dónde falla y hay que verificar siempre

- **Inventa APIs que no existen.** Métodos, parámetros y librerías plausibles pero falsos. Si no lo has visto en la documentación, no existe.
- **Da código desactualizado.** Patrones de hace tres versiones que hoy están desaconsejados.
- **Ignora tu contexto.** No conoce las convenciones de tu proyecto ni las decisiones que hay detrás.
- **Falla en seguridad.** Genera con frecuencia consultas concatenadas, validaciones ausentes o comprobaciones de autorización que faltan.
- **Es confiadamente incorrecta.** El tono seguro es idéntico cuando acierta y cuando no. Esta es la parte peligrosa.

- **No sabe qué *no* hacer.** Te dará una solución aunque el problema no deba resolverse así.

31.4 Cómo pedir bien

```
// Petición pobre
"haz una función de login"

// Petición buena
"Escribe un manejador de Express para POST /auth/login.
 Stack: Node 22, Express 4, Prisma, PostgreSQL, TypeScript en modo estricto.
 Requisitos:
 - Valida el cuerpo con Zod (email y password).
 - Compara la contraseña con bcrypt.
 - Devuelve el mismo error 401 tanto si el email no existe como si la clave
   es incorrecta, para no permitir enumeración de usuarios.
 - Emite un JWT de 15 minutos y un refresh token en cookie httpOnly.
 - Aplica límite de 10 intentos cada 15 minutos.
 - Sin any. Errores tipados.
 Devuélveme solo el manejador y sus tipos, sin explicación."
```

La diferencia no es el largo: es que la segunda incluye contexto, restricciones y criterio. Y ese criterio es tuyo, no de la herramienta. Por eso los fundamentos siguen siendo el activo: **no puedes pedir bien lo que no entiendes, ni revisar lo que no sabes leer.**

31.5 El protocolo de revisión

Antes de aceptar cualquier código generado, cinco comprobaciones:

1. **¿Entiendo cada línea?** Si no, o lo estudias o no entra. Sin excepciones.
2. **¿Existen de verdad estas funciones?** Contrasta con la documentación oficial.
3. **¿Gestiona los errores y los casos límite?** Vacío, nulo, duplicado, sin permisos.
4. **¿Es seguro?** Consultas parametrizadas, validación en servidor, comprobación de propiedad del recurso.
5. **¿Encaja con las convenciones del proyecto?** Nombres, estructura, estilo.

> **EN LA ENTREVISTA**
>
> Preguntar por el uso de IA es ya rutinario, y hay dos respuestas que puntúan mal: "no la uso" (suena a no estar al día) y "la uso para todo" (suena a no tener criterio). La respuesta que funciona describe un método: *"la uso para código repetitivo, para entender código ajeno rápido y para generar casos de prueba; reviso siempre la seguridad y los casos límite porque ahí falla con frecuencia; y cuando estoy aprendiendo algo nuevo lo escribo yo primero para no quedarme sin criterio"*.
>
> Y si te dan una prueba técnica: pregunta si puedes usar IA. Casi siempre dirán que sí, y preguntarlo demuestra honestidad. Lo que evalúan después no es si la usaste, sino si entiendes lo que entregaste.

EJERCICIOS · CAPÍTULO 31

1. Pide a un asistente una función para subir ficheros. Encuentra al menos tres problemas de seguridad en lo que te devuelva.
2. Coge un repositorio de código abierto que no conozcas y usa la IA para entender su arquitectura en 20 minutos. Después verifica en el código si lo que te contó es cierto.
3. Escribe tú una función compleja y pide después una revisión crítica. Anota qué encontró que a ti se te escapó.
4. Prepara y ensaya en voz alta tu respuesta a "¿cómo usas la IA en tu trabajo?".

PARTE 8

El portfolio: tres proyectos que sí convencen

Sin experiencia laboral, tu portfolio *es* tu experiencia. Estos tres proyectos están diseñados para demostrar, entre los tres, todo lo que evalúa un equipo técnico al mirar a un junior.

- 32. Qué hace bueno a un proyecto de portfolio
- 33. Proyecto 1 — La aplicación completa
- 34. Proyectos 2 y 3 — El diferenciador y la contribución

32. Qué hace bueno a un proyecto de portfolio

32.1 Lo que no funciona

Estos proyectos aparecen en el 70 % de los portfolios junior y ya no distinguen a nadie, porque son el resultado de seguir un tutorial y se nota:

- La lista de tareas.
- La calculadora.
- El clon de Netflix consumiendo una API pública.
- La web de portfolio que solo contiene el portfolio.
- Cualquier proyecto cuyo README sea el que genera la herramienta por defecto.

No es que estén mal como ejercicio de aprendizaje: están perfectos para eso. El problema es enseñarlos como prueba de que sabes trabajar.

32.2 Lo que sí funciona

Criterio	Por qué importa
Resuelve un problema real	Preferiblemente uno tuyo, o de alguien que conozcas. Se nota cuando hay un usuario de verdad detrás.
Está desplegado y funcionando	Un enlace que se abre en un clic vale diez veces más que un repositorio.
Tiene autenticación y persistencia	Demuestra que dominas el stack completo, no solo la parte visible.
Tiene pruebas y CI	La señal más clara de que trabajas como un profesional.
El README explica decisiones	Qué hace, cómo arrancarlo, qué decidiste y por qué, qué harías distinto. Esto es lo que más se lee.
El historial de Git es limpio	Commits pequeños, mensajes descriptivos, ramas por funcionalidad.
Se puede contar en dos minutos	Porque eso es exactamente lo que te van a pedir en la entrevista.

> **TRES PROYECTOS BIEN ACABADOS, NO DIEZ A MEDIAS**
>
> Un portfolio con quince repositorios abandonados en el tercer commit dice algo peor que uno con tres. La capacidad de *terminar* es una de las cosas más escasas y más valoradas del oficio: casi todo el mundo sabe empezar.

32.3 El README que te consigue entrevistas

```
# Nombre del proyecto

Una frase que explique qué problema resuelve y para quién.

🔗 **Demo:** https://miproyecto.vercel.app
📦 **API:** https://api-miproyecto.railway.app/salud

![captura](docs/captura.png)

## Qué hace
Tres o cuatro viñetas con las funcionalidades principales.

## Stack
React 19 · TypeScript · Node 22 · Express · PostgreSQL · Prisma ·
Vitest · Playwright · Docker · GitHub Actions · Railway

## Decisiones técnicas
- **Por qué PostgreSQL y no MongoDB:** los datos son claramente
  relacionales (usuarios, reservas, salas) y necesitaba integridad
  referencial y transacciones.
- **Por qué JWT y no sesiones:** la API la consumen dos clientes
  distintos y quería mantenerla sin estado.
- **Compromiso asumido:** no hay cola de trabajos para los correos;
  se envían de forma síncrona. Con más volumen lo movería a una cola.

## Arrancar en local

    git clone ...
    cp .env.example .env
    docker compose up -d
    npm install && npx prisma migrate dev && npm run dev

## Pruebas

    npm test              # unitarias e integración
    npm run test:e2e      # extremo a extremo

## Qué mejoraría con más tiempo
- Paginación por cursor en vez de por desplazamiento.
- Caché de las consultas de disponibilidad.
- Internacionalización.
```

La sección de **decisiones técnicas** y la de **qué mejoraría** son las que más peso tienen. Demuestran que hubo alternativas, que elegiste con criterio y que conoces las limitaciones de lo que has hecho. Un junior que sabe lo que su proyecto *no* hace bien es un junior con criterio.

33. Proyecto 1 — La aplicación completa

Objetivo: demostrar que dominas el stack de principio a fin. Es el proyecto que enseñas primero y el que más vas a tener que defender.

33.1 Elegir el dominio

Evita el género genérico. Busca algo con reglas de negocio propias, porque esas reglas son de lo que se habla en la entrevista. Ideas con sustancia:

- **Gestor de reservas** para un negocio local (una escuela de surf, un estudio, una consulta): disponibilidad, solapamientos, cancelaciones, franjas horarias. Tiene lógica temporal real, que es sorprendentemente difícil.
- **Panel de inventario** para un pequeño comercio: stock, entradas y salidas, alertas de mínimo, histórico.
- **Seguimiento de solicitudes de empleo**: tu propio proceso de búsqueda, con estados, recordatorios y estadísticas. Tiene la ventaja de que lo usarás de verdad.
- **Gestor de eventos y entradas**: aforo, listas, control de acceso con código QR.
- **Divisor de gastos compartidos**: quién debe qué a quién, con liquidación óptima. La lógica de saldos es un problema bonito.

33.2 Alcance obligatorio

Área	Requisito mínimo
Autenticación	Registro, inicio de sesión, cierre de sesión, rutas protegidas, dos roles
CRUD	Al menos dos entidades relacionadas, con las cuatro operaciones
Reglas de negocio	Algo que no sea un CRUD puro: validar solapamientos, calcular disponibilidad, aplicar descuentos
Listados	Paginación, filtros y ordenación en servidor
Estados de interfaz	Cargando, vacío, error y éxito. Los cuatro, siempre
Validación	En cliente (comodidad) y en servidor (obligatoria)
Pruebas	Unitarias de la lógica de negocio, integración de la API, una de extremo a extremo del flujo principal
Infraestructura	Docker Compose para desarrollo, CI en GitHub Actions, desplegado
Accesibilidad	Navegable con teclado, etiquetas correctas, contraste adecuado
Documentación	README como el del capítulo anterior

33.3 Plan de construcción en cuatro semanas

Semana	Trabajo	Al terminar

1	Definir el alcance por escrito. Dibujar el modelo de datos. Esquema de Prisma y migraciones. API con las rutas principales y validación. Pruebas de integración de cada endpoint.	API funcionando, probada con `curl` y con pruebas automáticas
2	Autenticación completa: registro, login, JWT, middleware de autorización, comprobación de propiedad de recursos. Frontend base: rutas, layout, cliente HTTP, contexto de autenticación.	Se puede entrar y ver una pantalla protegida
3	Pantallas principales: listado con filtros, formularios de alta y edición, detalle. Los cuatro estados en cada pantalla. Manejo de errores coherente.	La aplicación es usable de principio a fin
4	Pulido: accesibilidad, responsive, mensajes de error útiles. CI, despliegue, datos de demostración. README. Prueba de extremo a extremo. Grabar un vídeo de 90 segundos.	**Proyecto público y enseñable**

> **EL ERROR QUE HUNDE EL 80 % DE ESTOS PROYECTOS**
>
> Ampliar el alcance sin parar. Empiezas con reservas y acabas queriendo chat en tiempo real, pagos, notificaciones push y aplicación móvil. Resultado: nada terminado y tres meses perdidos.
>
> Escribe el alcance en un fichero `ALCANCE.md` el primer día y trátalo como un contrato. Todo lo que se te ocurra después va a una lista de "versión 2" que no vas a hacer. Acabar es la funcionalidad más importante.

33.4 Cómo contarlo en la entrevista

Prepara una explicación de dos minutos con esta estructura, y ensáyala en voz alta hasta que salga sin pensar:

1. **El problema** (15 s): "Una escuela de surf de Somo llevaba las reservas en un cuaderno y se solapaban las clases."
2. **La solución** (20 s): qué hace la aplicación, en dos frases.
3. **El stack y por qué** (20 s): las tecnologías y una razón para las dos decisiones principales.
4. **La parte difícil** (45 s): el problema técnico concreto que más te costó y cómo lo resolviste. Aquí es donde te evalúan de verdad.
5. **Qué harías distinto** (20 s): una o dos cosas, con criterio.

34. Proyectos 2 y 3

34.1 Proyecto 2 — El diferenciador

El primero demuestra que sabes construir. El segundo demuestra que tienes intereses propios y que sabes moverte fuera del camino trillado. Debe tener **una** cosa técnicamente interesante que el primero no tiene:

Ingrediente	Ejemplo concreto
Tiempo real	Tablero colaborativo o chat con WebSockets, con reconexión y estado compartido
Procesamiento de datos	Importar un CSV grande, limpiarlo, agregarlo y visualizarlo
Integración externa	Consumir una API con OAuth (Spotify, GitHub, Strava) y hacer algo que esa API no ofrece
Trabajos en segundo plano	Cola de tareas: generar informes, enviar correos programados, sincronizar datos
Datos geográficos	Mapas, rutas, consultas por proximidad con PostGIS
Scraping y automatización	Recoger información pública periódicamente y avisar de cambios
IA aplicada	Búsqueda semántica sobre tus propios documentos con embeddings

> **USA TU PROPIO CONTEXTO**
>
> Un proyecto que nace de algo que conoces de primera mano tiene detalles que un tutorial no tiene, y eso se nota al contarlo. Si te interesa la música, un gestor de sets, sesiones y eventos con integración de plataformas de audio es un proyecto con reglas de negocio reales, datos propios y una historia que contar. Si te interesa la criminología, un visualizador de datos públicos de criminalidad con filtros geográficos y temporales tiene exactamente lo mismo. Un entrevistador recuerda mucho mejor a quien construyó algo que le importaba.

34.2 Proyecto 3 — Contribuir a código abierto

Es el menos habitual en un portfolio junior y uno de los que más dice de ti, porque demuestra algo que un proyecto propio no puede demostrar: que sabes trabajar **dentro del código de otros**, siguiendo sus normas y aceptando revisión. Que es, literalmente, lo que harás en tu empleo.

Cómo empezar sin frustrarse:

1. Elige un proyecto que **uses de verdad** y que no sea gigantesco. Una librería mediana es mejor que React.
2. Lee su `CONTRIBUTING.md` entero antes de tocar nada.
3. Busca incidencias etiquetadas `good first issue` o `help wanted`.
4. Empieza por lo pequeño: documentación confusa, un mensaje de error poco claro, una prueba que falta. Son contribuciones reales y se aceptan rápido.
5. Comenta en la incidencia antes de ponerte, para no duplicar trabajo.
6. Un pull request enfocado, con pruebas y una descripción clara de qué cambia y por qué.

7. Acepta la revisión sin ponerte a la defensiva. Aprender a recibir comentarios sobre tu código es media carrera.

Tres o cuatro contribuciones aceptadas en un proyecto conocido son una carta de presentación difícil de superar, y además son *verificables*: cualquiera puede ver el pull request, la conversación y el código.

34.3 Tu página de portfolio

Una sola página, sencilla y rápida. Lo que debe contener:

- Quién eres en dos frases: qué buscas y qué sabes hacer.
- Los tres proyectos, cada uno con captura, una frase de problema, el stack y dos enlaces: demo y repositorio.
- Enlaces a GitHub, LinkedIn y correo.
- CV descargable en PDF.

Lo que no debe contener: animaciones de entrada que retrasan la lectura, una barra de "nivel de habilidad" con porcentajes inventados, una sección de "mis hobbies" antes de los proyectos, ni un formulario de contacto que no funciona. Quien la mira dedica entre 30 y 60 segundos: que todo lo importante esté visible en ese tiempo.

EJERCICIOS · PARTE 8

1. Escribe hoy el `ALCANCE.md` de tu Proyecto 1: problema, usuarios, entidades, funcionalidades incluidas y —sobre todo— funcionalidades explícitamente excluidas.
2. Dibuja el modelo de datos en papel antes de escribir una línea de código.
3. Escribe el README completo *antes* de programar. Te obliga a decidir qué hace el proyecto y detecta ambigüedades cuando aún son baratas.
4. Graba tu explicación de dos minutos y escúchala. Duele, y funciona.

PARTE 9

Cobrar por ello: empleo junior en España

La parte que convierte el conocimiento en un sueldo. Con el mercado junior estrechado, la estrategia de búsqueda importa tanto como el nivel técnico.

- 35. El CV y LinkedIn
- 36. Tu GitHub como carta de presentación
- 37. Dónde buscar y cómo presentarse
- 38. El proceso de selección, fase por fase
- 39. Preguntas de entrevista con respuestas modelo
- 40. Salario y negociación
- 41. Los primeros noventa días

35. El CV y LinkedIn

35.1 Cómo se lee realmente un CV

Un reclutador dedica entre 15 y 30 segundos a la primera criba. Busca cuatro cosas: si las tecnologías encajan con la oferta, si hay algo demostrable, si el formato es legible y si hay señales de alarma. Todo lo demás sobra.

Además, muchas empresas medianas y grandes usan sistemas automáticos de seguimiento de candidaturas que extraen texto del PDF. Consecuencia práctica: nada de CV en dos columnas con gráficos, ni tablas complejas, ni texto dentro de imágenes. Un PDF de una página, estructura lineal y palabras clave literales de la oferta.

35.2 Estructura recomendada

```
ÁLVARO CRESPO
Desarrollador Full-Stack · JavaScript / TypeScript
Santander, Cantabria (disponible para remoto) · correo · GitHub · LinkedIn

PERFIL
Desarrollador full-stack con formación autodidacta y tres proyectos propios
desplegados en producción. Experiencia práctica en React, TypeScript, Node
y PostgreSQL, con pruebas automatizadas y despliegue continuo. Estudiante de
Criminología (UNED), con formación en análisis de datos y método.

TECNOLOGÍAS
Lenguajes     JavaScript (ES2023), TypeScript, SQL, HTML, CSS
Frontend      React, React Router, TanStack Query, Tailwind CSS, Vite
Backend       Node.js, Express, Prisma, PostgreSQL, Zod
Calidad       Vitest, Testing Library, Playwright, ESLint
Herramientas  Git, GitHub Actions, Docker, Vercel, Railway, Linux

PROYECTOS

Reservas Surf · reservas-surf.vercel.app · github.com/usuario/reservas
Aplicación de gestión de reservas para una escuela de surf, en uso real.
Detección de solapamientos horarios, gestión de aforo y panel de
administración con dos roles.
React + TypeScript · Node + Express · PostgreSQL + Prisma · 78 % de cobertura
de pruebas · CI en GitHub Actions · desplegado en Vercel y Railway.

[Proyecto 2 con el mismo formato]

CÓDIGO ABIERTO
3 pull requests aceptados en [proyecto]: corrección de un fallo en el
parser de fechas y mejora de la documentación de la API.

FORMACIÓN
Grado en Criminología · UNED · 3.er curso (en progreso)
Formación autodidacta en desarrollo web, 2026 — aprox. 600 horas

IDIOMAS
Español nativo · Inglés B2 (lectura técnica fluida)
```

35.3 Reglas

Haz	No hagas
Una página. Siempre, sin experiencia	Dos o tres páginas de relleno
Enlaces reales y que funcionen	Enlaces rotos o a repositorios vacíos
Resultados concretos: "78 % cobertura", "en uso por 40 personas"	"Apasionado de la tecnología", "proactivo", "trabajo en equipo"
Adaptar las palabras clave a cada oferta	Enviar el mismo PDF a cien ofertas
Mencionar la experiencia no técnica si aporta	Listar quince tecnologías que tocaste una tarde
PDF con nombre profesional: `alvaro-crespo-fullstack.pdf`	`cv_definitivo_v3_FINAL.pdf`
Foto opcional, sobria si la pones	Barras de porcentaje de "nivel" en cada tecnología

> **TU PERFIL NO TÉCNICO ES UN ACTIVO, NO UN PROBLEMA**
>
> Venir de otro campo se puede presentar como debilidad o como diferencia. Una formación en criminología aporta método analítico, rigor con las fuentes, manejo de datos y comprensión de marcos normativos: todo eso es directamente aplicable en cumplimiento normativo, protección de datos, fintech, seguridad o sector público, que en España contratan mucho. Cualquier trayectoria previa con trato con clientes, gestión de proyectos propios o disciplina de trabajo autónomo se traduce igual. No lo escondas: enmárcalo.

35.4 LinkedIn

- **Titular:** lo más importante, es lo que se ve en cada búsqueda. "Desarrollador Full-Stack | React · TypeScript · Node · PostgreSQL | Buscando primera oportunidad" funciona mejor que "Estudiante apasionado por la tecnología".
- **Foto:** profesional, cara visible, fondo neutro.
- **Extracto:** tres párrafos cortos: qué haces, qué has construido (con enlaces), qué buscas.
- **Sección de proyectos:** con enlaces y capturas.
- **"Open to work":** actívalo, al menos en la versión visible solo para reclutadores.
- **Actividad:** publicar el progreso de tus proyectos una o dos veces al mes genera más contactos de reclutadores que enviar CVs a ciegas. No hace falta ser un creador de contenido: basta con ser visible.

36. Tu GitHub como carta de presentación

Cuando un desarrollador va a decidir si te entrevista, mira tu GitHub. No cuenta el número de repositorios: cuenta lo que se ve al abrir el primero.

Elemento	Qué hacer
Perfil	Foto, nombre real, ubicación, enlace al portfolio, biografía de una línea
README de perfil	Un repositorio con tu nombre de usuario: presentación breve y tus tres proyectos destacados
Repositorios fijados	Fija exactamente los tres buenos. El resto, a archivar o a privado
README de cada proyecto	Como el modelo del capítulo 32. Es lo que más se lee
Historial de commits	Mensajes descriptivos, commits pequeños, ramas por funcionalidad
Insignias	Estado de CI y cobertura, si son reales
Licencia	MIT en tus proyectos públicos
Issues y PRs	Usarlos en tus propios proyectos demuestra método

REVISA ESTO ANTES DE ENVIAR TU PRIMERA CANDIDATURA

- Ningún fichero `.env` ni clave de API en el historial (revisa también commits antiguos: borrarlo hoy no lo quita del historial).
- Ningún repositorio llamado "prueba", "test2" o "curso-dia-4" visible.
- Ningún README que siga diciendo "Getting Started with Create React App".
- Todos los enlaces de demo funcionando. Un enlace roto en el portfolio es peor que no tenerlo.

37. Dónde buscar y cómo presentarse

37.1 Canales, por rentabilidad real

Canal	Comentario	Eficacia
Contactos y recomendaciones	Amigos, antiguos compañeros, gente de comunidades técnicas. Una recomendación interna salta la primera criba entera	La más alta, con diferencia
LinkedIn	Ofertas y, sobre todo, reclutadores que te escriben si tu perfil está bien montado	Alta
Tecnoempleo	El portal IT de referencia en España. Unas 3.000 ofertas activas	Alta
InfoJobs	Mucho volumen, calidad variable, buena presencia de consultoras	Media-alta para empezar
Web de las empresas	Haz una lista de 30 empresas que te interesen y revisa sus páginas de empleo	Media, pero alta calidad
Comunidades y Discord	Comunidades técnicas en español, grupos locales, eventos	Media, con buen retorno a medio plazo
Portales remotos	Para trabajar desde Cantabria para fuera. Exige inglés y suele pedir algo de experiencia	Baja al principio, alta después

37.2 Prácticas y formación dual: la puerta lateral

Con el mercado junior estrechado, conviene no descartar las vías de entrada no convencionales, que además tienen mucha menos competencia:

- **Prácticas no laborales** (convenio con universidad o centro de FP). Tu condición de estudiante universitario activo te habilita para muchas.
- **FP Dual de Desarrollo de Aplicaciones Web (DAW)**: combina estudio y empresa, y una parte alta de los alumnos se queda en la empresa. Es una de las vías más eficaces que existen hoy en España para un primer empleo técnico.
- **Becas de grandes empresas**: consultoras y tecnológicas tienen programas de captación con procesos menos exigentes.
- **Trabajo para una pyme local**: pequeños negocios que necesitan una web o una herramienta interna. Se paga poco pero es experiencia real y facturable.

37.3 Cómo presentarse

La candidatura ciega con CV genérico tiene una tasa de respuesta muy baja. Lo que funciona, en orden:

1. **Averigua quién es el responsable técnico** del equipo y escríbele directamente en LinkedIn.
2. **Mensaje corto y específico.** Tres o cuatro frases, no una carta.

3. **Demuestra que has mirado la empresa.** Una frase concreta sobre qué hacen.
4. **Enlace directo al proyecto más relevante** para lo que ellos hacen.
5. **Haz seguimiento una vez** a los diez días. Una sola vez.

```
Asunto: Candidatura desarrollador junior full-stack

Hola [nombre]:

He visto la oferta de desarrollador junior en [empresa] y me encaja
mucho, porque vuestro stack (React, Node y PostgreSQL) es exactamente
con el que he construido mis dos últimos proyectos.

El más parecido a lo que hacéis es un gestor de reservas que está en
uso real en una escuela de surf: React con TypeScript, API en Express,
PostgreSQL con Prisma, pruebas con Vitest y despliegue automático.
Está aquí: [enlace a la demo] y el código aquí: [enlace].

Vengo de una formación en Criminología y me he pasado al desarrollo
por la parte de análisis y datos. Busco un primer equipo donde
aprender con gente con más experiencia.

Te dejo el CV adjunto. Gracias por tu tiempo.

Álvaro
```

37.4 Estrategia de volumen y expectativas realistas

Con el mercado actual, los números que conviene tener en la cabeza para no desanimarse:

Etapa	Proporción típica para un junior sin experiencia
Candidaturas enviadas	100
Respuestas	10–20
Primera entrevista	5–10
Prueba técnica	3–6
Entrevista final	1–3
Oferta	1

Esto no es un fracaso: es la estadística normal del proceso. Planifica 15–20 candidaturas *bien hechas* por semana, lleva un registro de cada una (un buen uso para el Proyecto 1) y no interpretes el silencio como un juicio sobre tu valía. La mayoría de las no respuestas no tienen nada que ver contigo.

38. El proceso de selección, fase por fase

38.1 Las cinco fases habituales

Fase	Quién	Qué evalúa	Cómo prepararla
1. Criba telefónica	Reclutador, 20–30 min	Encaje básico, disponibilidad, expectativa salarial, comunicación	Tener preparada tu presentación de dos minutos y una horquilla salarial
2. Prueba técnica	Sola, 3–8 h	Cómo escribes código de verdad	Ver más abajo
3. Entrevista técnica	Desarrollador o responsable, 45–90 min	Fundamentos, tus proyectos, cómo piensas	Capítulo 39
4. Entrevista cultural	Responsable o RR. HH.	Cómo trabajas en equipo, cómo reaccionas ante problemas	Método STAR, más abajo
5. Oferta	RR. HH.	—	Capítulo 40

38.2 La prueba técnica: cómo destacar

Casi todos los candidatos entregan algo que funciona. Lo que diferencia:

1. **Lee el enunciado dos veces** y pregunta lo que esté ambiguo. Preguntar puntúa; asumir, no.
2. **Un README** con cómo arrancarlo, qué decisiones tomaste y qué dejaste fuera por tiempo.
3. **Pruebas.** Aunque sean tres. Es lo que más recuerdan los evaluadores.
4. **Commits con historia.** No entregues un único commit llamado "solución": que se vea cómo trabajaste.
5. **Gestión de errores y casos límite.** Entrada vacía, datos inválidos, fallo de red.
6. **No te pases de alcance.** Si piden un CRUD, no entregues una plataforma. Hacer exactamente lo pedido, bien, es la señal correcta.
7. **Entrega a tiempo.** Si no llegas, entrega lo que tengas y explica en el README qué falta y cómo lo harías.

> **PRUEBAS TÉCNICAS ABUSIVAS**
>
> Una prueba razonable ocupa entre tres y seis horas. Si te piden construir un sistema completo, o una funcionalidad que huele a trabajo real sin pagar, es legítimo declinar o proponer una alternativa ("puedo enseñaros un proyecto equivalente que ya tengo hecho, o hacer una sesión de código en directo de una hora"). Una empresa que reacciona mal a eso te está dando información valiosa sobre cómo trata a su gente.

38.3 La entrevista en vivo

Si te ponen a programar mientras te observan, lo que evalúan es el proceso, no la velocidad:

- **Piensa en voz alta.** El silencio no se puede evaluar. Verbaliza tu razonamiento.

- **Pregunta antes de codificar.** "¿Puede haber duplicados?" "¿Qué pasa si la lista está vacía?" "¿Importa el rendimiento con un millón de elementos?"
- **Empieza por la solución sencilla** y di que sabes que se puede optimizar. Luego optimiza si da tiempo.
- **Escribe casos de prueba** antes o después. Recorre tu solución con un ejemplo concreto.
- **Si te atascas, dilo** y explica qué has descartado. Pedir una pista está bien visto; quedarse en blanco en silencio, no.

38.4 Preguntas de comportamiento: el método STAR

Para "cuéntame una vez que…", responde con esta estructura y evita divagar:

- **S**ituación: el contexto, en una frase.
- **T**area: cuál era tu responsabilidad concreta.
- **A**cción: qué hiciste *tú*, en detalle.
- **R**esultado: qué pasó, y qué aprendiste.

Ten preparadas tres historias que puedas adaptar a casi cualquier pregunta: una de un problema técnico difícil, una de un error que cometiste y cómo lo arreglaste, y una de colaboración o conflicto. Sin experiencia laboral, sirven perfectamente situaciones de tus proyectos, de tus estudios o de cualquier trabajo anterior.

38.5 Las preguntas que TÚ debes hacer

No preguntar nada se interpreta como desinterés. Además, esto es una entrevista en las dos direcciones y hay empresas a las que no te conviene entrar:

- ¿Cómo es un día normal en el equipo?
- ¿Cómo acompañáis a alguien que entra sin experiencia? ¿Hay alguien asignado?
- ¿Cómo es el proceso de revisión de código?
- ¿Qué cobertura de pruebas tenéis y qué parte del tiempo se dedica a calidad?
- ¿Con qué frecuencia desplegáis?
- ¿Cuánta deuda técnica hay y cómo la gestionáis?
- ¿Cómo se mide el buen desempeño en este puesto?
- ¿Por qué está vacante esta posición?

> **SEÑALES DE ALARMA EN UNA ENTREVISTA**
>
> Que no sepan responder cómo revisan el código; que hablen de "ambiente familiar" y "estamos muy comprometidos" como valor principal; que la rotación sea altísima; que el proceso tenga seis fases para un puesto junior; que no te digan el rango salarial al preguntarlo; que normalicen las horas extra. Todo eso es información que estás recibiendo gratis.

39. Preguntas de entrevista con respuestas modelo

Estas son las preguntas que más se repiten en procesos junior de perfil web en España. Las respuestas están resumidas: la idea es que las conviertas en tus propias palabras y las ensayes en voz alta.

39.1 JavaScript

Pregunta	Respuesta en corto
Diferencia entre `var`, `let` y `const`	`var` tiene ámbito de función y se eleva; `let` y `const` tienen ámbito de bloque; `const` no se puede reasignar, pero su contenido sí puede mutar.
`==` frente a `===`	`==` convierte tipos antes de comparar; `===` compara valor y tipo. Se usa siempre `===`.
Qué es un closure	Una función que mantiene acceso al ámbito donde fue creada aunque ese ámbito ya haya terminado. Ejemplo: un contador privado, o un debounce.
Qué es el bucle de eventos	El mecanismo que permite que JavaScript, siendo de un solo hilo, no se bloquee: ejecuta la pila, luego las microtareas (promesas) y luego las macrotareas (timeouts).
`map`, `filter` y `reduce`	Transformar cada elemento, quedarse con algunos, reducir a un único valor. Ninguno muta el original.
Promesa frente a async/await	Es la misma cosa: async/await es sintaxis sobre promesas que permite escribir código asíncrono de forma secuencial y usar try/catch.
Copia superficial y profunda	El spread copia el primer nivel; los objetos anidados siguen compartiéndose. Para copia profunda, `structuredClone`.
Qué hace `this`	Depende de cómo se llama la función. Las funciones flecha no tienen `this` propio: heredan el del contexto de definición.

39.2 React

Pregunta	Respuesta en corto
Estado frente a props	Las props vienen de fuera y son de solo lectura; el estado es interno y su cambio provoca un nuevo render.
Por qué hacen falta las keys	Para que React identifique cada elemento entre renders. Sin claves estables, el estado y el foco se asignan al elemento equivocado al reordenar.
Para qué es el array de dependencias	Indica cuándo debe volver a ejecutarse el efecto. Vacío, solo al montar.
Por qué no se muta el estado	React compara referencias para decidir si repinta; mutar no cambia la referencia y el componente no se actualiza.

Cuándo Context	Para datos que necesita medio árbol (usuario, tema, idioma). No como sustituto de un gestor de estado ni para datos que cambian muy a menudo.
`useMemo` y `useCallback`	Memorizar cálculos caros y referencias estables. Solo cuando hay un problema medido: por defecto, sobran.

39.3 Backend, bases de datos y web

Pregunta	Respuesta en corto
Qué pasa al escribir una URL	DNS → TCP → TLS → petición HTTP → el servidor procesa → respuesta → el navegador construye el DOM, aplica estilos y pinta.
GET frente a POST	GET obtiene y no modifica, es idempotente y cacheable; POST crea y no es idempotente.
401 frente a 403	401: no sé quién eres. 403: sé quién eres y no puedes.
Qué es un middleware	Función que se ejecuta entre la petición y la respuesta y puede modificarlas, cortarlas o pasarlas al siguiente.
INNER JOIN frente a LEFT JOIN	INNER solo devuelve las filas que casan en ambas tablas; LEFT devuelve todas las de la izquierda, con nulos si no hay pareja.
Qué es un índice y cuándo crearlo	Estructura que acelera las búsquedas. En columnas usadas en WHERE, JOIN u ORDER BY. Cuesta espacio y ralentiza las escrituras.
Qué es el problema N+1	Una consulta para la lista y una por cada elemento. Se resuelve con JOIN o carga anticipada.
Qué es una transacción	Conjunto de operaciones que ocurre entero o nada. Propiedades ACID.
Cómo guardas contraseñas	Con hash lento y con sal (bcrypt o argon2). Nunca en claro ni cifradas de forma reversible.
Cómo evitas la inyección SQL	Consultas parametrizadas o un ORM. Nunca concatenando entrada del usuario.
SQL frente a NoSQL	Relacional para datos estructurados con relaciones e integridad; documental para esquemas flexibles. Por defecto, relacional.

39.4 Las preguntas abiertas

Pregunta	Qué buscan y cómo enfocarla
Háblame de ti	No tu biografía: dos minutos sobre qué sabes hacer, qué has construido y qué buscas. Termina conectando con esa empresa.
¿Por qué el desarrollo de software?	Una razón concreta y verdadera, con un ejemplo. Evita "me gusta la tecnología desde pequeño".
Cuéntame un problema técnico difícil	La pregunta más importante. Usa STAR, céntrate en el *proceso* de diagnóstico, no en el resultado.

¿Cuál es tu mayor debilidad?	Una real, con lo que estás haciendo al respecto. "Soy perfeccionista" no cuela.
¿Cómo te mantienes actualizado?	Fuentes concretas que sigas de verdad, y algo que hayas aprendido recientemente por tu cuenta.
¿Dónde te ves en cinco años?	Honestidad con ambición razonable: ser autónomo técnicamente, profundizar en un área. No hace falta inventarse un plan de carrera.
¿Por qué deberíamos contratarte a ti?	Tres razones: lo que ya sabes hacer demostrable, tu velocidad de aprendizaje con pruebas, y lo que aportas de distinto.
Cuéntame un error que cometiste	Uno real, con impacto, y qué cambiaste para que no se repita. Responder "ninguno" es la peor respuesta posible.

QUÉ HACER CUANDO NO SABES ALGO

Va a pasar, y es normal en un junior. Lo que **no** se hace es inventar: un entrevistador técnico lo detecta en dos preguntas y pierde la confianza en todo lo demás que has dicho. Lo que sí funciona: *"no lo he usado, pero por el nombre entiendo que va de… ¿va por ahí?"*, o directamente *"no lo sé; es algo que me apunto para mirar"*. Reconocer los límites con naturalidad es una señal de madurez, y muchos entrevistadores lo puntúan explícitamente.

40. Salario y negociación

40.1 Qué pedir

Los rangos del capítulo 2, aplicados a un primer empleo:

Situación	Rango realista
Junior sin experiencia, consultora	18.000 – 24.000 €
Junior sin experiencia, empresa de producto	22.000 – 30.000 €
Junior con prácticas o un año, producto	26.000 – 32.000 €
Remoto para empresa extranjera	Por encima, pero difícil sin experiencia previa

40.2 La pregunta de la expectativa salarial

Te la harán pronto, a veces en la primera llamada. Tres reglas:

1. **Intenta que hablen ellos primero:** "Me gustaría conocer el rango que tenéis presupuestado para la posición, para ver si estamos alineados."
2. **Si insisten, da un rango**, no una cifra: "En torno a 24.000–28.000, aunque para un primer puesto me importa mucho el acompañamiento técnico y la tecnología."
3. **No mientas** sobre lo que cobras ahora ni sobre otras ofertas. Es fácil de comprobar y destruye la relación.

40.3 Negociar la primera oferta

Se puede negociar siendo junior, con medida. Lo que funciona:

- **Agradece y pide tiempo.** "Me alegra mucho, ¿puedo darte una respuesta en un par de días?" Nadie retira una oferta por eso.
- **Un solo intento, concreto y justificado.** "¿Habría margen para llegar a X? Vengo con los proyectos desplegados y experiencia práctica en vuestro stack exacto."
- **Negocia también lo que no es dinero:** días de teletrabajo, presupuesto de formación, horario flexible, revisión salarial a los seis meses por escrito. Esto último es especialmente útil en un primer empleo, donde el margen en la cifra inicial suele ser pequeño.
- **Pide la oferta por escrito** antes de rechazar otras opciones.

40.4 Lo que hay que mirar además del bruto

Concepto	Por qué importa
Número de pagas	24.000 € en 12 pagas frente a 14 cambia la nómina mensual

Convenio aplicable	Determina vacaciones, jornada y categorías. El de consultoras (TIC) es distinto del de una empresa de producto
Teletrabajo	Desde Cantabria, con Madrid y Barcelona concentrando las ofertas, es probablemente lo más valioso de todo
Formación	Presupuesto real y horas dedicadas, no "formación continua" en el anuncio
Quién te va a acompañar	En un primer empleo, tener un senior que revise tu código vale más que 3.000 € de diferencia
Tecnología	Entrar en un proyecto con tecnología de hace quince años puede costarte años de carrera

LA PERSPECTIVA CORRECTA SOBRE EL PRIMER SUELDO

El primer salario es el menos importante de tu carrera. Lo que decide tus ingresos a cinco años es cuánto aprendes en los dos primeros. Entre una oferta de 26.000 € en un equipo donde nadie va a revisar tu código y una de 22.000 € con un buen senior al lado, procesos de revisión y pruebas automatizadas, la segunda es financieramente mejor decisión aunque no lo parezca en la nómina de enero.

41. Los primeros noventa días

41.1 Qué se espera realmente de ti

Nadie espera que un junior sea productivo el primer mes. Lo que se evalúa es otra cosa, y conviene saberlo para no medirte con la vara equivocada:

Periodo	Expectativa real
Semanas 1–2	Montar el entorno, entender el producto, conocer al equipo, hacer tu primer cambio pequeño en producción
Semanas 3–6	Resolver tareas pequeñas bien definidas con ayuda. Empezar a moverte solo por el código
Semanas 7–12	Coger tareas medianas, necesitar menos supervisión, empezar a detectar problemas por tu cuenta

41.2 Reglas prácticas

- **Pregunta pronto, pero prepara la pregunta.** La norma habitual: atáscate 30–45 minutos, documenta qué has probado, y entonces pregunta. "Estoy intentando X, he probado A y B, creo que el problema es C, ¿voy bien?" es una pregunta que a nadie molesta.
- **Toma notas de todo.** Nombres, siglas, sistemas, decisiones. Preguntar dos veces lo mismo cansa; la tercera, preocupa.
- **Lee código antes de escribirlo.** Dedica tus primeras horas a leer el repositorio y a entender las convenciones del equipo.
- **Acepta las revisiones sin defenderte.** Los comentarios son sobre el código, no sobre ti. Pregunta el porqué cuando no lo entiendas: es la forma más rápida de aprender que existe.
- **Avisa pronto cuando vayas mal.** Un retraso comunicado el martes es un problema de planificación; el mismo retraso comunicado el viernes por la tarde es un problema de confianza.
- **No propongas reescribirlo todo.** Ese código raro que has visto suele tener una historia. Pregunta por qué está así antes de proponer cambiarlo.
- **Documenta lo que te costó averiguar.** Es la forma más fácil de aportar valor desde el primer mes, y la que más agradecen.

41.3 Y después

Pasado el primer año, lo que hace crecer a alguien es bastante predecible: coger las tareas que dan un poco de miedo, pedir revisar el código de otros, aprender el dominio del negocio y no solo la tecnología, y escribir lo que aprendes. El salto de junior a mid es sobre todo un salto de autonomía, y la autonomía se construye asumiendo problemas cuya solución no conoces de antemano.

UNA NOTA SOBRE EL PERIODO DE PRUEBA

En el sector tecnológico español suele ser de tres a seis meses según el convenio y la categoría. Funciona en ambos sentidos: si a las pocas semanas ves que la empresa no es lo que te vendieron —no hay acompañamiento, no hay revisiones, la presión es insostenible— también tú puedes irte sin drama. Un primer empleo malo no te define, pero quedarte dos años en uno donde no aprendes sí te cuesta caro.

APÉNDICES

Material de consulta

Lo que vas a abrir una y otra vez mientras trabajas.

- A. Plan de estudio de 12 semanas, día a día
- B. Chuletas de comandos
- C. Glosario
- D. Recursos recomendados
- E. Listas de comprobación
- F. Fuentes

A. Plan de estudio de 12 semanas

Calculado para 15–20 horas semanales. Si dispones de más, comprime; si dispones de menos, alarga sin cambiar el orden. Lo que no se puede hacer es saltarse bloques.

Semana 1 — El mapa y las herramientas

Día	Estudio (1 h)	Práctica (2 h)
Lunes	Parte 0 completa	Instalar Node, VS Code, Git, WSL si usas Windows
Martes	Cap. 5 — terminal	Los cuatro ejercicios del capítulo 5
Miércoles	Cap. 6 — Git, secciones 1 a 4	Primer repositorio con 10 commits limpios
Jueves	Cap. 6 — Git, secciones 5 a 7	Provocar y resolver un conflicto; practicar deshacer
Viernes	Cap. 7 — cómo funciona la web	Analizar la pestaña Red de tres webs distintas
Sábado	Cap. 8 — entorno	Configurar ESLint, Prettier y formateo al guardar
Domingo	Repaso	Explicar en voz alta y sin apuntes qué pasa al escribir una URL

Semanas 2–3 — El lenguaje

Bloque	Contenido	Entregable
S2 L-M	Cap. 9 — fundamentos	20 ejercicios pequeños resueltos
S2 X-J	Cap. 10 — funciones y closures	memoizar, once, componer, throttle
S2 V-D	Cap. 11 — objetos y arrays	Programa que analiza un conjunto de datos real (CSV descargado)
S3 L-M	Cap. 12 — asincronía	Cliente de una API pública con reintentos y timeout
S3 X	Cap. 13 — módulos y errores	Refactorizar lo anterior en módulos
S3 J-D	Cap. 14 — TypeScript	Convertir todo lo escrito hasta ahora a TypeScript estricto

Semanas 4–5 — Frontend

Bloque	Contenido	Entregable
S4 L-M	Cap. 15 — HTML y accesibilidad	Página semántica navegable solo con teclado
S4 X-V	Cap. 16 — CSS, Flexbox y Grid	Maquetar una página compleja responsive sin librerías
S4 S-D	Cap. 16 — Tailwind	Rehacer lo anterior con Tailwind

S5 L-X	Cap. 17 — React	Lista de tareas completa con filtros y persistencia
S5 J-D	Cap. 18 — React real	Aplicación con rutas que consume una API pública, con los cuatro estados

Semanas 6–7 — Backend y datos

Bloque	Contenido	Entregable
S6 L-M	Cap. 19 — Express	API CRUD en memoria con middleware y validación
S6 X-D	Cap. 20 — SQL	Esquema de tienda + 10 consultas con JOIN y agregación
S7 L-M	Cap. 21 — Prisma	Migrar la API a PostgreSQL con Prisma
S7 X-J	Cap. 22 — seguridad	Registro, login con JWT y comprobación de propiedad de recursos
S7 V-D	Cap. 23 — diseño de API	Paginación, filtros, formato de error coherente, README de la API

Semana 8 — Proyecto 1, primera mitad

Definir alcance por escrito, modelo de datos, esquema de Prisma, API completa con pruebas de integración, autenticación funcionando. Al acabar la semana la API debe estar terminada y probada.

Semana 9 — Calidad

Bloque	Contenido	Entregable
L-M	Cap. 24 — pruebas	Cobertura de la lógica de negocio del proyecto
X-J	Cap. 25 — código limpio	Refactorizar el proyecto: nombres, funciones, duplicación
V-D	Cap. 26–27 — arquitectura y fundamentos	Separar en capas; resolver 10 problemas de algoritmos

Semana 10 — Proyecto 1, segunda mitad y producción

Frontend completo del proyecto, Docker Compose, CI en GitHub Actions, despliegue en Vercel y Railway, README completo. **Al acabar la semana 10 debes tener un enlace público que funcione.**

Semana 11 — Proyecto 2 y la IA

Construir el proyecto diferenciador con alcance deliberadamente pequeño (una sola cosa interesante, bien hecha). Leer el capítulo 31 y aplicarlo al flujo de trabajo.

Semana 12 — Empleabilidad

Día	Tarea
Lunes	CV en una página. Revisión de GitHub completa (capítulo 36)

Martes	LinkedIn: titular, extracto, proyectos, "open to work"
Miércoles	Página de portfolio desplegada
Jueves	Preparar y grabar la presentación de dos minutos y las tres historias STAR
Viernes	Ensayar en voz alta todas las preguntas del capítulo 39
Sábado	Lista de 30 empresas objetivo. Primeras 10 candidaturas personalizadas
Domingo	Montar el registro de candidaturas y planificar el ritmo semanal

B. Chuletas de comandos

B.1 Terminal

```
pwd                          # dónde estoy
ls -la                       # listar con detalle
cd ruta / cd .. / cd ~ / cd -
mkdir -p a/b/c               # crear árbol de carpetas
cp -r origen destino         # copiar recursivo
mv viejo nuevo               # mover o renombrar
rm -r carpeta                # borrar (sin papelera)
cat / head -n 20 / tail -f   # ver ficheros
grep -rn "texto" src/        # buscar en el código
find . -name "*.js"          # buscar ficheros
wc -l fichero                # contar líneas
chmod +x script.sh           # hacer ejecutable
ps aux | grep node           # ver procesos
kill -9 PID                  # matar proceso
lsof -i :3000                # qué ocupa el puerto 3000
df -h / du -sh *             # espacio en disco
curl -i URL                  # petición HTTP con cabeceras
history | grep git           # buscar en el historial de comandos
```

B.2 Git

```
git init / git clone URL
git status                           # úsalo constantemente
git add . / git add -p               # -p permite elegir trozos
git commit -m "feat: descripción"
git log --oneline --graph --all
git diff / git diff --staged

git switch -c feat/nombre            # crear rama y cambiar
git switch main
git merge rama
git branch -d rama

git push -u origin rama
git pull
git fetch --all

git restore fichero                  # descartar cambios
git restore --staged fichero         # sacar del área de preparación
git commit --amend                   # corregir el último commit
git reset --soft HEAD~1              # deshacer commit, conservar cambios
git revert <hash>                    # deshacer de forma segura
git stash / git stash pop            # guardar temporalmente
git reflog                           # el historial de todo: tu red de seguridad
git blame fichero                    # quién escribió cada línea y cuándo
```

B.3 npm

```
npm init -y
npm install paquete                  # dependencia
npm install -D paquete               # dependencia de desarrollo
npm uninstall paquete
npm ci                               # instalación limpia desde el lockfile
npm run script
npm outdated / npm update
npm audit / npm audit fix
npx paquete                          # ejecutar sin instalar
npm ls paquete                       # por qué está instalado esto
```

B.4 Docker

```
docker build -t nombre:tag .
docker run -p 3000:3000 --env-file .env nombre:tag
docker ps / docker ps -a
docker logs -f <id>
docker exec -it <id> sh
docker stop <id> / docker rm <id>
docker images / docker rmi <id>
docker system prune -a               # limpiar todo lo no usado

docker compose up -d
docker compose logs -f servicio
docker compose down / down -v
docker compose exec db psql -U dev miapp
```

B.5 SQL

```
SELECT col1, col2 FROM tabla WHERE cond ORDER BY col DESC LIMIT 10 OFFSET 20;
SELECT * FROM t WHERE c BETWEEN a AND b;
SELECT * FROM t WHERE texto ILIKE '%buscar%';
SELECT * FROM t WHERE c IN (1, 2, 3);
SELECT * FROM t WHERE fecha >= NOW() - INTERVAL '30 days';

INSERT INTO t (a, b) VALUES (1, 2) RETURNING id;
UPDATE t SET a = 1 WHERE id = 5;
DELETE FROM t WHERE id = 5;

SELECT a.*, b.nombre FROM a INNER JOIN b ON b.id = a.b_id;
SELECT a.nombre, COUNT(b.id) FROM a LEFT JOIN b ON b.a_id = a.id GROUP BY a.id;

SELECT categoria, COUNT(*), SUM(total), AVG(total)
FROM pedidos GROUP BY categoria HAVING COUNT(*) > 5;

CREATE INDEX idx_nombre ON tabla(columna);
EXPLAIN ANALYZE SELECT ...;
BEGIN; ... COMMIT;  /  ROLLBACK;

-- psql
\l        listar bases de datos      \dt   listar tablas
\d tabla  describir tabla            \q    salir
```

C. Glosario

Término	Significado
API	Interfaz que expone un programa para que otros programas lo usen
Backend	La parte que se ejecuta en el servidor
Boilerplate	Código repetitivo y estándar que hay que escribir sí o sí
Build	Proceso de convertir el código fuente en lo que se despliega
Bug	Fallo en el software
Caché	Copia temporal de datos para no recalcular o volver a pedir
CI/CD	Integración y despliegue continuos: automatizar pruebas y publicación
CORS	Mecanismo del navegador que controla peticiones entre dominios distintos
CRUD	Crear, leer, actualizar y borrar: las cuatro operaciones básicas
Deploy	Publicar la aplicación en un entorno accesible
Deuda técnica	Coste futuro de decisiones rápidas tomadas hoy
DOM	Representación en memoria de la estructura de una página web
Endpoint	Una ruta concreta de una API
Framework	Estructura que impone una forma de construir; tú rellenas los huecos
Frontend	La parte que se ejecuta en el navegador del usuario
Hash	Función que convierte datos en una huella de longitud fija, irreversible
Hotfix	Corrección urgente desplegada fuera del ciclo normal
Idempotente	Que repetir la operación produce el mismo resultado que hacerla una vez
Librería	Conjunto de funciones que tú llamas cuando quieres
Linter	Herramienta que detecta errores probables y desviaciones de estilo
Merge	Integrar los cambios de una rama en otra
Middleware	Función que se ejecuta entre la petición y la respuesta
Mock	Sustituto falso de una dependencia, usado en pruebas
Monorepo	Varios proyectos en un mismo repositorio

ORM	Capa que traduce entre objetos del lenguaje y tablas de base de datos
Pull request	Propuesta de cambio abierta a revisión antes de integrarse
Refactorizar	Cambiar la estructura del código sin cambiar su comportamiento
Regresión	Algo que funcionaba y ha dejado de funcionar
REST	Estilo de diseño de APIs basado en recursos y verbos HTTP
Rollback	Volver a la versión anterior tras un despliegue fallido
Runtime	Entorno donde se ejecuta el código (Node, el navegador)
Sprint	Ciclo de trabajo de duración fija, normalmente dos semanas
Stack	Conjunto de tecnologías usadas en un proyecto
Staging	Entorno de pruebas que imita producción
Token	Credencial que acredita identidad o permisos
Webhook	Petición que un sistema externo envía a tu servidor cuando ocurre algo

D. Recursos recomendados

D.1 Documentación (la fuente primaria, siempre)

- **MDN Web Docs** — la referencia definitiva de JavaScript, HTML, CSS y APIs del navegador. Disponible en español.
- **react.dev** — la documentación oficial de React, con tutoriales excelentes.
- **typescriptlang.org/docs** — el manual y el *playground* para experimentar.
- **postgresql.org/docs** — densa pero exhaustiva.
- **nodejs.org/docs** y **expressjs.com**
- **prisma.io/docs**, **vitest.dev**, **playwright.dev**, **docs.docker.com**

> **EL HÁBITO MÁS RENTABLE**
>
> Acostúmbrate a ir a la documentación oficial antes que a un blog, un vídeo o un asistente de IA. Es más lenta al principio y mucho más rápida a partir del segundo mes, porque está actualizada, es completa y te enseña a leer especificaciones: una habilidad que se transfiere a cualquier tecnología que aprendas después.

D.2 Práctica

- **freeCodeCamp** — currículo completo y gratuito, con certificaciones.
- **The Odin Project** — el mejor plan de estudio gratuito de desarrollo web que existe, orientado a proyectos.
- **Frontend Mentor** — diseños reales para maquetar; ideal para practicar CSS.
- **Exercism** — ejercicios con mentoría humana gratuita.
- **Codewars** y **LeetCode** — algoritmos; con el nivel fácil e intermedio basta.
- **SQLBolt** y **PostgreSQL Exercises** — SQL interactivo desde cero.

D.3 Libros

- *Eloquent JavaScript*, Marijn Haverbeke — gratuito en línea, el mejor para fundamentos.
- *You Don't Know JS*, Kyle Simpson — gratuito, profundiza en las partes difíciles del lenguaje.
- *El programador pragmático*, Hunt y Thomas — sobre el oficio, no sobre un lenguaje. Muy recomendable.
- *Refactoring*, Martin Fowler — el catálogo de referencia.
- *Designing Data-Intensive Applications*, Martin Kleppmann — para más adelante, cuando te interesen los sistemas.

D.4 En español

- **midu.dev** — contenido de desarrollo web en español, muy al día.
- **Fazt**, **HolaMundo** — canales con cursos completos y gratuitos.
- **campusMVP** — artículos técnicos y análisis del mercado laboral español.

- **Comunidades de Discord y Telegram** de desarrollo en español: útiles para resolver dudas y para enterarte de ofertas.

E. Listas de comprobación

E.1 Antes de subir código

- ☐ Las pruebas pasan en local
- ☐ El linter no da errores
- ☐ No queda ningún `console.log` de depuración
- ☐ No hay secretos, claves ni datos personales
- ☐ Los mensajes de commit describen el cambio
- ☐ He revisado mi propio *diff* antes de pedir revisión
- ☐ El cambio hace una sola cosa

E.2 Antes de dar por terminada una funcionalidad

- ☐ Caso feliz probado
- ☐ Casos límite: vacío, nulo, cero, negativo, muy largo, duplicado
- ☐ Errores gestionados y con mensaje útil para el usuario
- ☐ Estados de interfaz: cargando, vacío, error, éxito
- ☐ Validación en servidor, no solo en cliente
- ☐ Autorización comprobada: ¿puede este usuario acceder a este recurso?
- ☐ Funciona en móvil
- ☐ Navegable con teclado

E.3 Portfolio listo para enviar candidaturas

- ☐ Tres proyectos terminados y desplegados, con enlaces que funcionan
- ☐ README completo en cada uno, con decisiones técnicas
- ☐ Pruebas y CI visibles al menos en el proyecto principal
- ☐ GitHub limpio: repositorios de prueba archivados, perfil completo
- ☐ Ningún secreto en ningún historial de commits
- ☐ CV de una página, en PDF, con enlaces activos

☐	LinkedIn con titular claro y "open to work" activado
☐	Presentación de dos minutos ensayada en voz alta
☐	Tres historias STAR preparadas
☐	Horquilla salarial decidida de antemano

F. Fuentes

Los datos de mercado y salarios de la Parte 0 y del capítulo 40 proceden de las siguientes fuentes, consultadas en septiembre de 2026. Los rangos salariales y los volúmenes de oferta cambian; conviene reconsultarlos antes de negociar.

- Tecnoempleo — *Informe de empleo en informática*, septiembre de 2026: volumen de ofertas por tecnología, distribución geográfica, tipos de contrato y años de experiencia requeridos. tecnoempleo.com/informe-empleo-informatica.php
- DigitalES — *Radiografía de Empleos y Sectores Emergentes*, 2026: caída del 33 % en ofertas junior y aumento del 13 % en sénior; concepto de "junior aumentado".
- Universidad de Stanford — *AI Index Report 2026*: descenso cercano al 20 % en el empleo de desarrolladores de 22 a 25 años en Estados Unidos desde 2024.
- *Guía salarial 2026 de desarrolladores web en España*: rangos por perfil y nivel de experiencia. cursosdesarrolloweb.es/blog/guia-salarial-2026-desarrolladores-web
- campusMVP — *Evolución de los puestos de trabajo tecnológicos en España (2022–2026)*: tendencias de perfiles y tecnologías.
- MuyComputerPro — *La IA lleva a un nuevo perfil en busca del primer empleo: el junior aumentado*, septiembre de 2026.

> **UNA ÚLTIMA COSA**
>
> Este manual tiene unas ciento cuarenta páginas y no sirve absolutamente de nada si solo se lee. El conocimiento técnico se consolida escribiendo código que no funciona, averiguando por qué, y volviéndolo a intentar. Esa frustración no es una señal de que no vales: es literalmente el proceso.
>
> Empieza hoy por la semana 1 del apéndice A. Dentro de doce semanas tendrás algo que enseñar.